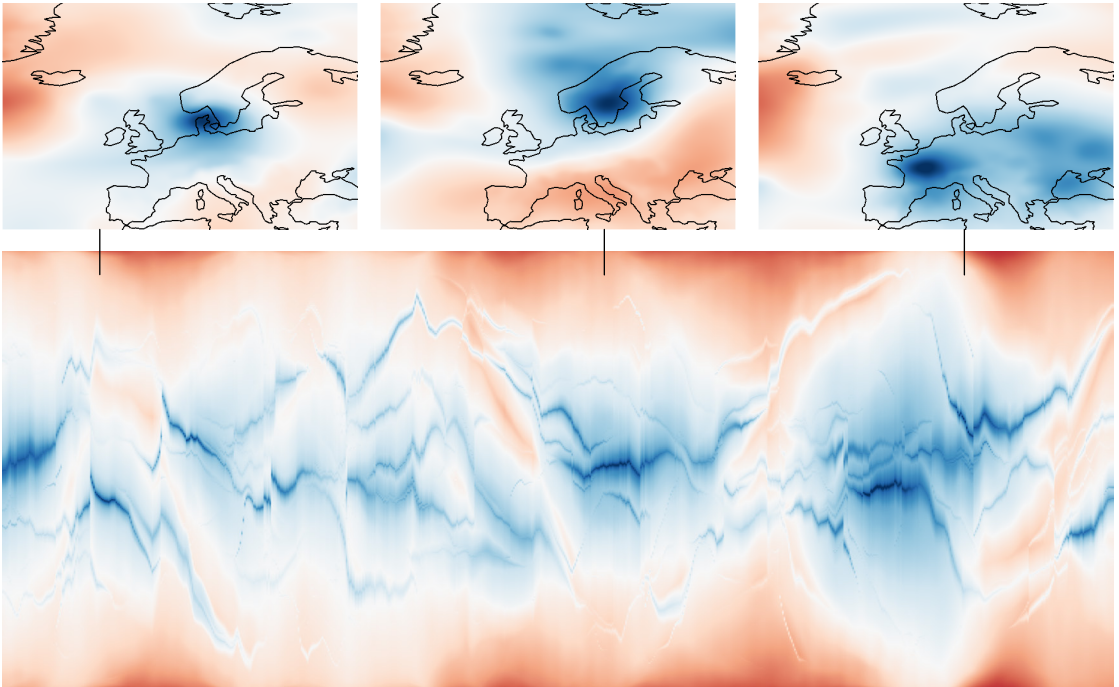


Doctoral Thesis in Computer Science

Static Visualizations for Dynamic Hierarchies

WIEBKE KÖPP



Static Visualizations for Dynamic Hierarchies

WIEBKE KÖPP

Academic Dissertation which, with due permission of the KTH Royal Institute of Technology, is submitted for public defence for the Degree of Doctor of Philosophy on Wednesday the 7th December 2022 at 10:00 CET in the Visualization Studio VIC, Lindstedtsvägen 5, Stockholm, Sweden.

Doctoral Thesis in Computer Science with specialization in
High-Performance Computing and Visualization
KTH Royal Institute of Technology
Stockholm, Sweden 2022

© Wiebke Köpp
© Tino Weinkauff, Paper A, Paper B, Paper C, Paper D
© Anke Friederici, Paper B, Paper C
© Marco Atzori, Paper B, Paper C
© Ricardo Vinuesa, Paper B, Paper C
© Philipp Schlatter, Paper B, Paper C

ISBN 978-91-8040-419-8
TRITA-EECS-AVL 2022:74

Printed by: Universitetsservice US-AB, Sweden 2022

Cover illustration: A *Temporal Merge Tree Map* (Paper D) and several snapshots representing European storm activity in December 1999. Colors indicate a positive (red) and a negative (blue) deviation from a multi-day average of mean sea level pressure. The dark blue areas correspond to cyclones or low-pressure systems and constitute leaves in the underlying temporal tree.

Abstract

Dynamic hierarchical data describes phenomena in a wide range of domains, from file management to demographics and business administration, as well as feature extraction results in spatial scientific data. As with all data, visualization is an integral step for gaining understanding about dynamic hierarchical data. In contrast to the visualization of individual static hierarchies, the visualization of dynamic hierarchies comes with additional challenges since many different aspects of a hierarchy may be subject to change.

This thesis presents novel visualizations, compact data structures, and layout optimizations for dynamic hierarchies where both topology and data may change. Particular focus is placed on the type of hierarchies that stem from features in scalar fields, namely merge trees and derived discretized feature hierarchies. We propose several visualization schemes that summarize dynamic hierarchies *statically* by creating stacked one-dimensional representations. The stacking dimension corresponds to the data's dynamic-inducing variable which is usually time. In contrast to animating individual visualizations for each variable setting or time step, our static overviews of the entirety of the data facilitate comparison both between multiple data sets and across the stacking dimension. For the preservation of a user's mental map, we utilize correspondences between parts of the hierarchy to optimize the one-dimensional representations toward stability in regards to the dynamic-inducing dimension. To evaluate our proposed methods, we apply them to several real-world data sets, compare them against existing approaches, and study how the resulting visualizations are affected by method parameters.

Keywords

Dynamic hierarchies, Static visualization, Pixel-based visualization, Scalar data, Topology, Layout stability

Sammanfattning

Dynamisk hierarkisk data beskriver fenomen inom ett brett spektrum av domäner, från filhantering till demografi, företagsadministration och som ett resultat av särdragsextrahering ur rumslig vetenskapliga data. Som för all data är visualisering ett viktigt steg för att kunna förstå dynamisk hierarkisk data. Jämfört med visualisering av enskilda statiska hierarkier medför dynamiska hierarkier ytterligare utmaningar eftersom många olika aspekter av en hierarki kan förändras.

Denna avhandling presenterar nya visualiseringar, kompakta datastrukturer och layout-optimeringar för dynamiska hierarkier där både topologi och data förändras. Avhandlingen har särskilt fokus på den typen av hierarkier som härrör från särdrag i skalära fält, d.v.s., merge trees och härledda diskretiserade särdragshierarkier. Vi föreslår flera visualiseringsscheman som sammanfattar dessa och allmänna dynamiska hierarkier *statiskt* genom att skapa staplade endimensionella representationer. Staplingsdimensionen motsvarar den variabeln som inducerar dynamiken av datan, vilket vanligtvis är tid. Istället för att animera individuella visualiseringar för varje variabelinställning eller tidssteg, underlättar våra översikterna av hela datan jämförelser mellan flera datamängder och över staplingsdimensionen. För att bevara en användares mentala karta använder vi korrespondenser mellan delar av hierarkin för att optimera de endimensionella representationerna mot stabilitet gentemot denna dimension. För att utvärdera våra föreslagna metoder tillämpar vi dem på flera verkliga datamängder, jämför dem med befintliga metoder och studerar hur de resulterande visualiseringarna påverkas av olika metodparametrar.

Nyckelord

Dynamiska hierarkier, Statisk visualisering, Pixelbaserad visualisering, Skalärdata, Topologi, Layoutstabilitet

Zusammenfassung

Dynamische hierarchische Daten beschreiben Phänomene in einer Vielzahl von Anwendungsbereichen, zum Beispiel in Dateisystemen, der Demografie und Betriebswirtschaft, sowie in Merkmalen und Gebieten in wissenschaftlichen räumlichen Daten. Wie bei allen Daten ist auch bei dynamischen hierarchischen Daten die Visualisierung ein wesentlicher Schritt zum Erlangen von Verständnis der Daten. Die Visualisierung dynamischer Hierarchien bringt jedoch im Gegensatz zur Visualisierung einzelner statischer Hierarchien zusätzliche Herausforderungen mit sich, da sich viele verschiedene unterschiedliche Aspekte einer Hierarchie verändern können.

Diese Arbeit umfasst neuartige Visualisierungen, kompakte Datenstrukturen und Layoutoptimierungen für dynamische Hierarchien, in denen sich sowohl die Topologie der Hierarchie als auch die mit der Hierarchie verbundenen quantitativen Daten ändern können. Dabei liegt besonderer Fokus auf der Art von Hierarchien, die aus Merkmalen in skalaren Feldern aufgebaut werden, insbesondere Merge Trees und abgeleitete diskretisierte Merkmalshierarchien. Wir führen mehrere Visualisierungsschemata ein, welche dynamische Hierarchien *statisch* durch das Stapeln eindimensionaler Darstellungen zusammenfassen. Die Stapeldimension entspricht hierbei der dynamisierenden Dimension der Daten. Dies ist in der Regel Zeit, es sind aber auch veränderte Parametereinstellungen eine mögliche Quelle von Veränderungen in Hierarchien. Im Gegensatz zu animierten einzelnen Visualisierungen für jede Parametereinstellung oder jeden Zeitschritt erleichtern unsere statischen Übersichten über die Gesamtheit der Daten den Vergleich sowohl zwischen mehreren Datensätzen als auch im Bezug auf die dynamisierende Dimension. Damit die mentale Landkarte eines Benutzers erhalten werden kann, verwenden wir Korrespondenzen zwischen Teilen der Hierarchie. Durch sie werden die eindimensionalen Darstellungen in Bezug auf die dynamische Dimension in Richtung Stabilität optimiert. Um unsere vorgeschlagenen Methoden zu evaluieren, wenden wir sie auf mehrere reale Datensätze an, vergleichen sie mit bestehenden Ansätzen und untersuchen den Einfluss von Methodenparametern auf die resultierenden Visualisierungen.

Schlüsselwörter

Dynamische Hierarchien, Statische Visualisierung, Pixel-basierte Visualisierung, Skalare Daten, Topologie, Layoutstabilität

You will always struggle with not feeling productive
until you accept that your own joy can be something you produce.
It is not the only thing you will make, nor should it be,
but it is something valuable and beautiful.

—Carl in *A Beautifully Foolish Endeavor* by Hank Green

Man darf nie an die ganze Straße auf einmal denken,
verstehst du?

—Beppo Straßenkehrer in *Momo* by Michael Ende

Acknowledgments

It is unlikely that I will find the words to truly capture how grateful I am for all the guidance, inspiration, learning opportunities, and support I have received over the years in which I worked on this thesis. But I shall try.

I wish to thank my supervisor Tino Weinkauff for taking a chance on me when I had basically no experience with topological features, their hierarchies, or their visualization. Thank you for introducing me to this multifaceted research area, for letting me run with ideas that went astray, for being open to doing things differently, and for staying calm in especially those situations where I felt like there was no longer a reason to be. I consider myself lucky to have had a supervisor that matches my enthusiasm and joy in creating figures, videos, and documents that are aesthetically pleasing. Thank you for tikzing with me, especially.

I wish to thank Anke Friederici for being a great collaborator, co-author, conference buddy, office mate, and friend, especially long after you have gone on to have other adventures outside of KTH. I treasure our drawing sessions, the initiative to get us to try kickboxing, and especially that time period working together on percolation; full of acronyms, colored paper clippings, elaborate time plans and to-do lists, and cheery commit icons. Sharing not one but two publications with you is an honor.

I express my thanks to everyone who is taking a role in the procedure of my doctoral defense. Thank you to Miriah Meyer, Florian Pokorný, and Hans-Jörg Schulz, for serving on the grading committee, to Erik Fransén, for chairing, and to Sten Ternström, for conducting the advance review. I also thank those who offered valuable advice before, during, and especially in the periods after my progress seminars. Thank you to Johan Hoffman, Pawel Herman, Christopher Peters, Mario Romero, Jörg Conradt, and Stefano Markidis.

Even before this PhD journey, I have been fortunate to have been treated to great mentorship that shaped the way I approach both research and teaching. Thank you to Alexandra Kirsch, Jürgen Richter-Gebert, Martin von Gagern, Sebastian Urban, and Patrick van der Smagt

These years would not have been the same without many collaborators from which I got to learn. Naming just a few: Peter, Martin, and Daniel, thank you for all the Inviwo help and keeping me on my toes with the occasional breaking change. Ollie, thank you for the great code reviews and even better questions. Marco, thank you for patiently explaining aspects of turbulent flow over and over again and for retaining humor when submission systems do not even let us submit our work.

The scientific process lives and breathes through sharing of our work, peer-reviewing, lengthy and detailed discussions at conferences, meetings, and even in emails and Tweets. It all may be an imperfect system, but nevertheless, I am grateful

for the time and effort volunteered by authors dedicated to open science, anonymous reviewers, conference and workshop organizers, and many others who keep the gears of science spinning. Your efforts continuously widen my horizon.

I also want to thank my office mates at 4424, my colleagues at CST and elsewhere at KTH, for being there for me and sharing the ups and downs of PhD life. There are too many to name, but special thanks go to: Himangshu, Gregorio, and Fangkai, for welcoming me so kindly when I first started. Michael, for always cheering me on over Zoom. Jiahui, for bringing laughter to our office. Björn and Ingemar, for practicing Swedish with me and making the studio such a great place. Steven, for answering all my thesis questions. Jens, for your thoughtful view on the world. Everyone working towards improving doctoral education, for caring.

Special thanks also go to some that will never see this thesis or even know what a thesis is. Here it may get a bit weird, but anyways: the spotty piggies, the woolly sheep, the cheeky goats, and the fluffy cats around *Stora Skuggan*. You provided much needed company during these pandemic times and surely you gained some new fans from my endless sharing of pictures and videos.

I am immensely grateful to have friends, close and far, who supported me during this time; although there were some tunnel vision periods, where they did not hear much from me. Thank you to Linus, Janin, Flo, Susanne, Tobi, Sebi, Myri, and Max. Thank you for the reminder that sometimes you just need to play more.

Similarly, I would not have made it to this point without family. I am grateful to: Ben and Marion, for lots of advice. Erhard, for being an inspiration to stay curious, Sanni, for letting me know I make you proud. Anke and Jörg, for all the love, for trusting me to make the right decisions for myself, and especially for sitting with me through the worst of "‘Angst for dem weißen Papier'" for any document that has both the term thesis and my name on its cover. Nils, for being a wonderful hiking companion and for letting your older sister still graduate a little bit before you.

Finally, thank you, Sjoerd. Without your kindness, your patience, and your many many pep-talks this thesis would not exist. Thank you for being my partner and a role model in so many ways. Thank you for giving me feedback not only on this thesis, but on other texts and images before too, for teaching me how to aggregate results for multiple optimization runs, and for just listening. You brighten up my world and hold me together, always.

Stockholm, November 2022

Wiebke Köpp

Contents

I	Overview and Summary	1
1	Introduction	3
1.1	Contributions	5
1.2	Publications and Division of Work	7
1.3	Reproducibility	10
1.4	Thesis Outline	13
2	Background and State of the Art	15
2.1	Notation and Definitions	15
2.1.1	Trees	15
2.1.2	Dynamic Trees	17
2.1.3	Features in Scalar Fields	20
2.1.4	Feature Hierarchies	22
2.1.5	Augmented Merge Trees	23
2.2	Related Work on Extracting Trees from Scalar Data	25
2.2.1	Feature Hierarchies and Merge Trees	26
2.2.2	Correspondences	30
2.3	Related Work on Visualizing Hierarchies	32
2.3.1	Visualization of a Single Hierarchy	33
2.3.2	Static Visualization of Dynamic Hierarchies	37
2.3.3	Dynamic Visualization of Dynamic Hierarchies	41
2.4	Related Work on Other Static Visualizations for Scalar Data	42
3	Distributed Percolation Analysis	45
3.1	Motivation	47
3.2	Method Overview	49
3.3	Evaluation and Results	52

4	Percolation Analysis on Sampled Scalar Fields	55
4.1	Motivation	56
4.2	Method Overview	57
4.3	Evaluation and Results	59
5	Temporal Treemaps	63
5.1	Motivation	63
5.2	Method Overview	64
5.3	Evaluation and Results	68
6	Temporal Merge Tree Maps	73
6.1	Motivation	73
6.2	Method Overview	74
6.3	Evaluation and Results	77
7	Concluding Remarks	81
7.1	Future Work	82
	References	85
II	Paper Reprints	113
	Temporal Treemaps: Static Visualization of Evolving Trees	115
	Distributed Percolation Analysis for Turbulent Flows	129
	Notes on Percolation Analysis of Sampled Scalar Fields	143
	Temporal Merge Tree Maps: A Topology-Based Static Visualization for Temporal Scalar Data	163

Part I

Overview and Summary

1

Introduction

Hierarchical data is common. Filesystems, specific files with formats such as HTML, XML, JSON, HDF5 and netCDF, geographic entities like countries, and the organizational structure of companies are only a few examples that have explicit hierarchical structure. For data that is not inherently hierarchical at first glance, such as scalar fields, the extraction of relevant substructures and subsequent hierarchical grouping are common steps for further analysis and visualization. A large number of different visualization approaches are available [184] to visualize the structure and associated quantities for such individual hierarchies represented as *trees*¹. This highlights the ubiquitous nature of this data class and its long history in visualization research.

However, most data does not remain static. Filesystems and files change over time, demographic and economic indicators evolve, companies get restructured, and scalar fields are often used to represent intrinsically time-dependent natural phenomena. Collecting data in relation to any of these examples constitutes taking a *snapshot* of the current state, which can be repeated at different points in time. In the case of measured or simulated data, several snapshots can additionally originate from variations in the setup for real-world experiments or in the input parameters for simulations. Regardless of what induced variations to the data, change may affect all parts of a hierarchy, i.e., both any entities within the hierarchy and their associated quantities.

¹ The terms *tree* and *hierarchy* are used interchangeably in this thesis.

Visualization approaches for any type of dynamic data fall into two broad categories. Dynamic data may be visualized either dynamically or statically [3, 9]. A *dynamic visualization* consists of an animated sequence of visualizations for each snapshot often re-utilizing existing visualization methods for the snapshots. Animations cater to human perception of dynamics since the dynamics of the data and the dynamics of the visualization concur. Due to this literal representation of time, dynamic visualizations are also referred to as *time-to-time mappings*. However, not all data analysis tasks are supported by animations. Differences across several data sets or different time intervals, momentary data fluctuations, the number of entities, or the length of time an entity exists can be difficult to perceive. Additionally, there are contexts where dynamic visualizations cannot be used at all, e.g., printed on paper. *Static visualizations* of dynamic data instead display all time steps together. Different time steps are either super- or juxtaposed, or time is represented by a dedicated spatial dimension. These visualizations constitute *time-to-space mappings*. Depending on the data, integrating the temporal dimension into space can cause an increased cognitive load due to too much information being shown at once. Furthermore, aspects of the data apart from time may suffer from being less well represented. In spite of that, static visualizations have the potential to alleviate some of the disadvantages of dynamic visualizations and can also serve as a useful complement to support the navigation of time in an animation. Note that both static and dynamic visualizations can additionally be subject to user interaction. This is an independent aspect that does not influence the classification of a visualization as representing time through time or space.

Irrespective of how time is represented in a visualization, insight into dynamic hierarchies can best be gained from visualizations that capture the dynamic nature of the data but simultaneously stay true to the hierarchical structure of each time step. Aside from high-quality snapshot visualizations, this requires visualizations to be *stable*. This means that changes in the visualization are a direct result and proportional to changes in the data. Otherwise, preservation of the *mental map* of the data is impeded [140]. Intuitively, visual changes *not* originating from changes in the data are at risk to be misinterpreted due to being perceived as data changes. Finding optimal layouts is a computationally challenging task since the number of possible layouts can be factorial or exponential in the number of inputs, i.e., number of entities in the dynamic hierarchy.

In this thesis, we focus on static visualizations of dynamic hierarchies where time is assigned to a spatial dimension, and all other aspects of the data are compressed into the second dimension and color variation. In other words, our aim is to summarize dynamic hierarchies through stacked one-dimensional representations, where the stacking dimension corresponds to the dependent variable of the data inducing dynamics, usually time.

We are especially interested in depicting hierarchies that have been extracted from scalar fields. Entities within the hierarchy associated with a scalar field correspond to features or original data points of the scalar field. Natural phenomena that can be represented by scalar data include climate, fluid flows, chemical reactions, biological processes, and so on. Considering features instead of the full-resolution scalar fields reduces the often enormous amount of data stemming from measurements or simulation by orders of magnitude. Feature-based analysis and visualization are a promising avenue for analysis of large-scale data sets while also contributing to better understanding of the underlying phenomena. Merges, splits, appearances and disappearances are common topological events occurring for features within scalar fields over time. Our visualizations thus need to support a large range of topological changes. Existing visualization approaches have limitations in this aspect both in regards to visualizing the full range of possible changes and in fully accounting for them when computing optimal layouts.

1.1 Contributions

We present several methods that visualize temporally evolving trees of different origins with changing topology and data.

Specifically, we introduce *Temporal Treemaps*, an improved version of the *Nested Tracking Graph* [136] visualization for threshold-based feature hierarchies. Our visualization exhibits a more stable layout and can be computed for general hierarchies. The latter is due to an adapted rendering scheme that emphasizes the hierarchy through a variation of cushion treemaps. We address layout stability by formulation as a graph drawing problem that considers the full temporal extent. The search space of layouts is navigated by a heuristic. Results are optionally further refined with simulated annealing. Fast processing throughout the assembly of the visualization is enabled by our temporally compact data structure.

For our methods to be applicable to data created in large-scale scientific simulations, we need to be able to efficiently compute features and their hierarchies in a distributed and parallel manner. With our novel memory-distributed parallel algorithm for calculating the so-called percolation function, we add to the efforts addressing this issue. The percolation function represents a specific mapping of a feature hierarchy to its one-dimensional value domain. Our algorithm is, to the best of our knowledge, the first to finely sample the percolation function in a distributed setting. It is based on a parallel and distributed version of the union-find algorithm with global synchronization steps per sample. We leverage properties specific to the percolation function to make the algorithm especially efficient for its extraction. In particular, inter-process communication can be avoided in between samples, since a large fraction of operations are local and can be freely reordered. Aside from

exhibiting significant speedup over previous approaches, our algorithm overcomes prior limitations in terms of memory location and overall data size.

For a more detailed exploration of percolation analysis, we conduct several experiments examining how the shape of the percolation function is influenced by different characteristics of a scalar field. Among the explored aspects are the data's value distribution and its degree of structure. We propose a method to automatically extract core indicators from the percolation function such as its critical value and transition width. To facilitate relating these values to known theoretical and simulated results on infinite graphs, we recommend a specific normalization strategy.

Finally, we introduce *Temporal Merge Tree Maps*. This method visualizes a sequence of augmented merge trees and thus the underlying time-dependent scalar data in full. The one-dimensional mapping for each tree is made to retain data values and merge tree, allowing us to show data values and features in context. The latter is not possible with existing methods, such as alternative domain linearizations or feature-based methods. For stability, we introduce a local greedy heuristic operating on consecutive time steps.

From a more generalized point of view, the contributions of this thesis fall into the following areas, which are mapped to the individual included publications in Table 1.1.

Tree extraction and representation: We introduce a compact data structure for dynamic hierarchies that is assembled upon loading or extracting snapshots. It significantly accelerates further processing. For a specific mapping and a specific hierarchy, we propose a novel distributed algorithm making this kind of mapping applicable to large-scale data.

Mapping of the tree data to 1D: We introduce several novel one-dimensional mappings of tree data. Some of these improve or are a variation of established methods. In all presented works, the one-dimensional space is a direct representation of the quantitative data within the hierarchy. For hierarchies that are extracted from scalar fields, entities within the hierarchies are associated with multiple values, and the one dimension may represent either the spatial or the value domain of the original data.

Optimization for stability: The individual mappings for each time step are steered to be stable. For this, we define objectives or *quality metrics* [16, 19] and suggest corresponding optimization procedures.

Evaluation of our methods: All methods are evaluated with various data sets of different size and structure. Where applicable, we also assess the effect of input aside from data on our methods. This includes the analysis of optimization parameters, filtering settings, or variations in the computational setup.

Contribution area	Paper A	Paper B	Paper C	Paper D
Tree extraction and representation	✓	✓	–	–
Mapping of the tree data to 1D	✓	–	✓	✓
Optimization for stability	✓	–	–	✓
Evaluation of the method	✓	✓	✓	✓
Comparisons to other methods	✓	✓	–	✓
Experiments on real-world data	✓	✓	✓	✓

Table 1.1: Mapping of key contributions from each paper to the contribution areas.

Comparison to other methods: Where existing methods for the same type of hierarchy or underlying data are available, we provide direct qualitative and/or quantitative comparisons.

Experiments on real-world data: We showcase applicability of our visualization approaches on real-world and openly available data.

1.2 Publications and Division of Work

This thesis is a compilation thesis that includes four publications, all of which have been published at *peer-reviewed* publication venues. In all cases, the author of this thesis was either the sole leading author (Papers A and D) or among the two leading authors (Papers B and C), involved in all aspects of the work and guided by appropriate supervision. Details for each included publication are given below, and a condensed overview for the respective degree of contribution is given in Table 1.2. Given in chronological order of initial submission, the included publications are:

 **Paper A:** W. Köpp and T. Weinkauff. “Temporal Treemaps: Static Visualization of Evolving Trees.” *IEEE Transactions on Visualization and Computer Graphics* 25.1 (2019): *Proceedings IEEE VIS*, pp. 534–543. doi 10.1109/TVCG.2018.2865265

Division of Work: The methods presented in this paper emerged from discussion between the author and Tino Weinkauff (TW). The author contributed to a majority of the implementation, especially focusing on data structures, optimization, rendering, and interactive exploration. Implementation aspects covered by TW were parts of the extraction of temporal trees from scalar fields and filesystems, and exporting of the optimized results. The author collected the data sets, performed the experiments, and analyzed the results. The manuscript was written in collaboration.

 **Paper B:** A. Friederici*, **W. Köpp***, M. Atzori, R. Vinuesa, P. Schlatter, and T. Weinkauff. “Distributed Percolation Analysis for Turbulent Flows.” *2019 IEEE 9th Symposium on Large Data Analysis and Visualization (LDAV)*. 2019, pp. 42–51.  10.1109/LDAV48142.2019.8944383  received a *Best Paper Honorable Mention Award* (*Both authors contributed equally)

Division of Work: The idea for this paper was developed by the author and Anke Friederici (AF) under the guidance of TW and constitutes an extension of prior work by AF, Marco Atzori (MA), Ricardo Vinuesa (RV), Philipp Schlatter (PS), and TW [69]. AF and the author worked concurrently on implementing the presented methods based on joint discussions. The author focused on synchronization and inter-process communication, and AF focused on data structures and local operations. AF collected the data that served as input for the experiments which were conducted by the author. The manuscript was written collaboratively by AF, the author, and TW and with input from MA, RV, and PS.

 **Paper C:** **W. Köpp***, A. Friederici*, M. Atzori, R. Vinuesa, P. Schlatter, and T. Weinkauff. “Notes on Percolation Analysis of Sampled Scalar Fields.” *Topological Methods in Data Analysis and Visualization VI. Theory, Applications, and Software*. Ed. by I. Hotz, T. B. Masood, F. Sadlo, and J. Tierny. 2021, pp. 39–54.  10.1007/978-3-030-83500-2_3 (*Both authors contributed equally)

Division of Work: This paper emerged from discussions between the author, AF, and TW that were grounded in seeking better understanding and contextualization of the results from Paper B. AF and the author worked simultaneously on implementing the presented methods based on joint discussions, with the author focusing on automatic analysis of results and assembly of the final visualizations and AF focusing on data structures and general data processing. In addition to the data sets from Paper B, further data sets were generated by the author. The experiments were conducted by AF. The manuscript was written collaboratively by AF, the author, and TW and with input from MA, RV, and PS.

 **Paper D:** **W. Köpp** and T. Weinkauff. “Temporal Merge Tree Maps: A Topology-Based Static Visualization for Temporal Scalar Data.” *IEEE Transactions on Visualization and Computer Graphics* 29.1 (2022): *Proceedings IEEE VIS*.  10.1109/TVCG.2022.3209387

Division of Work: The author conceptualized and implemented the presented methods, collected data sets, carried out experiments, and analyzed the results, all under the guidance of TW. The manuscript was written collaboratively with TW.

Contributor role	Paper A	Paper B	Paper C	Paper D
Conceptualization	●	●	●	●
Data curation	●	●	●	●
Formal analysis	●	●	●	●
Investigation	●	●	●	●
Methodology	●	●	●	●
Software	●	●	●	●
Validation	●	●	●	●
Visualization	●	●	●	●
Writing – original draft	●	●	●	●
Writing – review & editing	●	●	●	●

Table 1.2: Contributions of the author to the included publication according to the contributor roles in *CRedit* (Contributor Roles Taxonomy) [23, 146]. The icons indicate the degree of contribution with ● for “lead”, and ● for “equal”, ● for a degree in between these two, and ● for “supporting”. The categories *Funding acquisition*, *Supervision*, *Resources*, and *Project administration* are omitted since these were covered by the supervisor(s).

The author also contributed to the following *peer-reviewed* works outside the scope of this thesis:

-  **W. Köpp**, M. Atzori, M. Rezaei, N. Jansson, R. Vinuesa, E. Laure, P. Schlatter, and T. Weinkauff. *Effects of Blowing and Suction on the Turbulent Flow around an Airfoil*. Presented at the Gallery of Fluid Motion exhibit at the 73rd Annual Meeting of the American Physical Society’s Division of Fluid Dynamics and in adapted form at Visualization Showcase of the ACM Practice & Experience in Advanced Research Computing (PEARC) Conference Series 2021. Nov. 2020.  [10.1103/APS.DFD.2020.GFM.V0058](https://doi.org/10.1103/APS.DFD.2020.GFM.V0058)  received the *Best Visualization Showcase Award @ PEARC*
-  J. Lukasczyk, J. Beran, W. Engelke, M. Falk, A. Friederici, C. Garth, L. Hofmann, I. Hotz, P. Hristov, **W. Köpp**, T. B. Masood, M. Olejniczak, P. Rosen, J.-T. Sohns, T. Weinkauff, K. Werner, and J. Tierny. “Report of the TopoInVis TTK Hackathon: Experiences, Lessons Learned, and Perspectives.” *Topological Methods in Data Analysis and Visualization VI. Theory, Applications, and Software*. Ed. by I. Hotz, T. B. Masood, F. Sadlo, and J. Tierny. 2021, pp. 359–373.  [10.1007/978-3-030-83500-2_18](https://doi.org/10.1007/978-3-030-83500-2_18)
-  M. Atzori, **W. Köpp**, S. W. D. Chien, D. Massaro, F. Mallor, A. Peplinski, M. Rezaei, N. Jansson, S. Markidis, R. Vinuesa, E. Laure, P. Schlatter, and T. Weinkauff. “In-situ visualization of large-scale turbulence simulations in

Nek5000 with ParaView Catalyst.” *Journal of Supercomputing* 78.3 (2022), pp. 3605–3620.  10.1007/s11227-021-03990-3

The research leading to all listed works was conducted while embedded in the visualization community of the *Swedish e-Science Research Centre* (SeRC) and the project *In-Situ Big Data Analysis for Flow and Climate Simulations* (Project BD15-0082) funded by the *Swedish Foundation for Strategic Research* (SSF).

1.3 Reproducibility

Throughout conducting the research presented in this thesis, the author and collaborators have greatly benefited from others openly sharing their research. This has allowed for the replication of experiments, the comparison of runtimes and results, and the overall deepened understanding of the respective methodologies. Especially useful were the algorithms by Franke et al. [68], Gueunet et al. [83], Lukaszczuk et al. [136], and Zhou et al. [250], the data set collections by Günther [84], Klacansky [114], and Weinkauff [231], and the visualization and analysis frameworks *Topology ToolKit* (TTK) by Tierny et al. [208], *ParaView* by Ahrens et al. [2], and *Inviwo* (Interactive Visualization Workshop) by Jönsson et al. [111]. The latter framework is of particular importance. Many of the presented concepts have been implemented as extensions, so-called *modules*, of Inviwo. To ease the reproducibility of our work in a similar manner, details on compute resources, data sets, and software artifacts are listed in the following.

Compute Resources

The following compute resources were used to conduct experiments and produce results:

-  a laptop with a single 4-core 2.6GHz Intel Core i7-6700HQ and 32GB of main memory (used for final visualizations in Papers A to C and local testing for Paper B),
-  a workstation with two 18-core 2.3GHz Intel Xeon E5-2697 v4 processors and 256GB main memory (used for experiments for Papers A, C and D and final visualizations in Paper D), and
-  between one and 64 compute nodes of the supercomputer *Beskow* with two 16-core 2.3GHz Intel Xeon E5-2698v3 Haswell processors and 64GB of main memory each (used for experiments for Paper B). This resource was provided by the *Swedish National Infrastructure for Computing* (SNIC) at the *PDC Centre for High Performance Computing* (PDC-HPC).

These specific setups are given here to contextualize reported timings and memory footprints within each publication. Note that none of the presented methods have a strict requirement for these number of cores or this amount of memory. However, replication of some type of experiments, such as the scaling analysis in Paper B, and some data set sizes, such as the largest data set of Paper D, do require computational setups closer to the latter two listed resources.

Data Sets

The data sets used are available in different modalities () and shared in various ways (). Data sets associated with  are available as trees, while those with the icons  and  refer to two- and three-dimensional scalar fields respectively. All data sets with the icon  have been shared through inclusion in our own repositories. Those with the icon  can be retrieved through external sources. The icon  signifies that the data set either has been generated or simulated by us. When listed together with , the icon  indicates that we have shared additional pre-processing routines for the respective data set. The following list is an extended version of a similar format in Paper D covering data sets for all publications.

Viscous Fingers    The mixing process within a cylindrical water-filled container with infinite salt supply at the top simulated using particle clouds and provided by Geveci and Garth [74]. We use a topologically-simplified and resampled version of the salt concentration part of the *TTK Example Data* [211] and an extracted temporal tree by Lukasczyk et al. [136].

Cylinder   The flow behind a square cylinder obtained from a direct numerical Navier-Stokes simulation by Camarri et al. [35]. A uniformly resampled version of this flow from Funck et al. [71], parts of which are made available by Weinkauff [231], has been used to compute the Okubo-Weiss criterion $Q = 1/2(\|\Omega\|^2 - \|\mathbf{S}\|^2)$.

CPython   Evolution of file sizes within the *CPython* [48] source code repository since the initiation of the project spanning the time between August 1992 and March 2018. The shared temporal tree is based on sampling every 1000th commit in that time period.

World Population   Development of the world population between 1950 and 2100 according to population estimates until 2015 and different population projection models thereafter from data by the United Nations shared in *World Population Prospects: The 2017 Revision* [215, 216].

Europe's Population   Europe's population between 1975 and 2016 adhering to historical events such as German reunification and the latest mid-year population estimates in the *United Nations Demographic Yearbooks* [214] published between 1984 and 2016.

- Duct**    The periodic wall-bounded flow within a square pipe resulting from a direct numerical simulation of the Navier-Stokes equations by Vinuesa et al. [221]. We use an aggregated and normalized version of the flow velocities representing Reynolds-stress due to Atzori et al. [6, 7] of which we have provided one time step to be shared in the *Open Scientific Visualization Datasets* collection [114] and include scripts for aggregation ourselves.
- Isotropic Turbulence**    A wall-free periodic flow with homogeneous isotropic turbulence simulated with a direct numerical simulation by Yeung et al. [248], available in the *Johns Hopkins Turbulence Database* [247].
- Gaussian Random Fields**    We provide scripts to sample from 2D and 3D *Gaussian Random Fields* (GRFs) [129]. GRFs are a stochastic process with the unique property that the values in every finite subset of sample locations have multivariate Gaussian distribution.
- Nucleon**   A slice from the nucleon data set as processed by Zhou et al. [250]. The data set is available in the *Open Scientific Visualization Datasets* collection [114] and is courtesy of SFB 382 of the German Research Council.
- Ring**   Analytical data set used for evaluation by Franke et al. [68] and generated to form a Gaussian bell curve of given peak value and standard deviation along a circle with given center position and radius. All parameters except the center position vary linearly over time.
- Benzene**   The electrostatic field around a benzene molecule was calculated using the fractional charges method described by Stalling and Steinke [197]. The gradient of this field describes the force upon a positive point charge given in a certain location.
- Storms**    The 1-hourly mean sea level pressure anomaly for December 1999. The instantaneous data is obtained from the ERA5 reanalysis data set available at the *Copernicus Climate Change Service (C3S) Climate Data Store* [100]. We subtract the mean over 8 days following the data processing procedure used for detection of cyclones by Deroche et al. [57] and apply a light Gaussian smoothing to the result.
- Tangaroa**   The flow behind a model of the research vessel *Tangaroa* simulated with the Gerris flow solver [162] by Popinet et al. [163]. We use the velocity magnitude of a resampled version available from the visualization data set collection at the ETH Computer Graphics Laboratory [84].
- Random and Analytic Data**    Our repositories additionally include several analytic, random and constructed data sets for testing, illustrative purposes, and showcasing of optimization failure cases. For example, we include data following the structure of a full binary tree, temporal trees with a single merge or split, and random data of uniform and Gaussian distribution.

Software Artifacts

Software artifacts produced over the course of this thesis encompass source code for algorithms, scripts for data acquisition, and workflows for analysis and visualization. They are available in the following repositories:

- 📁/Wiebke/TemporalTreeMaps provides an Inviwo module for the methods presented in Paper A together with Inviwo workspaces that reproduce the figures from the paper and scripts that extract temporal trees from file systems, git repositories, 2D and 3D scalar fields, and user-provided text files.
- 📁/KTHVisualization/percMPI contains a stand-alone C++-based implementation of the methods presented in Paper B along with bash scripts that generate validation and scaling tests in a format that can be submitted to *Beskow's* job scheduler.
- 📁/KTHVisualization/percAnalysis contains a combination of an Inviwo module and Python scripts for the methods presented Paper C. Some of the data representation-specific aspects of this work are incorporated in the *DiscreteData* module of Inviwo's main repository.
- 📁/Wiebke/TemporalMergeTreeMaps provides an Inviwo module for the methods presented in Paper D along with an extension of Inviwo's TTK integration to temporal sequences and Inviwo workspaces that reproduce the figures from the paper. Further included are Python scripts that preprocess data sets and post-process analysis results.

Artifacts based on Inviwo additionally require compilation of the base framework available at 📁/Inviwo/inviwo and may require its non-standard extension modules available at 📁/Inviwo/modules.

1.4 Thesis Outline

This compilation thesis consists of two parts. The first part consists of several chapters that give an overview and context for the second part made up by the included publications. This introductory chapter is followed by a chapter on background and related work. Chapters 3 to 6 each present one of the included papers with specific motivation, key contributions as stated in the respective paper, an overview of the proposed method, and a summary of the evaluation and results. These chapters cover the included publications with order B, C, A, D, building up towards more detailed visualizations. Figures in these chapters are largely reproductions of figures from the respective publications with minor modifications. Chapter 7 concludes and discusses future work.

2

Background and State of the Art

This chapter encompasses relevant notation, definitions, and related work. We first formally define universal trees and then discuss features in scalar fields and how these lead to specialized trees. An overview of related work on visualizing individual and dynamic trees follows. We primarily focus on static visualization methods for dynamic trees, as those are the topic of this thesis. The chapter closes with related work on statically visualizing scalar data without the context of a tree.

2.1 Notation and Definitions

2.1.1 Trees

We define a *tree* $T = (N, E)$ as a data structure that represents hierarchical data with a set of nodes N and a set of directed edges $E = \{(\mathbf{n}_1, \mathbf{n}_2)\} \subset N \times N$. This abstraction allows us to treat diverse data under one unified framework and may also be called a *hierarchy*. We will use both terms interchangeably throughout this thesis. Sometimes, we will need to differentiate edges in a single tree against another set of edges that associates nodes of different trees in time. We then refer to the edges E as the *hierarchical edges*. Each node $\mathbf{n} \in N$ has at most one incoming

edge and the origin of this edge is the node's *parent*. A single node in the tree does not have an incoming edge. This node is called the tree's *root* and it is connected to all other nodes via a unique path each. The endpoints of outgoing edges are called the node's *children*. Nodes that have no outgoing edges are *leaves*. Those that do have outgoing edges are *inner nodes* or *inner parents*. When nodes share a parent, they are called *siblings*. All nodes that have the same distance from the root are part of a *hierarchy layer* or *hierarchy level*. Figure 2.1(a) shows this terminology for an example tree visualized as a *node-link diagram*. In this traditional tree visualization (see Section 2.3.1), nodes take on shapes like circles or rectangles, and edges are represented by lines or curves. Edge directions conventionally point from top to bottom, which is sometimes emphasized by additional arrows. In the context of general graphs, which are also defined by nodes and edges, trees defined as above are *directed acyclic graphs* with a single inter-connected component.

Numerical data values $d(\mathbf{n}) \in \mathbb{R}$ for $\mathbf{n} \in N$ can be attributed either to each individual node or to only the leaves of the tree. In the latter case, values for inner nodes are defined recursively through their children's values. The value at a parent node $\mathbf{p}$ is equal to the sum over the data values of its children $\mathbf{c}_i$:

$$d(\mathbf{p}) = \sum_{\mathbf{c}_i | (\mathbf{p}, \mathbf{c}_i) \in E} d(\mathbf{c}_i). \quad (2.1)$$

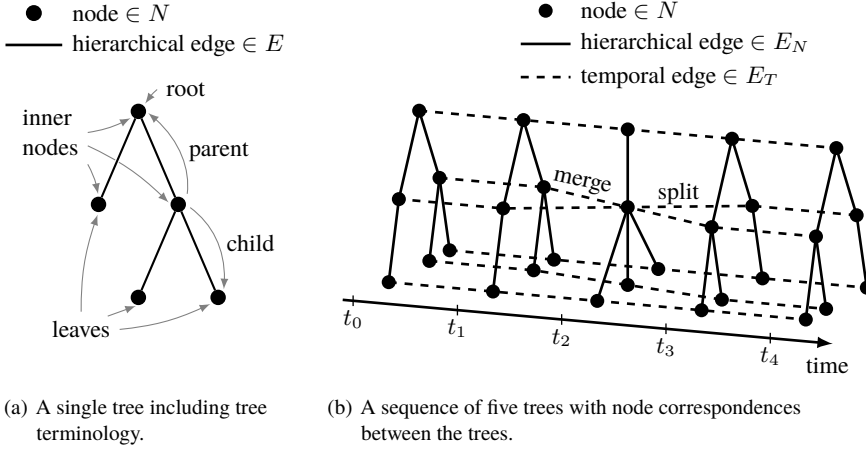
A file system hierarchy falls into this category. Files (leaves) take up a certain amount of memory on a hard drive, while the directories (inner nodes) are mere containers. A directory's size is determined by the sizes of the files and subdirectories it contains. This pattern also occurs in population data, which is often represented hierarchically according to districts, states, countries, and continental regions. Examples for both of these occur in the results of Paper A.

Alternatively, data is associated with all nodes. Here, we consider only trees where there is a relationship between the values of parents and children. In other words, we assume the value at a parent $\mathbf{p}$ can always be expressed as the sum of a function F on the children's values $d(\mathbf{c}_i)$ and an additional value $w_{\mathbf{p}} \in \mathbb{R}$ independent of the children.

$$d(\mathbf{p}) = F(d(\mathbf{c}_i)) + w_{\mathbf{p}} \quad (2.2)$$

The relationship in Equation (2.1) is a special case of this expression with $w_{\mathbf{p}} = 0$ for all parents $\mathbf{p}$ and $F = \sum$.

A common example is a type of tree where $w_{\mathbf{p}} > 0$ for all parents $\mathbf{p}$ and $F = \sum$, meaning the value of each node exceeds the sum of its children's values. This occurs in the threshold-based feature hierarchies (see Section 2.1.4 and Papers A to C). In this type of hierarchy, features are associated with their area or



(a) A single tree including tree terminology.

(b) A sequence of five trees with node correspondences between the trees.

Figure 2.1: Examples for a single tree $T = (N, E)$ with six nodes and a dynamic tree $T = (N, E_N, E_T)$ with individual snapshots. (adapted from Paper A)

volume and their nestedness gives rise to a hierarchy. However, children's features typically do not fully cover their parents' space. Consider an X-ray image of a human hand. Bones and soft tissue show up with varying brightness. A possible feature hierarchy consists of the entire hand as the root feature and the individual bones as child features. Another context with this pattern are organizational hierarchies, for example of universities. Each organizational unit is associated with several employees or budget. Units higher up in the hierarchy, such as schools, departments, and research groups, have additional staff and budgets not allocated to any of their subunits.

In Section 2.1.5 we will introduce two types of augmented merge trees. For one type, the value at each node is always larger than any of its children's values, i.e., $w_p > 0$ and $F = \max$. For the other type, the value is always smaller, i.e., $w_p < 0$ and $F = \min$.

The methods in this thesis are not applicable to trees where values between children and parents are not related at all.¹ An example would be the management hierarchy for a company's staff, with employees' ages as an associated value.

2.1.2 Dynamic Trees

We assume the change to a tree to happen at a set of *discrete* time points $[t_0, \dots, t_{n-1}]$. At these time points, a tree can change in different ways: Node

¹ This means F must be based on all children's values and cannot be a constant function such as $F = 0$.

data may change, tree topology may change, or both. Which parts of the tree are affected impacts how the dynamic tree and its associated data can be represented, both in terms of data structure and visualization. We discuss the data structure aspect in the following and come back to visualization in Section 2.3.

The simplest way that a tree with associated data can change is in terms of data values. Since the tree is otherwise static in terms of its structure, this type of dynamic hierarchy is fully described by a single tree $T = (N, E)$ and the changing data. The latter can be modeled by associating each node $\mathbf{n}$ with a time series $d(\mathbf{n}) = [d_0, \dots, d_{n-1}]$ for n time steps, where d_i is the value at time t_i .

A dynamic tree may also undergo structural changes, causing the topology of the tree to change. Nodes can appear, disappear, merge, split or move. We call these *topological events*. The set of topological events that occur in a given data set depends on the particular application domain. In a file system, all five operations are possible. Employees in a staff hierarchy do not split or merge, but this may happen for organizational units. In hierarchies based on geographical entities of the world (continents, countries, etc.), nodes rarely merge or split and hardly appear, disappear, or move.

In the case of a tree undergoing structural changes, storing a single tree does not suffice. We need to capture both the tree at each time step and information about which nodes correspond to each other. The latter part is necessary to show topological changes or their absence within a visualization. Together with changes in the values of the data, topological events pose the core challenge for creating *stable* or *temporally coherent* visualizations.

We define a dynamic tree $T = ([T_0, \dots, T_{n-1}], E_T)$ through a series of individual trees $T_i = (N_i, E_i)$ and an additional set of edges E_T for node correspondences. Each individual tree T_i is called a *snapshot* and is defined as before. The edges in E_T are called *temporal edges*. Edges in this set may only connect nodes from consecutive time steps t_i and t_{i+1} . This means E_T can be partitioned according to pairs (t_i, t_{i+1}) .

It can be useful to regroup the node and edge sets in other ways. An equivalent definition of a dynamic tree is $T = (N, E_N, E_T)$, where N and E_N are the unions over all nodes N_i and all hierarchical edges E_i of snapshots T_i . This definition more naturally allows for a transition to aggregated trees, as we suggest in Paper A. An example of a dynamic tree composed of five trees with two nodes merging and the resulting node splitting again is given in Figure 2.1(b).

If only appearing and disappearing nodes occur, this is often expressed without the use of snapshots. A single static tree $T = (N, E)$ and information about the times at which each node exists suffices. Several works (see Section 2.3) model this implicitly. Data values in the time series $[d_0, \dots, d_{n-1}]$ are set to zero at time points where the node does not exist.

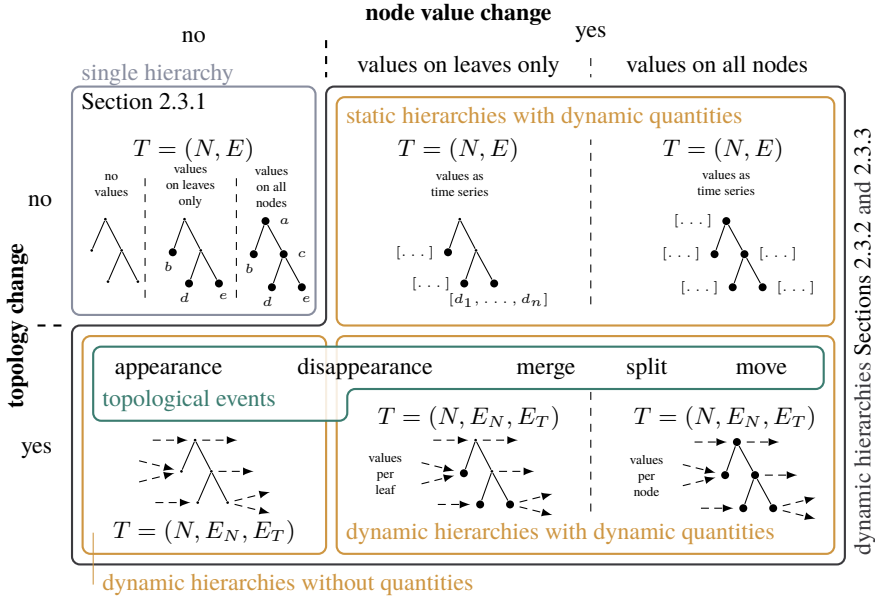


Figure 2.2: Taxonomy of changes that can occur in a hierarchy (adapted from Guerra-Gómez et al. [80]) including tree definitions and references to sections that discuss respective visualization methods.

A Taxonomy of Changes in Dynamic Trees

Together, possible changes over time and the way values are associated with nodes give rise to a taxonomy. It is depicted in Figure 2.2, and we will use it later to contextualize both our own work and related work. This taxonomy was initially presented by Guerra-Gómez et al. [80] for tree comparison problems. We present it here with slight modifications and augment it with the respective tree definitions. In addition to the categories already present in the original version, we further distinguish between the different types of topological changes. The following classes are present:

- the *single hierarchy*, where no change occurs at all,
- the *dynamic hierarchy without quantities*, where the topology changes, but neither nodes nor leaves have associated values,
- the *static hierarchy with dynamic quantities*, where the values change, but the tree structure remains static, and
- the *dynamic hierarchy with dynamic quantities*, where both the topology and the values change.

2.1.3 Features in Scalar Fields

In Sections 2.1.4 and 2.1.5, we will discuss two different trees based on features in scalar fields. Both make use of discrete representations of scalar fields and are based on particular features within them. This section presents these together with general terminology for scalar fields. Expanded overviews are given by Edelsbrunner and Harer [61], Hotz et al. [103], and Wenger [233].

A *scalar field* $s(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}$ is a *continuous* function that assigns a single real value to the d -dimensional space. We refer to the domain of s as the *spatial domain* and to its codomain as the *value domain*. The scalar fields' *gradient* $\nabla s(\mathbf{x})$ is made up of entries describing the rate of change in each orthogonal direction.

In our applications, scalar fields are given in a *sampled* or *discrete* form by a number of samples with neighborhood connectivity between the sample locations and an interpolation scheme. Formally, this constitutes a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with vertices $\mathcal{V}$ at a discrete number of spatial locations $\mathbb{X} \subset \mathbb{R}^d$, undirected edges $\mathcal{E}$ for the connections between neighbors, and data values associated with each vertex. Such a graph $\mathcal{G}$ is also known as *mesh*, and *lattice* or *grid*. The latter two terms additionally imply that the sample locations are regularly spaced. We consider two types of connectivity between neighbors. Neighbors are connected so that adjacent groups of vertices form either rectangular or triangular shapes and their respective higher-dimensional variants. The former can form a *uniform*, *Cartesian*, or *rectilinear grid*, whereas the latter is called a *simplicial complex* or *triangulation*. Interpolation schemes are defined according to the type of connectivity. Typical are piecewise-linear schemes for triangulations and bi- or tri-linear interpolation for rectangular grids in two or three dimensions. While not utilized for hierarchical tree extraction in our work per se, interpolation schemes allow us to approximate the values at non-sample locations and thus recover a continuous version. This may be necessary for certain supplemental context visualizations.

For time-dependent scalar data, the grid often remains static, and only the values vary over time. Thus, a sampled time-dependent scalar field is defined as $s(\mathbf{x}, t_i) : \mathbb{X} \times [t_0, \dots, t_{n-1}] \rightarrow \mathbb{R}$.

As an additional assumption for the scalar fields serving as an input for one of our methods, we expect that no degenerate cases, such as exactly equal neighboring values (plateaus), occur. Scalar fields containing such cases can easily be transformed by adding a small perturbation though. This transformation is called *simulation of simplicity* [63].

We consider a *feature* of a scalar field to be an m -dimensional geometrical object embedded in the d -dimensional domain. In our work, points, areas, or regions are of importance. Other application contexts may also consider line-like features. Intuitively, a feature is some compact space that is of interest because the spatial domain it encompasses shares a property.

Critical Points

Critical points in a scalar field are *minima*, *maxima*, and *saddles*. They constitute 0-dimensional features in a scalar field. We further distinguish between global and local minima and maxima. The *global minimum* is the location where the scalar field assumes its overall smallest value. All other minima are local. The same terminology also applies to maxima, with the largest value being associated with the *global maximum*.

In sampled scalar fields, we can find and determine the type of critical points based on neighboring samples in the graph. At a minimum, all neighbors have a higher value. At a maximum, all neighboring points have a smaller value. Saddle points have both larger and smaller values in their neighborhood, and the neighbors form at least two distinct areas of higher and lower values each.

This is in line with determining critical points for continuous scalar fields, which have in common that their gradient is equal to the zero vector, i.e., $\nabla s(\mathbf{x}) = \mathbf{0}$ and all values within an ϵ -neighborhood follow the rules above.

Level Sets

In our work, region- or area-based features of a scalar field are given by subsets of the domain in relation to a threshold of the value domain. For example, we study regions of high salt concentration (Paper A), high Reynolds stress (Paper C), and low pressure (Paper D).

Formally, we consider a scalar fields' *sublevel set* L_{α}^{-} , *level set* L_{α} , and *superlevel set* L_{α}^{+} . They are defined as the set of sample locations that have function values smaller than, equal to, and larger than a threshold $\alpha \in \mathbb{R}$ respectively:

$$L_{\alpha}^{-} = \{\mathbf{x} \in \mathbb{R}^d | s(\mathbf{x}) \leq \alpha\} \quad (2.3)$$

$$L_{\alpha} = \{\mathbf{x} \in \mathbb{R}^d | s(\mathbf{x}) = \alpha\} \quad (2.4)$$

$$L_{\alpha}^{+} = \{\mathbf{x} \in \mathbb{R}^d | s(\mathbf{x}) \geq \alpha\} \quad (2.5)$$

Any of these sets may be disconnected and consist of several compact regions or areas. Such a compact subspace is called a *connected component*. In our discrete setting, the connectivity of components is determined from grid edges, and a component's size relates to the number of grid vertices it encompasses. All sample locations part of a component are connected by grid edges, whereas none of its sample locations is adjacent to another component. Typically, each connected component is regarded an individual feature. Level sets in two dimensions are also called *isolines*. The term for level sets in three dimensions is *isosurfaces*, and the threshold α is called an *isovalue*.

We show an example scalar field with features based on sublevel sets at several discrete thresholds in Figure 2.3(a). Consider the sublevel set $L_{6.5}^-$ for $\alpha = 6.5$. There are seven vertices with a value below 6.5, i.e., $L_{6.5}^- = \{\textcircled{0}, \textcircled{1}, \textcircled{2}, \textcircled{3}, \textcircled{4}, \textcircled{5}, \textcircled{6}\}$. This sublevel set has two connected components. The first connected component C_1 encompasses $\textcircled{0}, \textcircled{1}, \textcircled{2}$, and $\textcircled{4}$, and the second component C_2 encompasses $\textcircled{3}, \textcircled{5}$, and $\textcircled{6}$. None of the vertices in C_1 shares edges with any of the vertices in C_2 and vice versa.

2.1.4 Feature Hierarchies

Sublevel sets and superlevel sets at distinct values α_i have the property that both sets and their connected components are nested. Consider two thresholds α, β with $\alpha > \beta$. Then the sublevel set for α contains the entire sublevel set for β , i.e., $L_\beta^- \subseteq L_\alpha^-$. Increasing the threshold can only increase the number of vertices in the sublevel set. Additionally, any connected component of L_α^- will contain one or more connected components of L_β^- in their entirety. In Figure 2.3, we can see this for the thresholds 6.5 and 2.5. $L_{2.5}^-$ consists of the vertices $\textcircled{0}$ – $\textcircled{2}$ and is fully contained by C_1 of $L_{6.5}^-$. A similar relation holds for superlevel sets.

This nesting relationship allows us to define a *feature hierarchy* $T = (N, E)$ based on sublevel and superlevel sets. The connected components of the sublevel or superlevel sets based on several thresholds α_i make up the set of nodes N . The nesting relationships between the connected components define the edge set E . Essentially, each threshold creates a layer within the hierarchy. The associated data value $d(\mathbf{n})$ for each node $\mathbf{n} \in N$ is the size of the respective connected component. As mentioned in Section 2.1.1, this implies that the data values in the hierarchy follow Equation (2.2) with $F = \sum$. An example of a feature hierarchy based on the connected components of sublevel sets is given in Figure 2.3(b). It is based on the example scalar field in Figure 2.3(a) and four thresholds $\alpha = 2.5, 6.5, 12.5, 19.5$. We can see that several of the sublevel sets have multiple connected components. The largest sublevel set $L_{19.5}^-$ comprises a single component that includes the vertices $\textcircled{0}$ – $\textcircled{19}$. Of these, the vertices $\textcircled{0}$ – $\textcircled{12}$ are already part of the sublevel set $L_{12.5}^-$, which consists of two components. This means that $L_{19.5}^-$ gives rise to the root node with value $|L_{19.5}^-| = 20$ and has two children.

Dynamic Feature Hierarchies

In the case of a time-dependent scalar field $s(\mathbf{x}, t_i)$ we additionally need a way to define the temporal edges E_T , i.e., correspondences between the connected components of consecutive time steps. We determine this edge set separately for each threshold or hierarchy layer. Several different methods to determine

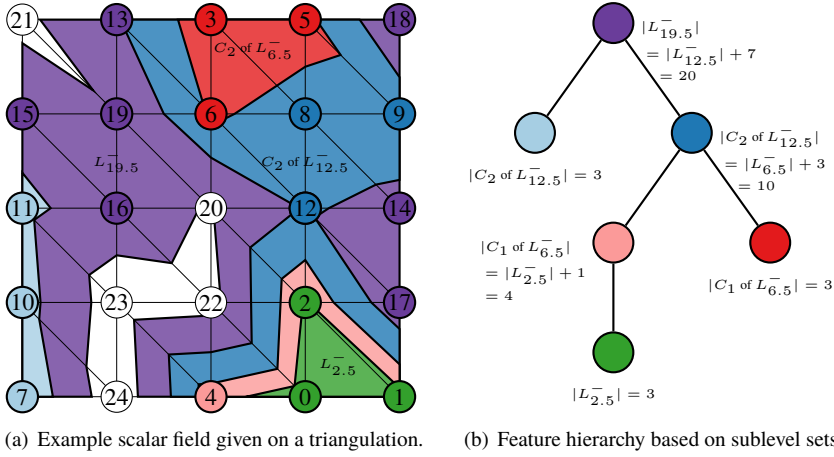


Figure 2.3: An example scalar field with a corresponding feature hierarchy derived from the connected components of the sublevel sets at $\alpha = 2.5, 6.5, 12.5, 19.5$. Vertices in the triangulation and nodes in the feature hierarchy are marked according to their sublevel set. The values associated with the nodes are equal to the number of grid vertices in the respective connected component.

feature correspondences have been introduced, and we discuss them in more detail in Section 2.2.2. However, a simple way to determine these is spatial overlap. Assuming connected components A in time step t_i and B in time step t_{i+1} , we add an edge in E_T if $A \cap B$ is non-empty. Determining edges per threshold means that only appearance, disappearance, merge, and split are possible topological changes.

In the context of tracked features, the literature often uses alternative terms for these topological events: Appearance is also called *birth* or *creation*. The terms *death* or *dissipation* are used for disappearance, A merge is also called an *amalgamation* and a split is also referred to as a *bifurcation*.

2.1.5 Augmented Merge Trees

One way to represent the topology of a scalar field is with a *merge tree*. The *merge tree* is a data structure that records how the connectivity of the different components within the sublevel or superlevel sets evolves under a changing isovalue α .

The change in the connectivity of connected components can only occur at sample locations $\mathbf{x}$ that are the critical points of the scalar field. Consider the case of sublevel sets of a scalar field. The sublevel set for a value α smaller than the global minimum is empty. As the isovalue increases, additional sample locations $\mathbf{x}$

become part of a new or already existing connected component in the sublevel set. If $\mathbf{x}$ with the value α induces a connectivity change depends on it being a critical point and thus the neighborhood of $\mathbf{x}$:

- If $\mathbf{x}$ corresponds to a local minimum, it gives rise to a new connected component. All its neighbors are of larger value and not part of the sublevel set L_{α}^{-} . $\mathbf{x}$ thus does not connect to any existing component in L_{α}^{-} .
- If $\mathbf{x}$ is a saddle point, it has several neighbors of smaller value that are part of different connected components. The saddle point $\mathbf{x}$ causes these to merge.
- If $\mathbf{x}$ is neither a minimum nor a saddle, all its smaller neighbors are part of the same component. $\mathbf{x}$ simply enlarges or extends this already existing component.

A value α corresponding to the global maximum leads to the sublevel set encompassing the entire domain with only a single component.

Recording these connectivity changes in a data structure leads to a *merge tree*. For sublevels sets, the specific type of merge tree is called a *join tree*. Its leaf nodes correspond to minima, the global maximum is the root, and all saddle points are inner parents with two children. These nodes representing critical points of the scalar field are referred to as the *supernodes* of the join tree. They are connected to each other via so-called *superarcs*, which represent the connected components of the sublevel sets. Data samples that do not cause a topological change may be included as *regular nodes*. They are assigned to the superarc corresponding to the connected component they caused to grow, and they become simple parents with just one child. When including regular nodes in the tree data structure, it is called an *augmented join tree*. The superarcs then constitute direct paths between the supernodes rather than the edges within the unaugmented version. An example for an augmented join tree is shown in Figure 2.4.

Following the same concepts considering superlevel sets, results in a *split tree*² for a decreasing isovalue. Join and split tree can also be combined into the *contour tree*, which encodes changes in the connectivity of level set components. More details on merge and contour trees are given by Carr [37]. Algorithmic considerations follow in the next sections.

Describing an augmented merge tree as a tree $T = (N, E)$ means that the nodes N are given by the union of all supernodes and all regular nodes, while the edges E can be seen as a representation of those edges of the underlying grid that are responsible for inclusion in a connected component. The associated data values are the original scalar values and follow Equation (2.2) with $F = \min$ for split trees and $F = \max$ for join trees. The feature hierarchy from the previous section

² Some works use the terms join tree and split tree interchanged. According to Carr et al. [41], this is due to “join” and “split” being flipped in conference and journal versions of the work by Carr et al. [40] that originally coined the terms.

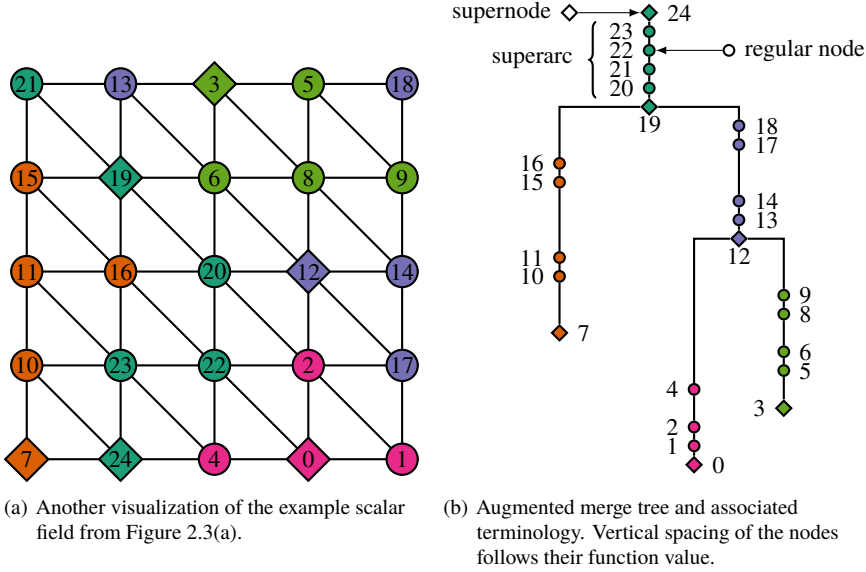


Figure 2.4: Example scalar field and its associated join tree. Compare to figure Figure 2.3 where a discretized version is shown. The vertices in the grid and nodes in the tree are marked according to their node type in the join tree and are colored according to the superarc to which they belong, i.e., the connected component which they represent. (adapted from Paper D)

can be seen as a discretized version of an augmented merge tree. It only retains components at distinct isovalues and their sizes; see Figure 2.3 vs. Figure 2.4.

Note that merge trees are often simplified with topology-based noise-removal and simplification schemes [62, 234]. This is an optional step for the methods we present in this thesis, yet often useful when noise is present in the original scalar data. This possibility is one of the core advantages of working with topological abstractions such as the merge tree.

2.2 Related Work on Extracting Trees from Scalar Data

In the following two sections, we give an overview of related work in regards to methods that extract feature hierarchies, merge trees and similar structures, and find correspondences between entities that represent nodes of those trees. Some of our

work is agnostic to the exact computation method for trees or correspondences. A different algorithm may replace the one we have utilized. For example, in Paper D, we use a specific implementation to compute augmented merge trees, which can be replaced by other methods from Section 2.2.1 that also result in an augmented merge tree. In Paper A, we derive the temporal edges by overlap, but any of the region-based methods in Section 2.2.2 may be used just as well. When discussing the computation of individual trees, we especially focus on methods that consider a distributed and parallel setting, which is the subject of Paper B.

2.2.1 Feature Hierarchies and Merge Trees

As we have seen in the previous two sections, extracting sublevel and superlevel sets for the creation of feature hierarchies or merge trees is related to *connected-component labeling*. This is a problem that occurs in many domains [93], but we will focus on the context of scalar fields here.

Even in this context, hierarchies based on connected components, i.e., merge trees, contour trees [40, 46, 123, 155, 165, 202], and variants of them have been introduced under several different names. The terms *enclosure tree* [22], *barrier tree* [67], *extremum graph* [105], *criticality tree* [47], *level-set tree* [116], and *volume skeleton tree* [201], describe all very similar structures – capturing relationships between critical points and connected components of sublevel, superlevel, or level sets. Additionally, merge and contour trees can be decomposed into branches, which can again be grouped hierarchically. This has led to the proposal of several meta-structures. Among them are the *branch decomposition tree* [157], the *triplet merge tree* [190], the *extended branch decomposition graph* [175], and the *interlevel set persistence hierarchy* [170].

Some of the tree variants mentioned above were initially introduced to improve the extraction of isosurfaces. In contrast to repeatedly computing a single isosurface using the *Marching Cubes* algorithm [132, 150], knowledge of connectivity changes in relation to an isovalue can be exploited to identify suitable isovalues [123] and extract contours faster by tracing [105]. Today, merge and contour trees are often used as a starting point for further data analysis and visualization.

A data structure central to labeling connected components in general and extracting many of the listed tree variants is the *Union-Find* data structure [203]. *Union-Find* is utilized in both one of the earliest connected components labeling algorithms by Hoshen and Kopelman [102] for percolation analysis (see Paper B and Paper C) and in state-of-the-art merge tree construction algorithms. It can also be used for deriving any of the trees based on scalar fields we use in the appended publications. Due to its centrality, we describe its use for finding connect components exemplified for a superlevel set in a sampled scalar field in

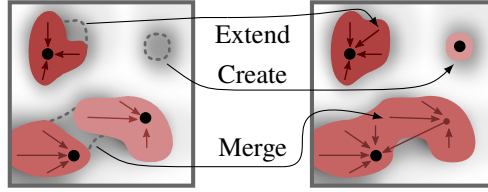


Figure 2.5: When connected components of the superlevel set of a scalar field are identified with the help of a Union-Find data structure UF , each grid vertex links to a representative of the connected component it belongs to. Here this is represented with black dots for these representatives and a number of exemplary arrows as links. As the algorithm progresses, the threshold decreases and new vertices are incorporated into the superlevel set. This leads to *creating* new components, as well as *extending* and *merging* existing ones. (from Paper B)

the following. We refer to the associated algorithm as the *Union-Find algorithm*.

The first step is to establish the order in which the vertices of the underlying grid are processed. For a superlevel set, all vertices are placed in a sorted list in descending order of the data values. This list begins with the global maximum and ends with the global minimum. Next, a union-find data structure UF is initialized. This data structure contains an initially empty space for each vertex. This space will eventually store a pointer either to another vertex or to itself once the entirety of the sorted list has been iterated over. A vertex that points to itself means that this vertex is regarded as the *representative* of the connected component to which it belongs. The data structure thus allows identification of connected components through following of pointers until such a circular pointer is encountered. To fill the UF data structure, we iterate over the sorted list starting with the highest data value. For each vertex v , we follow the pointers of all adjacent vertices and count the number of connected components, i.e., number of distinct representatives, that have already been recorded in UF for these adjacent vertices. When collecting these representatives, path compression [203] of the pointers replaces a path of several pointers with a direct link to the final representative. This improves the complexity of later component queries. The following cases occur (see Figure 2.5):

Create: None of the vertices adjacent to v contain pointers in UF yet. We create a new connected component by adding a circular pointer in UF for the current vertex v . The vertex v now represents the new component.

Extend: One or more adjacent vertices already contain pointers in UF , but they all point to the same single representative. The vertex v and its neighbors thus belong to the same component. We extend this component, adding a pointer to the component representative for the current vertex v .

Merge: Two or more adjacent vertices already contain pointers in UF , but these belong to more than one component. We decide on one of their representatives, e.g., the one for the largest or oldest component, to represent a new merged component. The current vertex v and all representatives of its neighbors are updated to point to the chosen representative.

To calculate the connected components of sublevel sets, merely the initial step of sorting vertices has to be reversed; see the definition of the join tree based on sublevel sets in Section 2.1.5.

Associated trees are built up while these operations are performed: Statistics about the connected components, such as their size, bounding box, etc. have to be gathered, and nodes and edges have to be added to a suitable tree data structure. To extract a merge tree, edges and nodes are added at every operation or vertex v as follows:

Create: We add v as a new supernode to the tree. It will be a leaf.

Extend: We add an edge from v to the neighbor with the closest value. v is a regular node in the merge tree and the edge is part of a superarc, unless v is the global extremum. In this case, it is added as a supernode.

Merge: v is added as a new supernode representing a saddle and edges for each connected component are added, again using the neighbor with the closest value, respectively.

More details, including step-by-step examples and pseudocode descriptions, are given in Carr et al. [40] and Carr [37].

If the goal is instead to extract a threshold-based feature hierarchy as described in Section 2.1.4, nodes and edges are only added when we have reached one of the desired thresholds. The current state of representatives within the union-find data structure yields the set of nodes N to be added, and the respective component sizes yield the associated values of the feature hierarchy. The edges of the hierarchy E are typically added in a second step by determining the enclosure relationships between the connected components.

Parallel and Distributed Algorithms

Recent work on computing (augmented) merge trees, contour trees, and connected components has focused mainly on their applicability to large data sets and accelerating their extraction through the use of parallel computing.

In this regard, one distinguishes between two paradigms, depending on how the parallel processes exchange data and access memory: *shared* memory and *distributed* memory [64]. With shared memory, parallel processes share access to a single common memory space, limiting this approach to a single machine.

Synchronization between processes is required when writing to memory. Otherwise, an inconsistent state may occur from two or more processes in race condition, attempting to alter the same data simultaneously. Reading the data, on the other hand, is possible in parallel and without synchronization. A commonly used programming interface for this type of paradigm is *OpenMP* [43]. With distributed memory, each process has access to its own independent memory, enabling distribution of data over several machines or *compute nodes* in a cluster. With the data segmented between several processes, the data at the segmentation boundaries is often duplicated in so-called *ghost cells* [158]. This is typically the only possible approach for large data sets that exceed the memory capacity of a single node. Synchronization between processes is enabled by an inter-process communication protocol such as MPI [78].

Parallel and distributed computation of merge trees, contour trees, and connected components is a very active research area. Several of the works mentioned in the following [38, 41, 104, 115, 142, 151, 235, 244] were developed concurrently or after Paper B. Of the work available before, none readily supported obtaining multiple percolation function samples with distributed memory.

Teuler and Gimel [205] as well as Tiggemann [209] each developed a parallel and distributed version of the Hoshen-Kopelman algorithm [102] for percolation analysis which is the analysis method utilizing union-find explored in Papers B and C. Similarly, distributed versions of the general union-find algorithm have been proposed [90, 139] with several improvements in terms of load-balancing and communication patterns [142, 244].

Several algorithms have been proposed to compute merge and contour trees in parallel on a shared memory machine. Pascucci and Cole-McLaughlin [156] describe two parallel algorithms to calculate contour trees where the geometrical domain is split between threads. An alternative is to divide the domain according to the scalar range, as done by Gueunet et al. [81]. Both approaches follow the scheme of iterating over a sorted version of the entirety of the scalar data. This is not the case for most other shared memory approaches. Several make use of an alternative algorithm based on first identifying all critical points and then connecting these through following a *monotone path* [46]. Maadasamy et al. [138] introduce a parallel variant for this algorithm specific to tetrahedral meshes. This is later improved upon by Acharya and Natarajan [1] who propose a particularly memory-efficient algorithm for computing contour trees. The algorithm creates a global result by computing intermediate results for parts of the domain using the monotone paths approach followed by merging them with union-find operations. This method has been designed for shared memory architectures, but since it already includes a domain partitioning, it lends itself to adaption for a distributed environment. Carr et al. [39] propose a data-parallel algorithm for computing contour trees explicitly

targeting SIMD (Single Instruction, Multiple Data) architectures. Another data-parallel algorithm by Carr et al. [41] makes use of *parallel peak pruning*, a process similar to simplifying merge trees. This algorithm can be further accelerated using a data structure similar to a branch decomposition [38].

Several approaches have focused on being able to extract augmented trees. Gueunet et al. [82] and Gueunet et al. [83] exploit modern scheduling techniques for high-performance computing by providing task-based formulations for merge and contour tree algorithms. A task-based parallelization strategy is also used by Werner and Garth [235] who introduce a specialized boundary data structure that allows for unordered execution of the tasks used previously. Rosen et al. [173] use a combination of range- and domain-based division of work further sped up prior identification of all critical points.

Saving a fully augmented merge tree requires memory strictly larger than the original scalar data. This means that algorithms for data sets so large that they cannot be held by a single processor generally do not compute a fully augmented merge tree. Instead, distributed approaches often result in an unaugmented global tree and may additionally include access to distributed local augmented versions. Morozov and Weber [143] compute a pair of simplified merge trees that are then merged pairwise [144]. Their approach is based on a *local-global merge tree* for which Nigmatov and Morozov [151] present a version suitable for adaptive mesh refinement data. A distributed variant for segmented merge trees is proposed by Landge et al. [125] who propose two in-situ variants that operate directly on a simulation's memory partitioning. Werner and Garth [235] use their task-based formulation in a distributed setting that accumulates a global unaugmented tree, while local augmented information is retained as the final result of the union-find data structure for each node.

Several distributed algorithms refrain from computing a globally valid tree entirely and instead offer the possibility to perform queries that are evaluated locally. The computed structure is called a *merge forest* and possible queries are, for example, the identification of all maxima or finding all vertices of a connected component given a threshold and vertex [104, 115].

2.2.2 Correspondences

Similar to the topic of connected components, tracking of features over time is a problem that occurs in many application contexts. For establishing temporal correspondences between feature hierarchies or merge trees, methods that associate either the regions of connected components (for temporal edges in feature hierarchies) or critical points (for temporal edges in merge trees) are relevant. Overviews of these are given in surveys on feature tracking in flow visualization [164],

topology-based methods in visualization [95], time-varying volume data [10], and the comparison of scalar fields based on topological structures [245]. The documentation of the *Feature Tracking Kit* (FTK) library [86], introduced with the aim of unifying the approaches based on regions and based on critical points, is also accompanied by an overview of methods from both categories.

Region-Based Tracking

Region-based tracking approaches for superlevel and sublevel sets in scalar fields [59, 106, 108, 145, 179, 182, 183, 187, 188, 191] often track the regions of connected components based on their overlap. A possible measure for this is the normalized volume difference as suggested by Silver and Wang [187]:

$$O(A, B) = \frac{\max(|A \setminus B|, |B \setminus A|)}{\max(|A|, |B|)} \quad (2.6)$$

If there is significant overlap in consecutive time steps, components are assumed to correspond to each other. Silver and Wang [187] report that thresholding the overlap measure from Equation (2.6) such that $O(A, B) < \alpha \in [0.5, 0.6]$ yields good results. This means that at least 40-50% of the larger feature must overlap with the smaller one. Several tracking methods implicitly use overlap as a metric by extracting higher-dimensional isosurfaces in space-time [106, 108] or propagating regions over time [191].

Additional or alternative aspects may be used to determine correspondences between regions. These include the distance between centroids of the regions [126, 179], prediction of a feature's future attributes [145, 166, 167], and the difference in the data value histograms [177] or data distribution [59, 107]. Recent approaches use optimization strategies to find either globally optimal tracks or a fully optimal matching between consecutive time steps based on these criteria. Saikia and Weinkauff [177] formulate the tracking of a selected hierarchically nested region as a Dijkstra shortest-path problem based on a metric combining histograms and overlap. They also present a particularly fast version to solve the shortest-path problem for all features simultaneously [176]. A similar approach using Dijkstra's algorithm is taken by Ma et al. [137]. Their feature definition is based on Gaussian Mixture Models (GMMs) and uses a metric based on histograms to determine the tracking for a user-labeled feature. Dutta et al. [60] employ a fuzzy rule-based system that learns a set of fuzzy rules on attributes such as differences in mass, volume, centroid, and velocity from manually tracked features and uses these rules to track the remaining features in a data set.

Ji and Shen [107] present an orthogonal approach that focuses on finding the best matching for all combinations of features in two consecutive time steps.

Earth movers distance is used to measure feature similarity and create a search tree of possible correspondences. This search tree is traversed with a branch-and-bound operation to find a set of correspondences with minimal cost. The work by Schnorr et al. [182, 183] also falls into this category. They find the best matching for all feature combinations through overlap and a combination of two graph-algorithms: first, solving a weighted bipartite matching problem yielding possible one-to-one correspondences, followed by solving a maximum-weight, independent set problem to identify merge and split events.

Critical Point-Based Tracking

Methods to track critical points instead of regions are often introduced in the context of vector fields. However, they can be easily utilized for scalar quantities by applying them to the respective gradient field. Tricoche et al. [210] and Garth et al. [73] track critical points in 2D and 3D flows by assuming temporal linear interpolation between time steps. Theisel and Seidel [206] introduce *Feature Flow Fields*, a derived vector field in which features are described by streamlines such that feature tracks can be established with integration. A numerically more stable version is due to Weinkauff et al. [230] and a combinatorial variant is presented by Reininghaus et al. [169]. Feature Flow Fields can also be used to solve the *Parallel Vectors operator* [160] which in turn allows to describe the temporal development of vortex structures [207]. It is also possible to track critical points based on different attributes, similar to the listed region-based approaches. For example, Skraba and Wang [189] determine tracks between critical points by considering their stability. Critical points are considered to be similarly stable when a similar perturbation of the scalar field leads to them disappearing.

A number of methods work on parts or entire merge trees and result in a matching of critical points. An exact tracking method for merge trees is suggested by Oesterling et al. [152]. The method considers each change to the sorting order of vertices when linearly interpolating between consecutive time steps and is thus computationally very intense. An alternative to this exact tracking is the matching of merge tree parts through distances between merge trees [130, 131, 195, 196, 237] or persistence diagrams [161, 170, 192].

2.3 Related Work on Visualizing Hierarchies

Due to their ubiquitous nature in various application domains, the space of methods to visualize trees is enormous. Overviews of how to visualize a single tree are given by Schulz [184], who provides the continuously updated visual bibliography browser *treevis.net*, Pandey et al. [154], who discuss a taxonomy of tasks con-

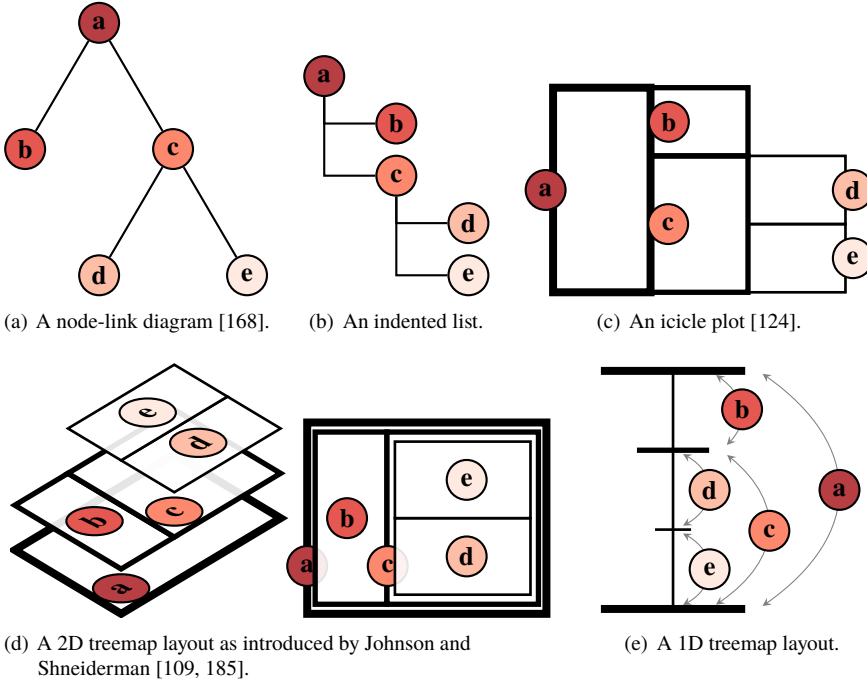


Figure 2.6: Classic tree visualization methods are node-link diagrams (a), indented lists (b), adjacency diagrams such as the icicle plot (c) and enclosure diagrams such as 1D and 2D treemaps (d,e). The same example tree is used for all examples. It has five nodes, three of which are leaves with equal associated value $d(\mathbf{b}) = d(\mathbf{d}) = d(\mathbf{a})$. (adapted from Paper A)

cerning tree visualizations, and Lima [128], who presents a historical perspective through examples ranging back to early hand-drawn artistic illustrations. Methods that display a single tree are also very prevalent for multiple trees. The survey by Graham and Kennedy [77] shows that most methods for multiple trees use either pairwise visualizations, small multiples, or animation; all of which show the individual trees. In the following, we first give a brief overview of classic methods to visualize a single tree and then discuss the visualization of dynamic trees with a focus on methods applicable to a larger number of trees.

2.3.1 Visualization of a Single Hierarchy

According to Pandey et al. [154] visualization methods for trees fall into the following categories: node-link diagrams, indented lists, adjacency diagrams,

enclosure diagrams, hybrid diagrams, and symbolic diagrams. A hybrid diagram is any diagram that combines aspects from the others, e.g., for different parts of the tree. Symbolic diagrams incorporate physical traits of trees found in nature. Both of these types of diagram are rare (see [154]) and we thus focus on the first four categories here. Examples are shown in Figure 2.6.

The *node-link diagram* (Figure 2.6(a)) and *indented list* (Figure 2.6(b)) capture the hierarchical structure through connecting shapes that represent the nodes. They differ mainly in node placement. For a node-link diagram, top-down, left-right, and radial layouts are common [30]. In an indented list, the nodes are placed in a linear list underneath each other. Each child has an additional vertical shift or indent in relation to its parent. Even without drawing the edges, the parent-child relationship could still be inferred. The color or size of a node may additionally encode associated quantities.

In adjacency and enclosure diagrams, nodes are always represented through shapes whose size is proportional to their value. For adjacency diagrams, shapes are placed adjacent to each other according to the edges of the hierarchy. Examples of this are *icicle plots* [124] (Figure 2.6(c)) that use rectangles and an axis-aligned layout and *sunburst plots* [198] that use rings in a radial layout. In enclosure diagrams, the shape for each node instead serves as a container for its children. An enclosure diagram may be built either by traversing the tree starting at the root and splitting a given shape recursively according to its children or by starting at the leaves and finding an enclosing shape for children with a common parent. An example of the latter approach is *circle packing*, where the representing shape is a circle [224, 236]. More common are enclosure diagrams with the former approach, also known as *treemaps*. These are especially useful for creating a one-dimensional representation of a tree with associated values; we thus discuss them in more detail later.

Schulz [184] presents an alternative classification and groups tree visualization methods according to the representation of the relationship between parents and children. They differentiate between explicit, implicit, and hybrid encodings. In an explicit encoding, edges are shown directly as lines or curves connecting the representations of nodes. This is always the case in *node-link diagrams* and can be the case in *indented lists*. When no explicit links are drawn, but node placement instead allows to infer the parent-child relationship, the visualization is considered to use an implicit encoding. *Adjacency diagrams* and *enclosure diagrams* fall into this category. Hybrid methods again use a mixture of the others.

Treemaps

The general concept of treemaps is introduced by Johnson and Shneiderman [109, 185]: The root is represented by a rectangular space. Its children are drawn inside

this space, abiding to the amount of data per child and a partition of this space along the x -axis. The following hierarchy layers follow the same process but alternate the partitioning axis with each layer. Figure 2.6(d) illustrates this by showing successive partitioning in 3D together with the final nested visualization.

A large number of treemap layout variations have been proposed. A recent survey by Scheibel et al. [180] lists a total of 81 algorithms published between 1991 and 2019. Early examples include variants that focus on improving the aspect ratio of the rectangles [25] or preserving a given order in the original data [186]. Treemaps are not restricted to nested rectangles. Other shapes used to create treemaps include general polygons within rectangles [18], Voronoi cells [12], and bubbles [75]. Some works additionally incorporate information beyond the data values. For example, Görtler et al. [75] and Sondag et al. [193] specifically target the expression of uncertainty in the data.

Treemap rendering methods emphasize a node's hierarchy level in different ways. Johnson and Shneiderman [109, 185] use boundary lines between the rectangles. Balzer and Deussen [12] combine boundaries of varying thickness with color variation. Wijk and Wetering [239] create curved and shaded surfaces of varying height in their method called *cushion treemaps*. We adapt this strategy in Paper A for our temporal treemaps.

Note that treemaps can also be defined in other dimensions. As noted already by Shneiderman [185], treemaps can be derived from a subdivided cube in 3D or a subdivided line in 1D. Neumann et al. [148] make use of the 1D version, augmented with arcs for additional non-hierarchical relations. A 1D treemap frees up one display dimension for other information. We exploit this in Paper A for incorporating the time dimension.

Specialized Methods for Trees Based on Scalar Data

All tree visualization methods can also be applied to trees based on scalar data. In the following, we describe some specialized methods. Of these, many are introduced in the context of contour trees, but can be just as well be used on the simpler merge tree.

A practical orthogonal layout for node-link diagrams of contour trees is proposed by Heine et al. [98]. As in the merge tree example in Figure 2.4, nodes are restricted in their vertical placement by their value. Interpreting edge directions from the drawing means that contour tree nodes can have several children and several parents. This means finding a layout of the trees that avoids edge crossings is more complex than for merge trees where a node always has a single parent. Heine et al. [98] use *simulated annealing* to optimize the layout. Pascucci et al. [157] present the *toporrery* where the nodes and links are placed radially in 3D. The method is modeled after an orrery, a mechanical model representing the

hierarchical nature of the solar system with moons orbiting planets that orbit the sun. A similar radial and three-dimensional representation are *topological cactii* introduced by Weber et al. [228]. In this method, the radial layout from toporrrery is augmented with statistics about the connected components. Links are drawn as tubes with their radius corresponding to area, volume, or a transformed version of these. Finally, spikes are added to display a second quantity.

A variation of a treemap for the branch decomposition tree of a merge tree called *Mergemaps* is introduced by Kamakshidasan and Natarajan [112]. Neither the merge tree nor its branch decomposition tree obey the rule of values being given only on the leaves. Thus, the method operates on a variant called *aggregate tree* which transforms the branch decomposition tree, so that this rule is followed. Essentially, an additional leaf is added to each inner parent whose value exceeds the sum of its children. The value of this leaf is set equal to the excess, and the parent's value is removed entirely.

Another abstraction of the contour tree is the *topological landscape* [227], which is computed through a process that subdivides an initially rectangular mesh similar to the one for constructing a treemap. By distorting the mesh with height, the result is a two-dimensional landscape with the same contour tree as the original data. Variants of topological landscapes improve upon preserving geometrical distances [17], volumes [91] and add interactivity to explore large-scale data [54].

Topological landscape profiles can also be computed in one dimension. Examples include the use of hill-like icons parameterized by size and persistence for each superarc applied to multi-dimensional point clouds by Oesterling et al. [153], and similar variants for barrier trees [222, 223] of optimization landscapes and level set trees [116, 117] of density estimates. Adapting these methods such that they can be applied to scalar fields and choosing suitable parameters, these yield equivalent results as the 1D merge tree representation we propose in Paper D.

Other 1D representations of contour trees occur in the *contour spectrum* by Bajaj et al. [11] and the histogram embeddings by Kraus [122]. For the contour spectrum, statistics over the total surface area and gradient integral of the level set, and volume of sublevel and superlevel set are shown in a 2D function plot in relation to the isovalue. The contour tree is embedded in the same space. Kraus [122] also embeds the contour tree in a function plot. They show a histogram with horizontal subdivisions according to the sizes of the connected components of the level set at each isovalue. The *percolation function* we compute and analyze in Papers B and C can be seen as an alternative 1D representation showing a different statistic over connected components in relation to the isovalue.

2.3.2 Static Visualization of Dynamic Hierarchies

In our own work, we introduce several static visualization methods for dynamic hierarchies where values may be given on either nodes or leaves, and both node values and the topology of the tree change. In the following, we give an overview of related work on other static visualizations grouped by taxonomy classes in Figure 2.2. A number of methods fall into other classes than our work. For a complete comparison, see the summary in Table 2.1. The table also includes dynamic approaches discussed in the next section (Section 2.3.3).

The major challenge of visualizing dynamic trees is to display the dynamic hierarchy such that both the individual hierarchy and potential associated values per time step, and the temporal evolution is accurately represented. Several works employ optimization approaches in relation to this.

Dynamic Hierarchies without Quantities

Integrating time into hierarchies that only undergo topological changes can again be done with an explicit approach that shows temporal correspondences as edges or an implicit approach using adjacency.

An example of explicit encoding is the work on *Dynamic Indented Plots* by Burch et al. [29]. Indented lists for each time step are arranged next to each other with edges between consecutive time steps indicating topological changes. Edges are color-coded according to the type of topological change, and an order to derive the linear arrangement in the indented lists is assumed to be given. Thus, no layout optimization is involved, e.g., for the reduction of edge crossings. Supported changes include moving, adding, and deleting nodes. Nodes that move are simply connected. For a node that is added (or deleted), a connection is drawn between the location of that node and that same position in the previous (or following) time step where the node does not exist yet (or anymore).

Explicit connections are also used by Vehlow et al. [217], who connect hierarchically structured graphs represented by an adjacency matrix. For large-scale dynamic graphs with hierarchical nesting Burch et al. [32] develop a method where a splatting technique deals with edge crossings instead of avoiding them.

Another explicit representation of temporal edges is observed in *tracking graphs*, which may or may not include hierarchy information. Tracking graphs are directed acyclic graphs, and good layouts for these avoid edge crossings. Even the earliest publications on feature tracking (see Section 2.2.2) show node-link representations of the tracked features. An example that includes a tracking graph with a hierarchy is the work by Widanagamaachchi et al. [238], who extract features and their correspondences on multiple levels. The result is recorded in a *meta-graph* that allows interactive exploration. To minimize edge crossings, a greedy

Method	Mode	Node Value Change	No values	Value on Leaves	Values on Nodes	Topology Change	Appear/Disappear	Split/Merge	Move	Optimization
Dynamic Hierarchies without Quantities										
<i>Dynamic Indented Plots</i> by Burch et al. [29] and methods for dynamic hierarchical graphs [32, 217]		×	✓			✓	✓		✓	⊘
<i>BarcodeTrees</i> by Li et al. [127]		×	✓			✓	✓	✓	✓	✓
<i>(Dynamic) Tracking Graphs</i> by Widanagamaachchi et al. [238] and other tracking graph visualizations in feature tracking publications (Section 2.2.2)		×	✓			✓	✓	✓		✓
<i>Cone Trees</i> by Robertson et al. [172] and other works where the hierarchy can be edited interactively [240]		×	✓			✓	✓		✓	
Static Hierarchies with Dynamic Quantities										
<i>TimeEdgeTrees</i> by Burch and Weiskopf [33]		✓		✓		×				
<i>MultiStream</i> by Cuenca et al. [49] and other stream graphs with hierarchy information [14, 226]		✓		✓		×				
Dynamic Hierarchies with Dynamic Quantities										
<i>TreeVersity</i> and its successor <i>TreeVersity2</i> by Guerra-Gómez et al. [79, 80]		✓			✓	✓	✓			
Value-augmented indented lists by Burch et al. [31]		✓			✓	✓	✓			
<i>(Dynamic) Nested Tracking Graphs</i> by Lukaczyk et al. [135, 136]		✓		✓		✓	✓	✓		✓
<i>RoseRiver</i> by Cui et al. [51] and its predecessor [50]		✓			✓	✓	✓	✓		✓
<i>SplitStreams</i> by Bolte et al. [21]		✓		✓	✓	✓	✓	✓	✓	⊘
<i>Chronicle</i> by Wittenhagen et al. [241] and predecessors [45, 204]		✓			✓	✓	✓	✓	✓	⊘
<i>Fuzzy Contour Trees</i> by Lohfink et al. [131] and other approaches superimposing contour trees [130, 242, 246]		✓			✓	✓	✓	✓	✓	✓
Animated Treemaps [15, 20, 42, 44, 76, 87, 88, 181, 194, 200, 213, 219, 220, 225]		✓		✓		✓	✓			✓
<i>Barrier Tree Sequences</i> by Heine et al. [96, 97]		✓			✓	✓	✓	✓	✓	✓
<i>Temporal Topological Landscapes</i> by Evers et al. [65] and Herick et al. [99]		✓			✓	✓	✓	✓	✓	✓
<i>Temporal Treemaps</i> in Paper A		✓		✓	✓	✓	✓	✓		✓
<i>Percolation Analysis</i> in Papers B and C		✓			✓	✓	✓	✓	✓	⊘
<i>Temporal Merge Tree Maps</i> in Paper D		✓			✓	✓	✓	✓	✓	✓

Table 2.1: Overview of related work on visualization of dynamic hierarchies classified according to the taxonomy in Figure 2.2. Additionally included are the mode of the visualization and whether any optimization regarding stability over time is involved. For the mode,  represents static visualizations using a *time-to-space mapping* and  represents animation with a *time-to-time mapping*. Note that optimization is not desirable in application domains where the hierarchy has some inherent order. The respective methods are marked with ⊘ in the optimization column.

heuristic is used. It places each node in relation to all its corresponding nodes from the previous time step.

The *BarCodeTrees* by Li et al. [127] are an example of an implicit approach. They propose using a variation of an indented plot that uses height or width instead of indent for a hierarchy layer. Nodes that directly correspond to each other are aligned, and additional empty space signifies added or deleted nodes. Without adding edges for moved nodes, their perception is limited.

Static Hierarchies with Dynamic Quantities

One possibility of showing changing values in relation to a tree that does not change topologically is by augmenting one of the classic single-tree visualizations. With *TimeEdge Trees*, Burch and Weiskopf [33] introduce a method that augments node-link diagrams. The edges are drawn as straight lines and serve as a timeline with the start time at the parent and the end time at the child. The quantities associated with the edges are then represented by either color, edge width, or a combination.

The opposite is also possible, where a classic visualization for time series data is augmented with hierarchical information. An example is the use of several 1D graphs directly stacked on top of each other. Each graph represents the evolution of an individual node and is drawn as a thick colored band. *ThemeRiver* by Havre et al. [92] is among the earliest, but recent work refers to this method as a *Stream Graph*. The marginless stacking of the bands results in the evolution of each band affecting the space occupied by the other bands. Strategies to choose good orders which reduce the amount of “wiggles” in such visualizations are due to Byron and Wattenberg [34] as well as Bartolomeo and Hu [13] and Bu et al. [26]. Several works [14, 49, 226] integrate hierarchical information into stacked graphs. Hierarchy is conveyed by color encoding [49, 226], joint display of a tree [49, 226], or separately showing hierarchy layers [14] and facilitates interactive exploration of the data [14, 226].

Dynamic Hierarchies with Dynamic Quantities

Consider both the topology of the tree *and* the node data change over time. An early method with a focus on this kind of data is due to Guerra-Gómez et al. [79, 80] who also developed the taxonomy from Figure 2.2. They propose *TreeVersity2* [80], an extension of *TreeVersity* [79] applicable to trees where nodes may only appear and disappear, but values can be associated with either nodes or leaves. The method allows for the comparison of two hierarchies through a node-link visualization enhanced with a glyph called *Bullet* on each node. For comparison of multiple hierarchies, an area-based visualization called *StemView* is included. The glyph *Bullet* encodes different aspects of value changes, e.g., direction of change, absolute

change, and relative change for a node and whether the node was added or removed. *StemView* additionally shows starting or ending values for each node. A value-augmented version of the indented list by Burch et al. [31] models the appearance and disappearance of nodes by setting the corresponding values to zero. Here, an indented list is drawn next to a colored-coded matrix. Each row corresponds to a node, and each column corresponds to a point in time.

Lukasczyk et al. [136] present *Nested Tracking Graphs*, which additionally support merges and splits. The work is presented in the context of feature hierarchies as defined in Section 2.1.4. Sublevel or superlevel components are extracted at up to three selected levels, and correspondences are determined with overlap. The method uses the GraphViz library [72] to calculate a layout with minimal edge crossings for each hierarchy level. While this produces the desired results for each level, the final result is based on nesting the layer-wise layouts. This reintroduces a larger number of crossings due to the neglect of hierarchical information during the layout computation. We improve upon this in Paper A. In a later extension, Lukasczyk et al. [135] introduce *Dynamic Nested Tracking Graphs* that relax the requirement to pre-select distinct thresholds. Instead, nodes, edges and corresponding direct visualizations of the connected components from several viewing directions are pre-computed for a set of several thresholds. This allows for downstream interactive data exploration through approximation at thresholds not part of the pre-computed set.

Merges and splits in connection to hierarchies have also been explored by Cui et al. [50] and Cui et al. [51]. They visualize the evolution of topics in text corpora with merges and splits and a hierarchical definition of the topics. The hierarchy is represented by cuts through the tree at possibly varying hierarchy levels. Thus, the entire tree per time step is not shown. Optimizing the layout incorporates both stability with respect to different tree cuts and temporal evolution through a set of constraints. An optimal solution is found with a general-purpose constraint solver.

Moves do not occur in threshold-based feature hierarchies based on layer-wise overlap. Thus, the works for this kind of dynamic hierarchy ([135, 136] and Paper A) do not support or account for moving nodes. Bolte et al. [21] introduce *Split Streams* to accommodate this additional topological change. They furthermore improve the perception of the hierarchy within each time step by adding a split and a margin in between time steps. In their work, an explicit ordering of the nodes is assumed to be part of the input. Thus, a stage to reduce edge crossings is described as optional and is only recommended to be included when no ordering is given.

Moving, splitting, and merging nodes are also supported in *Chronicler* [241], a method to interactively explore the source code history of a single file. The tree is based on syntax with nodes representing code snippets and the hierarchy being

imposed through structural code elements and scopes such as a function, a loop, an if-statement, etc. The value of a node is equal to the number of lines of code it represents, and changes to the tree come from code revisions. The linear order of the code snippets imposes an order of the nodes in each revision. The method improves upon earlier work by Chevalier et al. [45] and Telea and Auber [204], who put particular emphasis on how to identify moving, merging, and splitting nodes in trees based on source code.

A group of works that support all topological changes is based on the computation of a unified tree. The final visualization shows either only the unified tree [212, 242, 246] or uses it to find a common layout where the individual trees are superimposed. Examples of the latter are *Fuzzy Contour Trees* for ensemble data [131] and time-varying data [130]. Direct correspondences are shown only implicitly. However, the process of finding a single unified tree over many trees may be used as an alternative to our optimization of the layout in Paper D.

2.3.3 Dynamic Visualization of Dynamic Hierarchies

Several approaches use a *time-to-time mapping* to visualize dynamic hierarchies. Creating a stable layout is even more critical in this case. In a static visualization, the crossing of edges or other obvious visual discontinuities can be used to indicate that the nodes correspond to one another, but the arrangement in time is unstable. Unfortunately, this concept is not easily transferable to animation.

Early work on novel treemap layout strategies [15] already discusses the concept of a *stable* layout in relation to time-dependent data and introduces metrics that measure layout stability. A challenge is to balance between a good treemap layout for each individual time step and the number of layout changes. For rectangular treemaps, Vernier et al. [218, 220] present a quantitative evaluation in relation to a large number of stability metrics. An overview of layout metrics is also given by Bethge et al. [20] and Hahn et al. [87]. Rectangular treemap layouts targeting stability either use the same traversal order of the nodes in each time step (e.g. [15]), have a time-dependent state where the layout of one time step is updated [194, 219], or use global commonalities between all trees [44, 181].

A similar theme can be found in variations of Voronoi treemaps for temporal data. A group of methods ensures consistent ordering during initialization [88, 94], while others perform incremental updates based on the previous time step [76, 200, 213, 225]. An entirely different approach is taken by Bethge et al. [20], who use a neural network to predict the value of several layout metrics in relation to changing data and also mix between rectangular treemaps and Voronoi treemaps for different parts of the dynamic hierarchy. All animated treemaps consider only nodes appearing and disappearing, but no nodes merging, splitting, or moving.

Methods in which only the topology changes can use animated node-link diagrams. Wilson and Bergeron [240] and Robertson et al. [172] present node-link diagrams in 2D and 3D radial layouts, respectively. The visualizations are animated in response to a user interactively changing the hierarchy. Wilson and Bergeron [240] specifically visualize trees that are created from partitioning a set of objects. Each object has multiple associated data values, and partitions are defined on the basis of user-selected value ranges. They suggest several layouts, some also integrating these ranges, and use animation to blend between different ones.

Both topology and value changes occur in barrier tree sequences. Heine et al. [96, 97] present a stable node-link diagram layout. The layout is based on finding a union of all trees and thus has some similarity with the approaches that superimpose or summarize merge and contour trees.

An animated version of topological landscapes also makes use of such a super structure. Herick et al. [99] present a method based on the calculation of time-varying split and join trees that are combined to a time-varying contour tree. This structure then enables the calculation of temporally coherent individual landscapes per time step. As an alternative to animation, they also propose to apply volume rendering to a stack of 2D landscapes. In an extension of this work, Evers et al. [65] showcase the application of the method to a 3D simulation ensemble.

2.4 Related Work on Other Static Visualizations for Scalar Data

To visualize dynamic scalar data statically with one dimension representing time, the entirety of each scalar field per time step needs to be compressed from its spatial domain of d dimensions into 1D. In Paper C, we present a method that uses augmented merge trees as a means to retain features of the scalar fields and optimize their representation towards stability. However, there are alternatives to this approach, which we discuss here.

Generally, a compressed version of a scalar field can be derived from traversing the data samples in some linear order. A group of methods that create such an order are *space-filling curves* which traverse the d -dimensional space in a continuous fashion without self-intersections. The early versions of these curves, which are widely used in computer science today, are due to Peano [159] and Hilbert [101]. Peano and Hilbert were inspired by Cantor [36] who proved that the set of points in infinite d -dimensional space and the set of points on the infinite unit line segment have the same cardinality and can thus be mapped to one another with one-to-one correspondences. Space-filling curves have many applications, but for scalar fields, they facilitate the comparison of data sets by superimposition [55, 232] and creating

spatio-temporal visualizations [68] similar to the one we propose in Paper D.

Most space-filling curves take only spatial locations into account, but are otherwise data-unaware. Dafner et al. [52] and Zhou et al. [250] propose exceptions with their data-aware space-filling curves. A drawback of the above methods is that in contrast to our proposed linearization in Paper D none keeps features of a scalar field intact.

Franke et al. [68] provide a comparison of several curves in terms of quality measures quantifying spatial coherence. As their work does not focus solely on scalar fields, their comparison includes alternatives based on hierarchical clustering of points [85]. Another group of works deriving a linear order from a set of data samples focuses on point trajectories [27, 28, 243]. Among the projection approaches explored are space-filling curves, multi-dimensional scaling [178], as well as classic methods for dimensionality reduction. The latter are adapted by Wulms et al. [243] for temporally coherent visualizations. Although the methods for point trajectories share the challenge of temporal coherence with our work, they cannot be directly applied to scalar fields.

3

Distributed Percolation Analysis

In this chapter, we give an overview of the work on distributed percolation analysis presented in Paper B. *Percolation analysis* is inspired by *percolation theory* which is concerned with the connectivity of randomly connected infinitely large graphs. The derived analysis method enables the examination of statistics for connectivity in finite graphs and scalar fields. It is based on computing the so-called *percolation function*, which quantifies the relative size of the largest connected component in a superlevel set for a decreasing threshold.

The sizes of connected components at distinct thresholds are also the quantity recorded in threshold-based feature hierarchies for a scalar field. Therefore, performing a percolation analysis can be understood as mapping an associated feature hierarchy to 1D through its value domain. Locations in the 1D mapping correspond to the thresholds at which the connected components in the hierarchy are recorded, i.e., selected samples from the value domain. The quantity associated with each location results from the percolation function at that threshold. Paper B focuses on a single time step and turbulent flows as a specific application. Multiple time steps and general scalar fields are the subject of Paper C.

Before we discuss the motivation and contributions of Paper B, we briefly review the concept of percolation theory in the following.

Percolation Theory

Percolation theory is a mathematical concept due to Broadbent and Hammersley [24] that considers the connectivity of a randomly chosen subgraph $\mathbf{L}'$ of an infinitely large lattice $\mathbf{L}$. Probability p steers how this subgraph is derived. Vertices of $\mathbf{L}$ are considered to be *open* with probability p and *closed* otherwise. The central question of percolation theory is whether the *open* subgraph $\mathbf{L}'$, made up only of the *open* vertices and adjacent edges, contains a so-called *percolating cluster*. A percolating cluster is an infinite connected component that spans the entire domain of the original lattice $\mathbf{L}$. Interestingly, the probability $P(p)$, that this infinite cluster exists, exhibits a sharp transition at a particular value p_c that depends exclusively on the connectivity of the original lattice $\mathbf{L}$. For probability $p > p_c$, the open subgraph $\mathbf{L}'$ contains only finite connected components, whereas $\mathbf{L}'$ certainly contains an infinite percolating cluster for $p > p_c$. This value p_c is referred to as the *percolation threshold* or *critical value*.

The sharp transition is the reason why percolation theory is often used to model properties that undergo a *phase transition*. An intuitive example of such a property is the permeability of a rock. Depending on how porous the rock is, water can flow through it. Higher values of p , as in Figure 3.1(b), correspond to a very porous rock, represented by a larger fraction of open vertices. It allows for water to pass through. However, when a large fraction of vertices is closed, i.e., p is low as in Figure 3.1(a), no path of open vertices spans the entire domain and the rock becomes impermeable. The percolation threshold p_c marks the porosity at which the rock switches between being permeable and impermeable.

From a theoretical point of view, researchers have determined percolation thresholds for many different lattice types [252] partially through exact theoretical derivations and partially through various estimation schemes. In addition to the above scenario, researchers also consider the connectivity of subgraphs $\mathbf{L}'$ where p determines the probability of open edges and their adjacent vertices instead. This is called *bond percolation* and the type of percolation described previously is *site percolation*. For example, an infinite uniform 2D grid (2D lattice with 4-connectivity) has a site percolation threshold $p_c \approx 0.5927$ [66] and a bond percolation threshold of $p_c = 1/2$ [89, 113].

Percolation analysis translates the concepts from percolation theory to real data. Instead of a probability, vertices are considered to be open or closed in relation to a threshold. Instead of a probability of the existence of a percolating cluster, one computes a derived function, a *percolation function*, based on the volumes of the connected components. An example of the result of a particular percolation function ($P_{\max}$, defined in the next section) applied to random data sampled on either a 1024^2 or 128^2 lattice is given in Figure 3.1(c). The transition can also be observed in the function, however its sharpness depends on the data size.

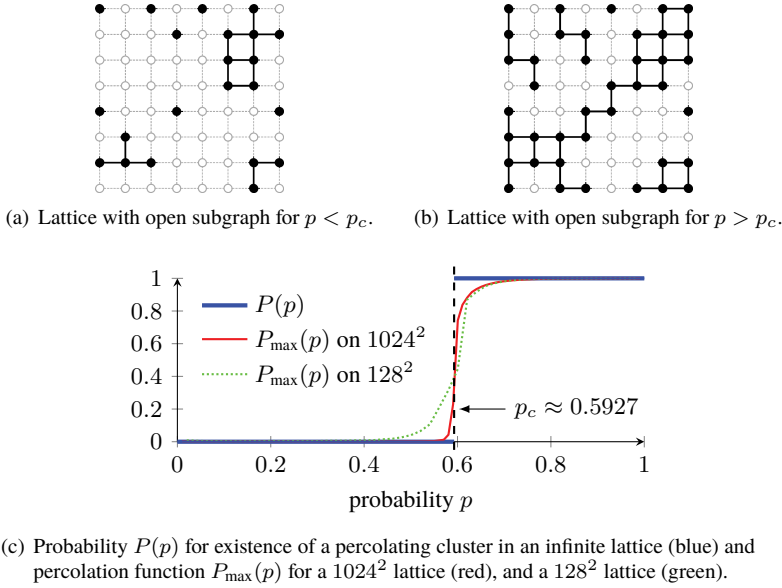


Figure 3.1: The vertices in an infinite lattice are considered to be *open* ($\bullet$) with probability p or *closed* ($\circ$) otherwise. An infinite *open* connected component exists with probability $P(p)$ zero for $p < p_c$ and one for $p > p_c$. The effect, although less pronounced, can also be observed for real data.

Percolation analysis finds its application in many areas where sharp transitions occur. Aside from the geology example given above, other application domains include cosmology, material science, and epidemiology [174]. Our focus here is on its application to turbulent flows, which we detail in the next section.

3.1 Motivation

Turbulence has an effect on the efficiency of vehicles, the wear and tear of mechanical parts like pipes or engine parts, and is generally involved in many other fluid flows occurring in nature. Therefore, accurate modeling, simulation, and contributing to a better understanding of turbulent flows are of great importance.

One of the fundamental building blocks of turbulent flows are *coherent structures*. These are fine-grained structures, often spread out throughout the entire domain, where the flow exhibits a particular behavior over a time period. They are typically defined in such a way that high values of a scalar field form the structures. The definition of coherent structures should account for differences

in the turbulent flow where varying velocity magnitudes are caused by boundaries or other geometrical objects. A strategy explored for the *Duct* flow [6, 7] is to normalize each velocity component by subtracting its temporal mean and dividing it by its temporal root mean square. Normalizing also may include completely disregarding some parts of the domain near the wall.

Studying coherent structures in terms of statistical properties, i.e., frequency, intensity, spatial distribution, and temporal evolution, is equivalent to investigating these properties in the connected components of a superlevel set for a well-chosen scalar field. However, their fine-grained nature requires simulations of high temporal and spatial resolution to attain a degree of detail that facilitates understanding. Simulating such highly resolved turbulent flows is a computationally intense process that requires the computational power of supercomputers with access to large amounts of computing time, storage, and numbers of processors.

Facilitating analysis of coherent structures involves two major challenges: First, we need to ensure that the coherent structures are distributed homogeneously in the domain. This means that the normalization strategy used to account for flow variations was successful and that we are treating all structures on an equal footing. Fluid dynamicists analyzing turbulent flows then aim to select a suitable threshold h such that coherent structures correspond to components of the superlevel set of a scalar field $s(\mathbf{x}, t)$. Threshold h should be chosen such that extracted structures are intense, numerous, and small enough to be meaningful.

As shown by Moisy and Jiménez [141] in the context of isotropic turbulent flow, percolation analysis with a specific *percolation function* $P_{\max}$ can address both these aspects. They define the percolation function $P_{\max}$ as

$$P_{\max}(h) = \frac{V_{\max}}{V_{\text{total}}} . \quad (3.1)$$

V_{total} denotes the total volume of the superlevel set for a given threshold h , and $V_{\max}$ is the volume of its largest connected component.

If coherent structures are indeed distributed homogeneously within the scalar field $s(\mathbf{x}, t)$, the percolation function exhibits a sharp transition from almost zero to almost one. The threshold at which the transition occurs is accordingly called the *percolation threshold* H_c . Suitable thresholds for coherent structure extraction are typically chosen relative to it, e.g., as $2H_c$. The value can be approximated from the percolation function $P_{\max}$ as the point of its steepest slope,

$$H_c = \operatorname{argmax}_h \frac{\partial P_{\max}}{\partial h} , \quad (3.2)$$

which requires a dense sampling of $P_{\max}$. Previous work struggled with this aspect, as the algorithms deployed had high computational complexity. For example,

Moisy and Jiménez [141] use a flood-fill-like algorithm with a runtime of $O(nk)$, where n is the number of grid vertices and k is the number of tested thresholds. Several publications report the need to restrict the computational domain for percolation analysis to a subset to avoid high computational costs [53, 133]. An improved algorithm by Friederici et al. [69] exploits the *union-find* data structure as described in Section 2.2.1 with a runtime of $O(n \log n)$. Although this algorithm is much faster, it cannot easily be applied to turbulent flow data that is too large to be hosted on a single compute node.

For this setting with large data, we require an algorithm that works on distributed memory. As we have seen in Section 2.2.1, there are many algorithms that identify connected components in either a shared memory or distributed memory setting. However, none of the works available at the time readily supports our setting of obtaining multiple percolation function samples with distributed memory. This leads us to the contributions of Paper B.

Key Contributions with their Contribution Areas

Tree extraction and representation: We present a novel parallel and distributed algorithm to finely sample the percolation function for conducting percolation analysis of turbulent flows.

Evaluation of our method: We provide a scaling analysis of our algorithm for data sets with different sizes.

Comparison to other methods: We provide a comparison against related methods from topological data analysis.

Experiments on real-world data: We showcase the utility of percolation analysis for analyzing turbulent flows using different large data sets.

3.2 Method Overview

We introduce the first *parallel* algorithm for computing the percolation function on data *distributed* across several MPI processes [78] that run on several compute nodes. It is based on a distributed version of the union-find algorithm (see Section 2.2.1) with additional data structures that allow us to easily compute the percolation function from Equation (3.1).

Parallelizing the union-find algorithm in a memory-distributed setting requires addressing two core challenges: First, the algorithm begins by sorting all vertices in descending order of the data values. Sorting is a global operation and is not straightforward when data is distributed. Second, the extent of connected components increases while the threshold is lowered and eventually crosses the boundaries of a single process. Assembling statistics for these *global* components or clusters

requires synchronization between different processes to maintain consistency.

In contrast to related algorithms that compute merge trees or connected components for a single threshold, the specific application of recording the percolation function comes with a number of properties that aid us in designing a particularly efficient algorithm.

First, the percolation function only needs to be evaluated for a reasonable number k of threshold samples $\{h_1, \dots, h_i, \dots, h_k\}$. Values of k with $10^2 < k < 10^4$ are already sufficient to obtain a well-resolved graph. In comparison to the number of data points, this number is relatively small, and we may process a large number of vertices without the need to evaluate volume statistics. Additionally, the kind and order of union-find operations between thresholds are of no relevance, granted that the final state for the connected components remains the same. As long as we make sure that all vertices with data values smaller than h_i (and only those vertices) are processed before querying the volume statistics for that threshold, we are free to re-order the processing of any vertices in between. Similarly, the exact positions of the connected component representatives are not important for computing volume statistics. This allows us to follow adapted rules for choosing representatives. For example, we can choose representatives to always be local such that process boundaries do not need to be crossed. The latter two properties are not given in topological algorithms such as the computation merge trees, where representatives have inherent meaning as minima or maxima, the exact order of vertices needs to be retained, and merges follow a deterministic order where the newer components always merge onto the oldest. Finally, we observe that the most prevalent union-find operation in turbulent data sets is the operation that *extends* an existing component. We consider this when balancing the cost of *extends* against *creates* and *merges*.

We exploit these properties to devise an algorithm that only synchronizes at the threshold samples, keeps as much information as possible local, and communicates the least possible amount of information between processes. To ease this synchronization, we introduce a single *global process* that serves as a separator of local processors with layers of vertices between them. All *local processes* and the global processes additionally hold a copy of adjacent vertices as *ghost vertices*. A simple case with two local processes is shown in Figure 3.2(a). We then interleave local processing with a global synchronization step. The separation layer ensures that directly neighboring vertices are never processed simultaneously.

Our algorithm consists of the following steps for each value range $(h_i, h_{i-1}]$ for $i = 1, 2, \dots, k$ (see also Figure 3.2(b)):

1) Local Process Update Every local process updates a local union-find data structure covering only the part of the domain that it holds itself, as well as a copy of a global union-find data structure that again only contains the information

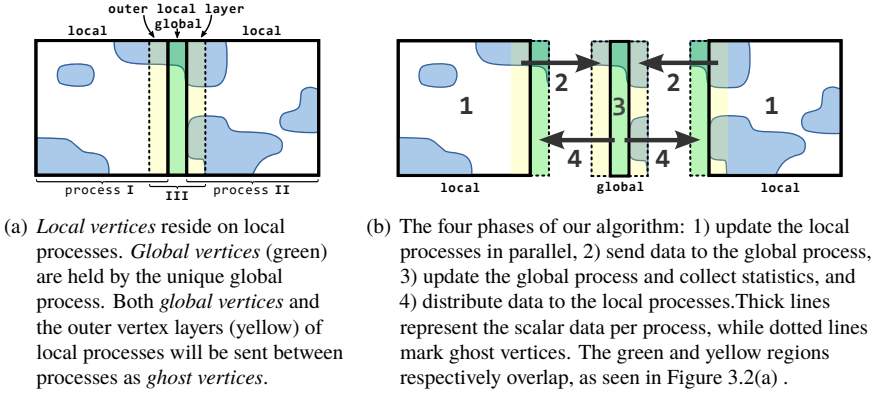


Figure 3.2: Layout of our memory distribution and the four phases of our algorithm.

local to the respective process. Any component that contains at least one vertex in the boundary vertex layer is considered a *global component*. The global process handles merges and creations of these to ensure consistency. Assume we have three global components **A**, **B**, and **C**, and two local processes. One needs to merge **A** and **B**, and the other needs to merge **A** and **C**. If both processes applied their respective *merge* operation directly, we would end up with two components, a larger **B** and a larger **C**, each now including **A**, instead of a single large component that encompasses all three. Thus, local processes merely record these global types of operation and do not apply them directly.

2) Local to Global Process Communication Each local process sends its updated boundary vertices, updates to volumes of global components, recorded global events, the sum of all local volumes, and the largest local volume to the global process.

3) Global Process Update The first action taken by the global process is to mirror the creation of all new global components. Then, it processes the separation layer, i.e., all global vertices. Merges are again only recorded. These are processed together with the merges received by the local processes. A single component may be involved in multiple merges, and several processes may have recorded the same merge. Thus, the global process constructs a reduced list of *merge sets* that summarizes which global components should become a one and applies these. Finally, the statistics are assembled. We combine updated data on global components with the statistics received about all local components and end up with the total volume in general, and the volume of the largest connected component. Thus, we can now record the final value of the percolation function at h_i according to Equation (3.1).

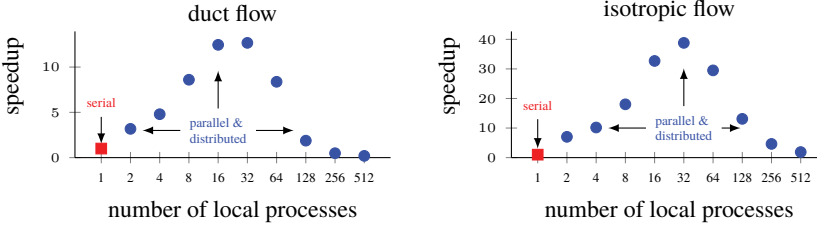


Figure 3.3: The strong scaling analysis shows the speedup of our distributed algorithm (blue) over the serial algorithm (red). Our algorithm scales very well until around 16-32 processors, at which point the lower workload per local process and increased communication costs lead to a lower speedup.

4) Global to Local Process Communication Before computing the next range of values $(h_i, h_{i+1}]$, the local process must enter a valid state. The global process sends its updated boundary vertices, the reduced merge list, and the aggregated number of newly created global components. Each local process then creates new components in its data structure for global information and also applies all global merge operations. At this point, the full domain is synchronized and in a valid state: Vertices that belong to the same component point to a correct local representative in every process, regardless of whether the component is local or global. Thus, we can now continue with the next threshold and start at step 1) again.

3.3 Evaluation and Results

We evaluate our algorithm using three fully turbulent flows: a wall-bounded flow in a *Duct* [6, 8] of size $193 \times 194 \times 1000$; and two wall-free flows exhibiting homogeneous *Isotropic Turbulence* [248] with sizes 512^3 and 4096^3 . The two smaller data sets still fit into the memory of a compute node. This allows us to compare against the serial algorithm of Friederici et al. [69] and against the augmented merge tree computation method of Gueunet et al. [83] that computes in parallel on a shared memory machine. We use the 4096^3 data set to showcase the feasibility of computing the percolation function with our method for a data set size that previous methods could not accommodate.

We perform a *strong scaling analysis* [4] that measures how the wall clock time changes with an increasing number of processes working on the same total problem size. Using $k = 1\,000$ threshold samples and ten runs of the algorithm, speedup is measured as the average parallel runtime in relation to the average serial runtime. Figure 3.3 shows the speedup of the parallel (blue) over the serial algorithm (red). The average serial computation times are ≈ 82 and ≈ 651 seconds for the duct and the isotropic flow, respectively. Initially, the performance more than doubles

while doubling the number of processes. This is mostly due to reducing cache misses, which we confirmed with Cray performance tools [58]. Eventually, we notice that the speedup throttles and goes down around 16-32 processors. At this point, the connected components of the superlevel sets have outgrown their small local processors and span over several processors, thereby inducing increased communication costs.

An increase in communication cost can also be observed when the number of threshold samples increases. While this yields a higher precision for the percolation function, it also entails more global synchronization, as synchronization is necessary for each threshold sample. Testing a varying number of thresholds on the duct flow with 32 processes shows that the fraction of computation time spent on exchanging information increases from 22% with $k = 100$ to 75% with $k = 1\,000$, and 96% with $k = 10\,000$.

As an additional sanity check, we compare our approach with a state-of-the-art method for computing augmented merge trees due to Gueunet et al. [83]. While this method is not designed to compute the percolation function and uses different parallelization technologies (shared memory parallelization through OpenMP [43] instead of message passing through MPI), the method still shares some similar algorithmic ingredients with ours. We expect that the computation times are in the same ballpark. Running both algorithms on the same data and with a similar concurrency level indeed yields similar computation times.

For the larger version of the *Isotropic Turbulence* flow, percolation analysis is conducted with 64 local processes. This is the smallest number of processes where the analysis is feasible. Each local process already requires approximately 31.8GB of memory for input data and data structures in this setting. Thus, each process has to run on its own compute node. We repeat a run of the algorithm four times and obtain computation times between 14 and 21 minutes. This is an indication of rather variable communication costs between compute nodes.

The computational results for the *Duct* flow and *Isotropic Turbulence* flow at the smaller resolution are shown in Figures 3.4 and 3.5. The duct flow has been normalized according to the normalization approach of Atzori et al. [6], while normalization is not necessary for the isotropic flow. This data does not include walls that cause varying velocity profiles. Its turbulent structures share the same statistical properties independent of the spatial translation (homogeneous) and rotation (isotropic) of the underlying frame of reference.

Both percolation curves exhibit a percolation crisis, as expected. The associated connected component renderings confirm that a threshold $h = 2H_c$ before the percolation crisis captures the coherent structures well, whereas thresholds at and after the crisis yield few connected components that consume most of the volume, which is unsuitable for the turbulence analysis.

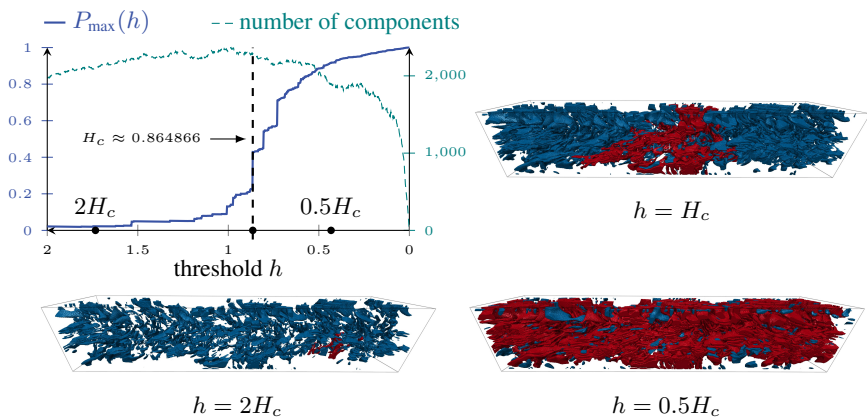


Figure 3.4: Percolation analysis for the *Duct* flow data set. We show superlevel sets for thresholds $2H_c$, H_c and $0.5H_c$ in blue, with the respective largest component highlighted in red. The existence of the percolating cluster can be observed for thresholds below H_c . For larger thresholds, the superlevel set consists of numerous small connected components.

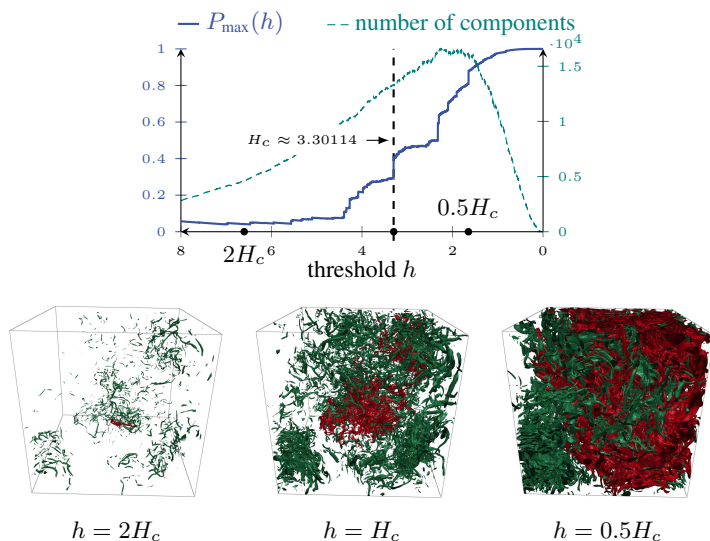


Figure 3.5: Percolation analysis for the *Isotropic Turbulence* flow data. Below, the superlevel sets for selected thresholds are shown in green, with the respective largest component highlighted in red.

4

Percolation Analysis on Sampled Scalar Fields

The work presented in the previous chapter highlighted that evaluating the percolation function on randomly sampled data (see Figure 3.1) and on turbulent data with homogeneously distributed structures (see Figures 3.4 and 3.5) can yield a sharp transition in this function. However, the width of that transition varies with the size of the data set and the critical value varies with the dimensionality of the data, the lattice, the underlying data distribution, and the value range of the scalar field.

This naturally raises a few questions: How does the percolation function generally behave for data with more structure? Can transition width and critical value be used to put a data set into context by comparing to corresponding values on uniform random data or known values for the underlying lattice? Is the function stable over time for time-dependent data sets?

Through exploring these aspects, we identify computational aspects to consider and potential pitfalls to avoid when computing and analyzing a percolation function. We present these in Paper C and summarize them in this chapter.

The computational modifications we introduce to extract the percolation function constitute a variation of the mapping from a feature hierarchy to one dimension.

4.1 Motivation

Automatic analysis of a percolation function to obtain the critical value p_c and other characteristics, such as the width Δ of the percolation transition, is of interest for both estimations on infinite lattices and on general scalar data.

As described by Newman and Ziff [149] *Monte Carlo* simulations on a sufficiently large finite lattice can be used to approximate p_c . Averaging a function that is 1 where the percolating cluster exists and 0 elsewhere over all Monte Carlo runs yields an estimation of the percolation probability $P(p)$ from which p_c can be estimated. Both the site percolation threshold $p_c^{2D} \approx 0.5927$ for the 2D lattice with 4-connectivity (infinite structured grid in 2D) and the site percolation threshold $p_c^{3D} \approx 0.3116$ for the 3D lattice with 6-connectivity (infinite structured grid in 3D) are determined with this procedure [56].

In this context, Stauffer and Aharony [199] suggest setting p_c simply where a percolation function ranging from 0 to 1 first assumes the value 0.5. Similarly, they propose to define the width of the transition Δ as the interval where the function ranges between either 0.1 and 0.9, or 0.2 and 0.8, both of which they found to yield empirically reasonable estimates. Ziff [251] gives an overview of more involved methods that can estimate precise values with low estimated error but incorporate several other theories regarding the size and growth of connected components.

In this work, we consider general sampled scalar fields defined on structured grids, i.e., on finite lattices of the same size and connectivity as the ones analyzed through Monte Carlo experiments. In contrast to uniform random data, general scalar fields may have a data range that is not between zero and one, and their data distribution is seldom uniform. We explore how such different characteristics influence the shape of the percolation function and otherwise give the following contributions:

Key Contributions with their Contribution Areas

Mapping of the tree data to 1D: We discuss the consequences of computing percolation functions on sampled data. This includes how the dimensions of the grid influence the percolation function. We also describe a normalization of the input data that is crucial for comparing percolation functions between different data sets and to the theory. We propose a method for analyzing the percolation function and its features, combined with a comprehensive visualization for parameter- and time-dependent data sets.

Evaluation of our method: In our experiments, we find that our normalization scheme indeed aids in the comparison between data sets and the relation to infinite graphs.

Experiments on real-world data: We investigate the sensitivity of the percolation function to the amount of structure in data by designing a family of Gaussian random fields with varying degrees of structure. On a simulated turbulent duct flow data set, we observe that the critical values are stable and consistent across time.

4.2 Method Overview

To derive a percolation function on a general scalar field resembling the percolation probability $P(p)$, we need to consider that the data is finite and that the data may have an arbitrary value distribution.

Excluding the Extremes of the Value Range

The finite nature of the data affects the value range suitable for computing the percolation function $P_{\max}(p)$. Consider the evaluation on the entire data value range of a scalar field, i.e., the interval $[p_{\min}, p_{\max}]$. Processing only the first vertex at $p_{\max}$ yields a single component; thus, $P_{\max}(p_{\max}) = 1$. However, the existence of an infinite cluster in an infinite domain would have probability $P(p) = 0$. Even for subsequent vertices, volume computations remain rather erratic. They depend only on a few data samples. We obtain reliable values after having iterated over a sufficiently large number of data points. In this work, we therefore start to record the percolation function only after having reached a value $p_{\max} - \epsilon$ where ϵ is chosen so that the excluded value range $[p_{\max} - \epsilon, p_{\max}]$ amounts to the highest 1% of all data points.

Histogram Normalization

To account for varying data distributions, we propose a simple normalization scheme. In percolation theory, the parameter p refers to the probability of a site or vertex in a lattice being open. Sampling uniform random data with that probability will result in approximately p percent of open sites. However, for an arbitrary data distribution, values and percentages do not align. Thus, we need an additional step to adapt the input value of the percolation function to align it with the infinite case. This can be done by also recording the function in terms of percentages. Hence, instead of setting $p = s(\mathbf{x})$ as in previous work such as by Moisy and Jiménez [141], we set p in relation to the fraction of vertices in the superlevel set, i.e., V_{total} divided by the total number of vertices. For the remainder of this chapter, p denotes the percentage of vertices in the superlevel set, and $\bar{p}$ will be used to refer to a normalized and inverted version of the scalar data value $s(\mathbf{x})$. An example of how this affects sampling locations and the percolation function is given in Figure 4.1.

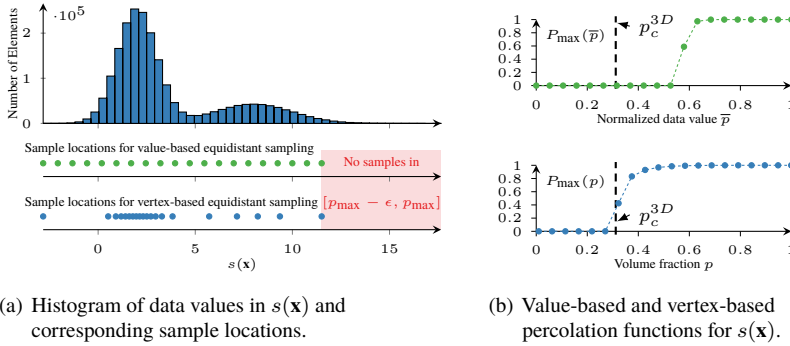


Figure 4.1: Histogram, sampling schemes, and percolation function for a scalar field $s(\mathbf{x})$ randomly sampled from a mixture of two Gaussians on a grid of size 128^3 . Either sampling method yields a sharp transition in the percolation function. However, the sharp transition in the value-based sampling stems only from the large fraction vertices that is processed around the transition. The value at the transition is far from the theoretical threshold p_c^{3D} for 3D data, hindering comparison between data sets and to known results on infinite graphs. Vertex-based sampling facilitates better comparability.

Analysis of a Single Percolation Curve

In order to determine the critical value p_c and the width Δ of the percolation transition, we approximate our result with a function that is directly parametrized by these values and captures the general shape of the percolation function. Since the percolation function in purely random data follows an S-shape, suitable candidates come from the family of sigmoidal functions. Due to its prevalence in the analysis of percolation curves from Monte Carlo simulations [171, 249], we make use of the *error function* $\text{erf}(x)$. The function ranges between -1 and 1, is monotonically increasing, symmetric with respect to the y -axis, and has its point of maximal slope at $x = 0$. By scaling and shifting the function, we can arrive at a function with an inflection point and maximal absolute slope at p_c and yields values at p_c , $p_c - \Delta$, and $p_c + \Delta$ that are close to the ones suggested by Stauffer and Aharony [199].¹

$$P_{\text{erf}}(p, p_c, \Delta) = \frac{1}{2} \left(1 + \text{erf} \left(\frac{p - p_c}{\Delta} \right) \right) \quad \text{with } \Delta > 0. \quad (4.1)$$

This function is thus suitable to be fit to our computational result $P_{\max}$ in order to estimate p_c and Δ . We use a nonlinear least-squares fitting algorithm available

¹ More precisely, the values are $P_{\text{erf}}(p_c, p_c, \Delta) = 0.5$, $P_{\text{erf}}(p_c - \Delta, p_c, \Delta) \approx 0.07865$, and $P_{\text{erf}}(p_c + \Delta, p_c, \Delta) \approx 0.92135$.

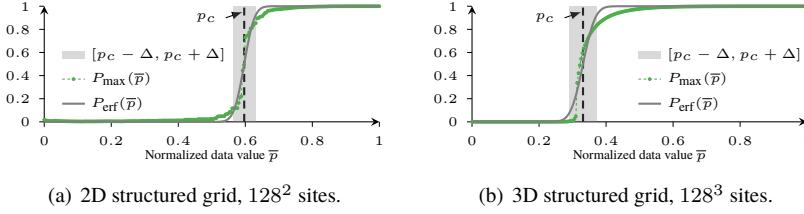


Figure 4.2: A fit of the function P_{eff} (gray) to the percolation functions P_{max} (green) for uniform random noise in 2D and 3D structured grids allows estimation of the percolation threshold p_c and transition width Δ .

in SciPy [110] to perform this step. The fitting algorithm uses an initial guess for p_c and Δ as a starting point. We set these values by estimating p_c as the point of maximal slope for a fitted polynomial and Δ as the half point of the interval where that polynomial ranges between 0.1 and 0.9. Figure 4.2 shows the fitted function P_{eff} for uniformly randomly sampled data in 2D and 3D. In both cases, the fitted curve nicely resembles the shape of the sampled percolation function. We observe that the percolation function is more asymmetric in the 3D case and deem it an interesting direction for future work to investigate alternative estimation schemes that better capture this asymmetric nature of the curve.

Analysis of Percolation Curve Ensembles

For the analysis of multiple percolation curves for a data set varying over a parameter t , such as time or another dynamic inducing dimension, the procedure for computation and analysis of the percolation function is repeated for every sample of t . To easily assess the development of p_c and Δ over parameter t while still seeing the connection to the sampled curves, we visualize all three together in a heatmap. The heatmap consists of texels, one for each sample (p, t) with its color encoding the function value $P_{\text{max}}(p)$. One axis of the heatmap corresponds to threshold p , whereas the other varies over parameter t .

4.3 Evaluation and Results

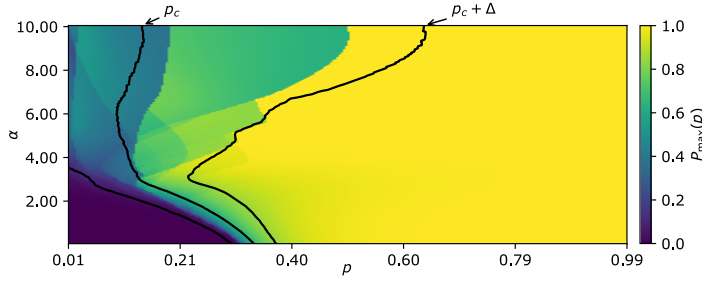
We evaluate our computation and analysis procedure with several experiments. We use different random data sets, both with uniform distribution and varying structure, as well as the two turbulent data sets we also discussed in Paper B.

First, we randomly sample uniform noise of dimension $128 \times 128 \times L_z$ for increasing values of L_z . This essentially alters the dimensionality of the data

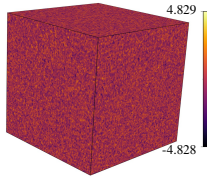
set from 2D to 3D, and we would expect the percolation threshold to behave accordingly. Indeed, we find that the estimated value p_c for $L_z = 1$, i.e., the 2D case, is close to the site percolation threshold for the infinite square lattice $p_c^{2D} \approx 0.5927$, and p_c moves quickly toward the threshold for the infinite cubic lattice $p_c^{3D} \approx 0.3116$ as we increase the dimensionality. Note that it is possible that data intrinsically has different dimensionality than the underlying lattice. If just replicating the same 2D slice over the third dimension, we expect p_c to still be close to the theoretical two-dimensional value.

To explore the impact of structure in a data set, we employ *Gaussian Random Fields* (GRFs) [129]. A GRF is a stochastic process from which samples can be efficiently generated with a fast Fourier transform of Gaussian noise multiplied by the power spectrum $P(k) = k^{-\alpha}$ for frequency k in Fourier space. Ultimately, a GRF enables us to generate a family of scalar fields with varying degrees of structure, depending on α . For low values of α , $P(k) \approx 1$ irrespective of k and the multiplication has little effect, leading to a scalar field composed of white noise. For higher α , multiplication with $P(k)$ causes lower frequencies to become more pronounced, thus introducing more structure into the data. We make several observations about the percolation threshold p_c and the width of the percolation transition Δ in relation to α for the example GRF shown in Figure 4.3(a). First, while all curves end with $P_{\max}(p) = 1$, not all start near the minimal value 0. This is the case once the degree of structure in the data is large enough such that at least one large connected component already exists at a low fraction of processed data samples. At approximately that point, no classic sharp percolation transition is visible anymore either. Instead, we see that several small discontinuities form that turn into large disconnected segments as α increases even further. Each such visible discontinuity marks a merge of two already large components, resulting in a significant increase of $P_{\max}(p)$ at the merge. Following a merge, the relative volume of that component decreases again as it grows to a lesser degree than other structures. This continues until another merge occurs. Eventually, with no values around zero and considerable jumps in the function, the percolation value p_c has no connotation to “point of steepest decrease” and the width Δ of the fitted function grows substantially as well. In general, these observations indicate a strong correlation between the level of structure and the shape of the percolation function.

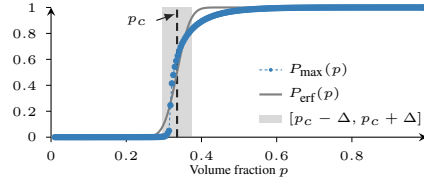
In the previous chapter, we showed how percolation analysis aids in identifying suitable thresholds for coherent structures in turbulent flows. When a flow is time-dependent, thresholds are commonly determined by averaging over several time steps. Using our proposed automated analysis and time-based visualization, we can assess whether this is reasonable to do. Figure 4.4(a) depicts the percolation function for 2000 time steps of the *Duct* flow giving an overview for a long time



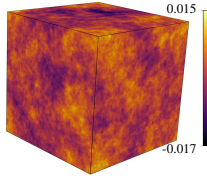
(a) Percolation function $P_{\max}(p)$ for generated GRFs with power spectrum $P(k) = k^{-\alpha}$ of dimension 128^3 and multiple $\alpha_i \in [0, 10]$ with $\alpha_{i+1} - \alpha_i = 0.05$.



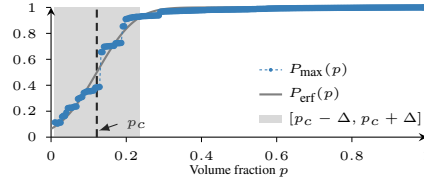
(b) GRF for $\alpha = 0.0$.



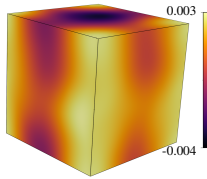
(c) Percolation function for the field in Figure 4.3(b).



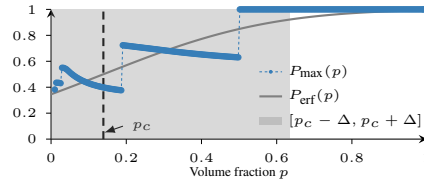
(d) GRF for $\alpha = 3.5$.



(e) Percolation function for the field in Figure 4.3(d).



(f) GRF for $\alpha = 10.0$.



(g) Percolation function for the field in Figure 4.3(f).

Figure 4.3: A series of percolation functions for random data where a parameter α steers the degree of structure. We observe the transition until about $\alpha \approx 3$, at which point $P_{\max}(0)$ does not start from zero anymore, large discontinuities form and the width Δ of the fitted function increases considerably.

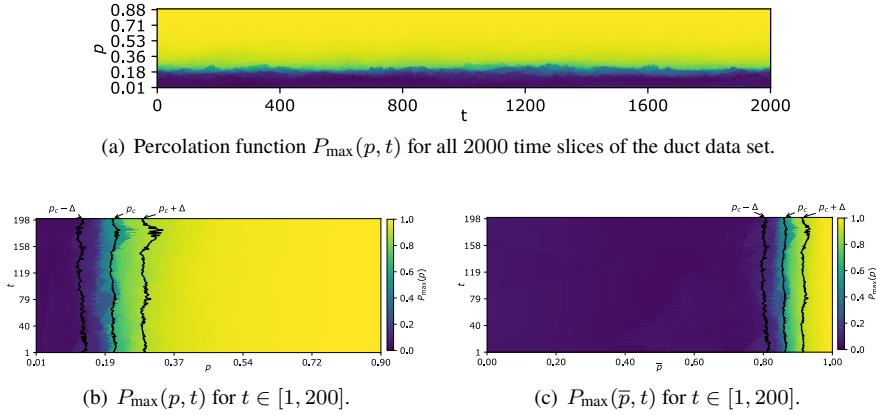


Figure 4.4: Percolation function for the Reynolds-stress field of the *Duct* data set. The first visualization gives an overview of all 2000 time slices, while the following plots depict the first 200 times. The second row show the difference in vertex-based (left) and value-based (right) for this data set with clear differences in the value at which the transition occurs.

span. For a more detailed view, Figure 4.4(b) shows the first 200 time slices together with approximated percolation threshold p_c and the transition width Δ . These visualizations reveal only minor variations in the percolation function and reveal that the temporal evolution of the percolation function is rather stable, making averaging a suitable procedure. Instead computing the percolation function for normalized scalar values, as shown in Figure 4.4(c), results in the same kind of stability. However, the actual parameters are vastly different. This is also apparent in an additional experiment, where we remove any structure from the data by permutation. Randomly permuting the input data keeps the histogram intact, but removes any spatial correlation. In this case, percentage-based evaluation yields a stable percolation transition in the vicinity of the theoretical value of a 3D lattice, whereas the value is notably shifted for evaluation based on scalar values. This confirms that histogram normalization is effective in enabling the comparison of real data with the theoretical results.

Finally, we repeat our analysis of the *Isotropic Turbulence* flow (see Figure 3.5, [248]) with our normalization scheme. For this data set, we find that the percolation transition is much more pronounced with our normalization scheme. However, we also see that the transition occurs at much larger values of p compared to the theoretical value. This indicates that the data does have some structure.

5

Temporal Treemaps

Visualization of the percolation function over time provides some indication of how a feature hierarchy evolves in terms of its connected components at different thresholds. However, neither the sizes of the individual components nor the nature of their interactions are reflected. Showing these in detail is one of the subjects of Paper A, which we summarize in this chapter. We consider the case of temporally evolving trees, where inner node values are a sum of their children's values (see Equation (2.1)). In contrast with previous work, we can work with data exhibiting topological events such as merges, splits, appearances and disappearances. We only exclude moves, i.e., cases where a child changes its parent. From this follows that other topological events occur only within a hierarchy layer.

5.1 Motivation

As we have seen in Section 2.3, visualizing a series of evolving trees is challenging. With animation, it is difficult to observe the data in its entirety. With a static visualization, the limited space available in a 2D image creates an additional hurdle toward a coherent visualization, especially when topological events such as merges and splits occur. Existing methods visualize this kind of data with static, nested, and space-filling visualizations where the tree at each time step is represented by a one-dimensional treemap [49, 135, 136]. However, these methods are based on layouts

that either do not support topological events [49] or prioritize their representation at the expense of accurately presenting the hierarchical nature of the data [135, 136].

In this work, we introduce *Temporal Treemaps*. Our method optimizes the layout for a sequence of 1D treemaps by deriving a set of constraints on the placement for each node, considering both topological events and tree structure. Specifically, we give the following contributions:

Key Contributions with their Contribution Areas

Optimization for stability: We present a novel layout algorithm for temporally evolving trees with changing topology and data. It considers the entire hierarchy and time span at once to produce a layout with as few intersections as possible. This is based on a combination of a heuristic and a simulated annealing optimization approach. The layout algorithm runs on the order of a few seconds.

Mapping of the tree data to 1D: We propose a rendering scheme that emphasizes the hierarchy through an adaptation of classic cushion treemaps [239]. This makes our method applicable to data sets where the sum of the children’s data equals the parent’s data.

Tree extraction and representation: We propose a data structure for temporally evolving trees that records only the changes to the tree. Besides its space efficiency, it drastically reduces the computation times of the layout algorithm.

Experiments on real-world data: We showcase the wide range of applicability using data from feature hierarchies, evolution of file system hierarchies, and world population.

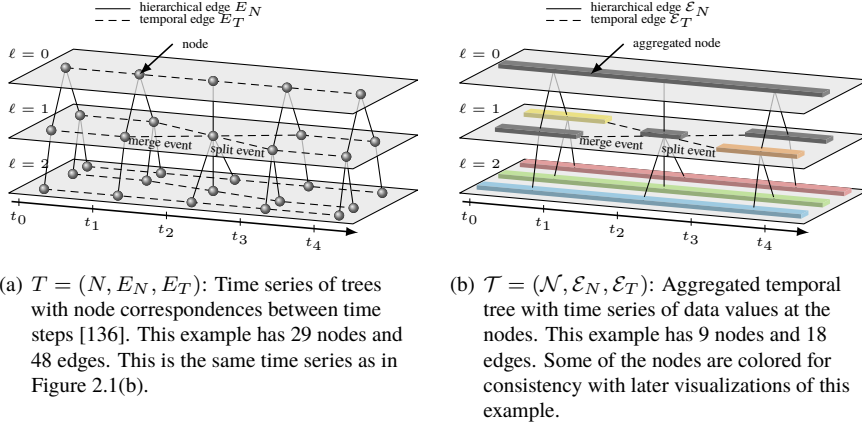
Comparison to other methods: For feature hierarchies, we compare our method against *Nested Tracking Graphs* [136] and observe significantly fewer intersections in our results.

Evaluation of our method: Finally, we run a parameter study comparing different input parameters of the optimization showing that it behaves quite stable on average.

5.2 Method Overview

Aggregated Data Structure

Operating on a dynamic tree $T = (N, E_N, E_T)$ consisting of a time series of individual trees is space-consuming and can lead to long runtimes for layout algorithms based on individual nodes. Therefore, we introduce the concept of an *aggregated temporal tree* $\mathcal{T} = (\mathcal{N}, \mathcal{E}_N, \mathcal{E}_T)$, where each node exists over a time



(a) $T = (N, E_N, E_T)$: Time series of trees with node correspondences between time steps [136]. This example has 29 nodes and 48 edges. This is the same time series as in Figure 2.1(b).

(b) $\mathcal{T} = (\mathcal{N}, \mathcal{E}_N, \mathcal{E}_T)$: Aggregated temporal tree with time series of data values at the nodes. This example has 9 nodes and 18 edges. Some of the nodes are colored for consistency with later visualizations of this example.

Figure 5.1: Aggregated temporal trees (right) encode the hierarchy with lower memory footprint than a time series of trees (left), which is especially relevant when only a few nodes change their data values in every time step, e.g., in a hierarchy based on a file system.

period $[t_a, t_b]$ and stores a series of data values for any time step where its value changes. Converting between T and $\mathcal{T}$ and vice versa is straightforward. Any two nodes that directly correspond to each other are aggregated into a single node in $\mathcal{N}$, and the edges in E_N are reduced accordingly to form $\mathcal{E}_N$. The remaining set of temporal edges $\mathcal{E}_T$ consists of only those edges that are part of merges or splits. The resulting encoding is compact and records only changes to topology and data. Figure 5.1 shows an illustration for the example in Figure 2.1(b). Note that all algorithms discussed here work on both the non-aggregated T and the aggregated $\mathcal{T}$ but do incur different computational costs. Most notably, aggregation reduces the computation times for layout optimization from minutes to seconds.

Optimization

Our data structure $\mathcal{T}$ is not only a tree, but also a hierarchically nested graph. A restriction of $\mathcal{T}$ to an individual hierarchy level ℓ is again a graph. Finding intersection-free graph layouts is challenging in general, but finding a good layout is particularly challenging with hierarchical nesting: The graph of hierarchy level ℓ shall be drawn inside the graph of hierarchy level $\ell - 1$. This means that any topological event in a layer $\ell - 1$ impacts the graph drawing in layer ℓ .

Lukasczyk et al. [136] present the first approach to drawing nested graphs. They independently optimize a layout for each hierarchy level and force the nodes to nest inside each other in the final drawing phase. While this produces optimal results

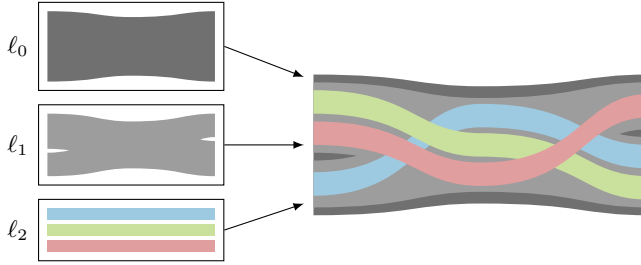


Figure 5.2: Lukasczyk et al. [136] calculate graph layouts for each hierarchy level independently with GraphViz [72]. While the layout for each hierarchy level has as few intersections as possible (left), the nested final visualization forces children to abide to the hierarchical nesting in T . Children are placed inside their parents', yet with the same order as in the level-wise layout. This may cause a number of intersections, as in this example (right), which has been created with the original software of [136].

in the scope of a single level, the final compositing step induces a larger number of intersections/crossings since the layout does not accommodate events in other hierarchy levels. An example is shown in Figure 5.2.

Our approach instead considers a bottom-up perspective by ordering the leaves of $\mathcal{T}$ and deriving the order of parents from their children. We accommodate both topological events in each hierarchy level and the hierarchical nesting by imposing constraints on the order of leaves. Each *ordering constraint* requires that a certain set of leaves are placed next to each other in a given time span, with no other leaves in between. *Hierarchical constraints* impose that all leaves reachable from any internal node are next to each other during the lifetime of that node. *Topological constraints* impose that all leaves reachable from the nodes involved in a merge and split are next to each other at the time step of the event. Figure 5.3 illustrates both types of constraints and a result when all constraints are met using the same example as shown in Figures 5.1(b) and 5.2.

Consider a temporal tree with a total of n leaves. Then the solution space of our optimization problem consists of all $n!$ possible permutations. Our goal is to find a permutation that violates no or only a small number of constraints. If an ordering violates no constraints, then the nested drawing of the temporal tree can be done without intersections.

We propose combining two optimization approaches to navigate this search space: A heuristic and *simulated annealing*. In both cases, a step in searching for an optimal solution consists of selecting a currently unfilled constraint and then adapting the leaf order so that the constraint becomes fulfilled. This involves

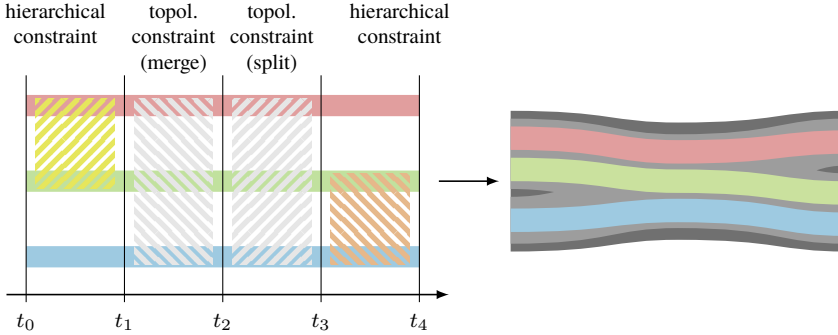


Figure 5.3: We want to derive an order for the red, green, and blue leaves of the temporal tree $\mathcal{T}$ that adheres to constraints due to the hierarchical nesting (yellow and orange constraints) and due to topological events (gray constraints). Fulfilling a constraint means that all involved leaves are placed next to each other during the time span at which the constraint is relevant. In this example, the three leaves can be properly sorted just considering the shown two hierarchical constraints. All (two) orders that fulfill these two constraints lead to an intersection-free layout.

moving all leaves that cause the constraint to be violated while keeping the order of the moving leaves intact. This ensures that other constraints are affected as little as possible. Nevertheless, the move of leaves may cause other constraints to become unfulfilled or fulfilled.

There are multiple ways to perform this move. The major difference between heuristic and simulated annealing lies in random selection. For the heuristic, we construct a *first in, first out* queue (FIFO) of constraints to iterate over and always pick the best new order in terms of the minimum number of violated constraints.

With simulated annealing, both constraints and a new order to fulfill the selected constraint are chosen at random. Simulated annealing is based on a number of parameters that steer whether this new order should constitute an improvement. Initially, a new leaf order that is worse than the previous one *may* still be accepted by simulated annealing. This allows escaping from local minima. As the algorithm progresses, this behavior is less likely to foster convergence.

For most data sets we tested, the heuristic already suffices to find an optimal solution, but in more complex settings, it tends to get stuck in loops where it fulfills and breaks the same cycle of constraints repeatedly. Here, simulated annealing, initialized with the result of the heuristic, can escape the loop and improve further upon the ordering.

Given an order of the nodes from the optimization, the next step is to assemble the final visualization. We target two rendering methods: the nested drawing akin

to Lukasczyk et al. [136] for feature hierarchies, as well as a cushion rendering that we will describe next.

Adaption of Cushion Maps

When data values of inner nodes are equal to the sum of data of its children, a nested drawing causes the leaves of the tree to consume the entire drawing space. To enhance the perception of the hierarchy beyond just color variations, we suggest an adaptation of the classic *Cushion Treemaps* by Wijk and Wetering [239]. The main idea is to add height to each node and simulate a virtual “landscape” that reveals the nesting information through shading. Essentially, each tree node is assigned a parabola with height depending on the hierarchy level of the node. Assuming that a node is placed between $[y_0, y_1]$ in a given time step, the parabola is given as:

$$\Delta z(y) = \frac{4f^\ell h (y - y_0)(y_1 - y)}{y_1 - y_0}, \quad (5.1)$$

where h is a base height for all cushions and $0 < f < 1$ determines the height decay over increasing hierarchical levels. A leaf is then assigned the sum of parabolas of all of its parents. In this process, we take special care to incorporate nodes’ splitting and merging by means of a smoothly merging/splitting cushion. This requires the introduction of some auxiliary vertices.

5.3 Evaluation and Results

We showcase results on data sets from different application areas and with different degrees of change in terms of topology and data.

The feature hierarchies for the *Viscous Fingers* and the *Cylinder* data set shown in Figures 5.4 and 5.5 contain all topological events apart from moves. Both figures include a side-by-side comparison with the respective result obtained with the layout algorithm by Lukasczyk et al. [136] and we can clearly see that our method is able to produce results with no or almost no intersections. Out of all tested data sets, the *Cylinder* data set is the only one that benefits from the additional use of simulated annealing. Thus, we use it to further investigate the stability of the optimization. Simulated annealing includes a stochastic component and three different parameters that steer its acceptance of suboptimal solutions. It is thus important to consider multiple different runs. Here, we test a total of 360 runs ($3 \times 4 \times 3 = 36$ parameter combinations with 100 runs each), and although we generally observe that the majority of the runs performs well, there is some spread. Fortunately, a single run takes less than a second, and it is easy to include multiple independent runs in practice.

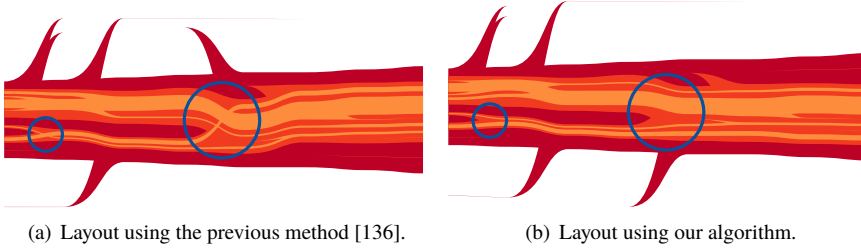


Figure 5.4: Closeup of an excerpt of the *Viscous Fingers* data set of [136] for the time range $[40, 47]$. Our layout is free from intersections, whereas some bands cross each other for the original method.

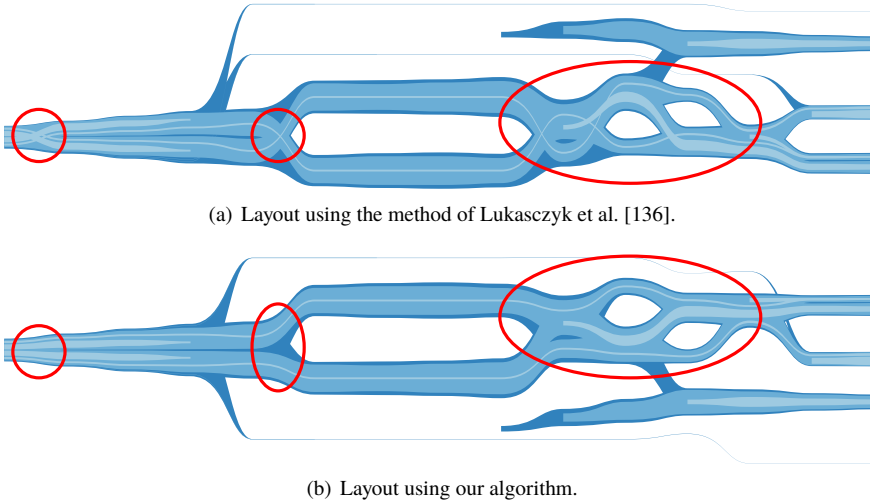


Figure 5.5: Layout of a nested tracking graph for the *Cylinder* data set covering about 15 time steps. Our method produces a layout with a single intersection, whereas several intersections occur with the original method for *Nested Tracking Graphs* [136]. The red circles indicate regions with differences.

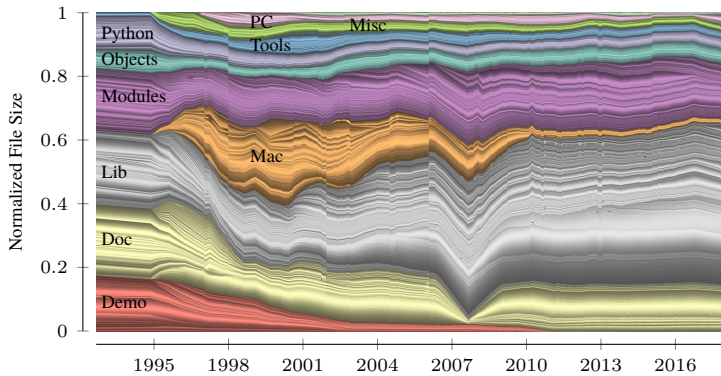


Figure 5.6: Evolution of the *CPython* repository from 1992 until 2018. Major structural changes are identifiable. For example, some folders disappear almost entirely in relation to others growing continuously.

The file system hierarchy example in Figure 5.6 shows the development of the *CPython* source code repository [48] since its early development. It spans the time between August 1992 and March 2018. The temporal tree is obtained by sampling every 1000th of the 101181 commits and consists of 372127 nodes in non-aggregated form and 11643 nodes in aggregated form. This hierarchy does not contain merges and splits but exemplifies that our method can be applied to large hierarchies. Indeed, our proposed colored cushion rendering effectively reveals major trends and interesting events.

Our method supports different types of data normalization. Figure 5.7 illustrates this, showing the development of the *World Population* until the year 2100 as projected by the United Nations in *World Population Prospects: The 2017 Revision* [215, 216]. The projection model for the data shown here is the one deemed most likely in terms of assumptions on the temporal development of fertility, mortality, and net migration.

Finally, we use data about *Europe's Population* to highlight how the adapted cushion rendering behaves for topological events. Europe has been subject to a number of topological events in the 20th century, with countries merging or splitting. Our visualization of the latest available demographic data [214] in Figure 5.8 adheres to such events like the German unification in 1990 and the dissolution of the Soviet Union in 1991. The cushion rendering is effective in conveying these events.

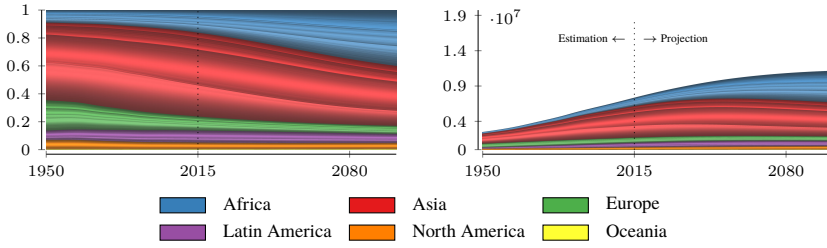


Figure 5.7: We support normalization of the data with our cushion rendering method. The data on the left is normalized per time step, whereas the right shows the data in absolute terms. Both plots show the development of the *World Population* until 2100 according to the *Medium* population projection model in a United Nations report [215] representing the median of several thousand projection models for fertility and mortality.

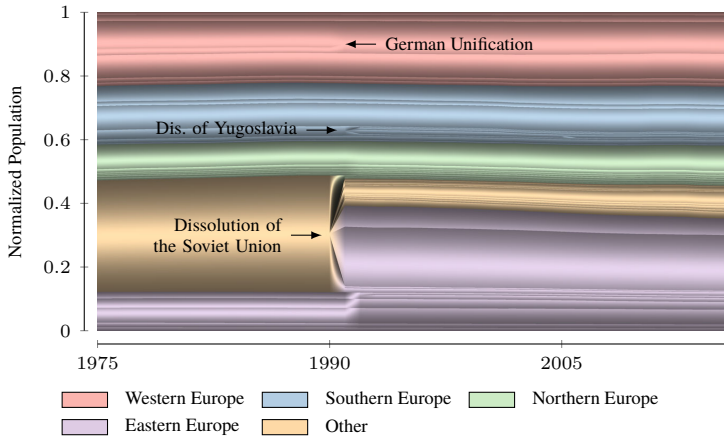


Figure 5.8: Development of *Europe's Population* between 1975 and 2016. The cushion transitions highlight a number of splitting and merging events on the country level. Additionally, we exemplify here that moving nodes, albeit only nodes moving within a hierarchy layer, can be represented. This is accomplished by pre-processing the temporal tree by splitting the parents of moving nodes into parents before and after the move. For data with many moves, use of this approach would diminish the compactness of our aggregated data structure.

6

Temporal Merge Tree Maps

The method presented in Paper A/Chapter 5 constitutes an overview of the evolution of sublevel and superlevel sets and their nesting hierarchies. However, it is not applicable to merge trees. Topological events for evolving merge trees are not restricted to discrete levels, and their accurate determination is computationally expensive [152]. Related methods [135, 238] thus calculate tracking information for several user-selected thresholds instead. This chapter presents *Temporal Merge Tree Maps*, the method we introduce in Paper D. With this method, we take an alternative approach without calculating explicit tracking information.

6.1 Motivation

Temporal Treemaps and *Nested Tracking Graphs* [136] show how the features evolve in relation to selected thresholds. This neglects the data context in which the individual features live. For example, data value distributions within each feature or data value changes between features are not shown. On the other hand, alternatives where all individual data samples are fully preserved (see Section 2.4), do not keep features intact. This makes certain analysis tasks impossible that are supported by the feature-based approach, e.g., counting the number of features. We present a method that combines the advantages of both types of approaches. Specifically, our contributions are as follows:

Key Contributions with their Contribution Areas

Mapping of the tree data to 1D: We propose a feature-based domain linearization that keeps features intact and preserves their context by involving all data samples.

Optimization for stability: We address temporal coherence by formulating a discrete optimization problem and introduce a practical greedy heuristic that allows us to find a solution with little computational effort.

Comparison to other methods: We provide several comparisons to linearization and feature-based methods.

Evaluation of our method: We study the stability of the optimization for a suitable data set with predictable output and use a large-scale data set to investigate the effect of topological simplification to our method.

Experiments on real-world data: We showcase the application of our method to 2D and 3D time-dependent data sets.

6.2 Method Overview

The inputs to computing a *Temporal Merge Tree Map* are a series of n scalar fields $s(\mathbf{x}, t)$ with $t \in [t_0, \dots, t_{n-1}]$ and m data samples per time step, as well as a series of the n respective augmented merge trees $[M_0, \dots, M_{n-1}]$ either based on sublevel or superlevel sets. The latter are computed with the open-source library *Topology Toolkit (TTK)* [208].

The algorithm to compute the final visualization has three main components: Mapping each time step, optimizing temporal coherence, and assembling the visualization. We briefly summarize them in the following.

Mapping a Single Time Step

Each merge tree is linearized by a procedure that essentially reverses the merge tree extraction algorithm. Figure 6.1 exemplifies this for the augmented merge tree from Figure 2.4. We begin at the root of the merge tree and iterate over subsequent regular nodes and supernodes with depth-first traversal. While doing so, each node is assigned a location in 1D within a range $[x_L, x_R]$ of thus far unassigned sample locations. Initially, this range is $[0, \dots, m-1]$, representing the interval of indices for the m data samples. This aids direct conversion into pixels. While traversing one superarc, nodes are placed to the left and right ends of the currently unassigned range (see Figures 6.1(a) and 6.1(c)). This ensures that the nodes remain part of the same single connected component. Supernodes are placed in the center of the unassigned range, leaving enough space to the left and right for their children (see Figures 6.1(b) and 6.1(c)). We can choose which child arc to place to the left

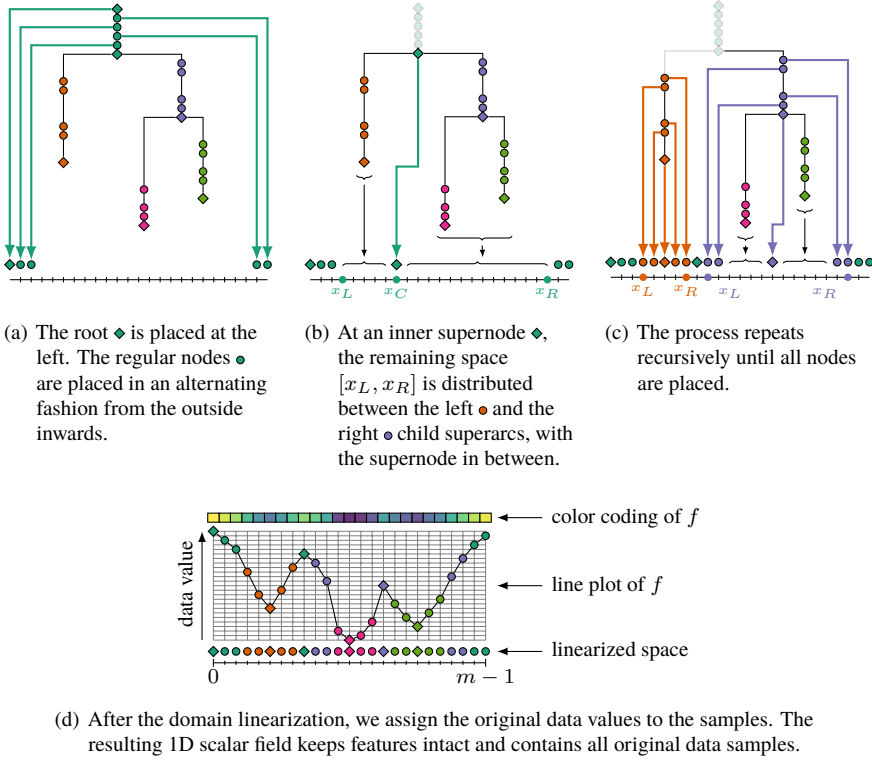


Figure 6.1: We linearize each time step of our input scalar field (2D/3D) with the help of a depth-first traversal of its augmented merge tree. Importantly, under very mild assumptions, the resulting 1D scalar field has the same merge tree as the original data, i.e., we keep features intact during our linearization process.

and which to the right. This flexibility enables us to maximize temporal feature coherence of adjacent time steps. The placement of the saddle in between its children causes the merging behavior of connected components in one dimension to be the same as in the original data. This holds irrespective of the traversal order we have chosen for the children, Associating each linearized node in 1D with its original data value yields a 1D scalar field that we can visualize by mapping values to colors as shown in Figure 6.1(d).

Overall, this procedure leads to a 1D linearization where linearization and original data have the same merge tree assuming the underlying scalar data is free from degenerate cases like plateaus or multi-saddles. Even when these cases occur, the resulting differences are minor (see the supplemental material for Paper D).

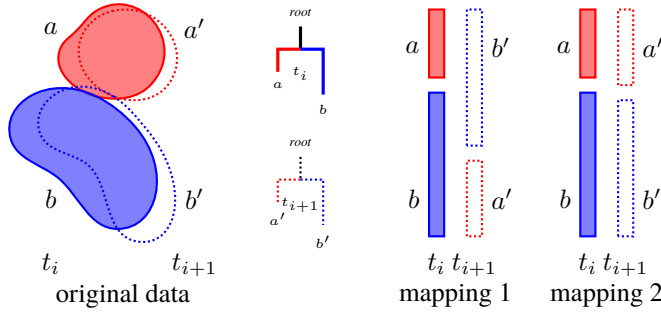


Figure 6.2: Linearization of two consecutive time steps follows the principle that overlap in the linearized domain and overlap in the original data should match as closely as possible. Here, mapping 2 is favored over mapping 1.

Mapping all Time Steps

Features moving in a scalar field are inherently spatially and temporally coherent, i.e., their value and position changes are smooth. Linearizing the data and thus compressing the data dimensions into one restricts the possible spatial movement and extent of features. Despite this, our goal is to portray the dynamics of the data as accurately as possible.

We choose the difference in overlap of subtrees as a metric to measure how well temporal dynamics are captured. A placement that causes the overlap in one dimension to closely resemble the overlap in the original data is preferred, as we can see in Figure 6.2. Formulated as an optimization problem, we seek to find optimal traversal orders for all merge trees that minimizes E , an objective function made up of the sum of *differences* in subtree overlap over all pairs of subtrees in all pairs of consecutive time steps:

$$E = \sum_{t=t_0}^{t_n-2} \sum_{S_j \in M_t} \sum_{T_k \in M_{t+1}} \left(p^{nD}(S_j, T_k) - p^{1D}(S_j, T_k) \right)^2. \quad (6.1)$$

The search space of the associated optimization problem is formed by all possible merge tree traversal orders, i.e., it contains a total of 2^N possible configurations, where N is the total number of inner supernodes over all time steps. Such a large search space is impossible to fully enumerate even for moderately sized data sets.

We propose a greedy heuristic that is based on making locally optimal decisions while iterating over the sequence of trees from time step to time step and per inner supernode. It is initialized by choosing one time step t for which we regard the

traversal order of the merge tree M_t as fixed. For the next (and previous) time step, we start at the root of the merge tree and iterate over its nodes while picking the best traversal order for each inner supernode. To do so, we compute the contribution of the supernode’s children to the overall objective from Equation (6.1) for the two possible traversal orders and pick the one with the smaller contribution. This procedure leverages that mismatches higher up in a merge tree cause larger errors compared to mismatches in lower parts of the tree.

Assembling the Final Image

To create an image from the aligned linearizations, we use a combination of sub-sampling and super-sampling. When the number of data points in a time step m significantly exceeds the dimensions of a display or a GPU texture size limit, we use subsampling to reduce the number of pixels to below the GPU texture size limit. This is 4096 on our hardware. If the number of time steps n is significantly lower, we upsample the data along the temporal dimension with linear interpolation.

6.3 Evaluation and Results

For the evaluation of our method, we use a mixture of 2D and 3D data sets and include one example each here.

The temporal merge tree map for the two-dimensional *Storms* data set is shown in Figure 6.3. Our visualization facilitates data analysis tasks such as counting features, comparing their life time, the size of their footprint, and so on. In addition, it is possible to identify large-scale trends. For example, a single color dominates the entire spatial domain on both December 26 and December 23.

The 3D flow around a *Cylinder* shown in Figure 6.4 exhibits periodic vortex shedding leading to the well-known von Kármán vortex street. Our temporal merge tree map is able to reveal the three different phases of the simulation.

The other data sets featured in the paper are used to either compare against other methods and to investigate aspects such as stability of the optimization or the use of topological simplification.

To compare against other domain linearizations, we apply our method to *Nucleon* and *Ring* data sets used by Franke et al. [68] and Zhou et al. [250] and incorporate our results in recreations of the result figures from these works. Both focus on space-filling curves and the direct comparison verifies that more structure is retained with our visualization. The corresponding visualizations can be found in the paper. We can also observe this in Figure 6.5(c), where a Hilbert curve is used to linearize the *Storms* data set in each time step.

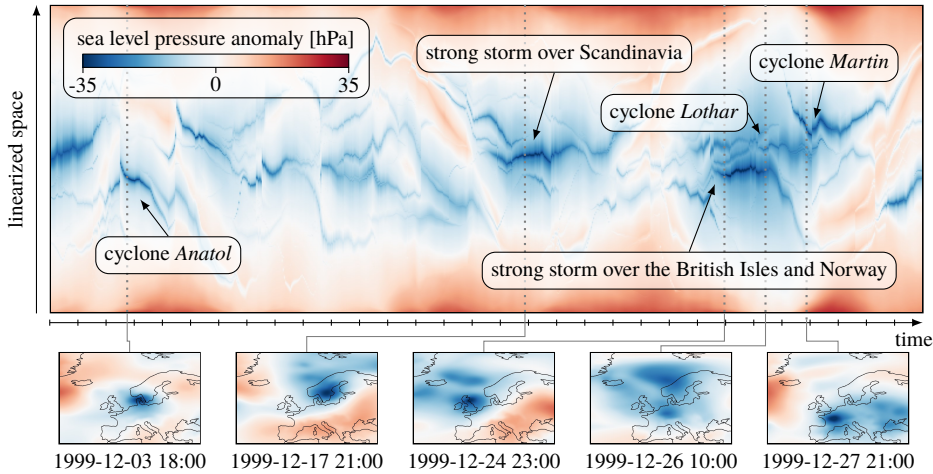


Figure 6.3: The shown *Storms* data set represents storm activity over Europe in December 1999. It covers a total of 744 time steps and can be computed in under 15 seconds, when utilizing some parallel processing. The visualization supports comparison of the dark blue features, representing cyclones, while still displaying the data context of each feature.

Figure 6.5 further includes two additional views of this data set to demonstrate differences with the fully feature-based approaches [136]. Figure 6.5(b) depicts a temporal tree map with a discrete colormap, and Figure 6.5(d) shows a *Nested Tracking Graph* using the same set of thresholds to define its layers. Both methods create a similar first-glance impression, but our method clearly shows a better temporal coherence.

In the following, we briefly comment on additional evaluations covered in the paper for which visualization are not included here.

To further contrast against threshold-based approaches, we additionally include results for four analytical 2D scalar fields that are indistinguishable with certain selected thresholds despite vastly different data distributions. The same set of scalar fields also highlights that some structural information, such as the shape of features, is not retained with our visualization.

Additionally, we conduct experiments to further investigate the effect of optimization and topological simplification. The symmetric *Benzenedata* set for which we expect a visualization reflecting the symmetry of the data aids in verifying that the optimization yields good results irrespective of the starting time step. The optimized temporal merge tree maps are also consistently better than assigning traversal orders randomly or omitting optimizing entirely.

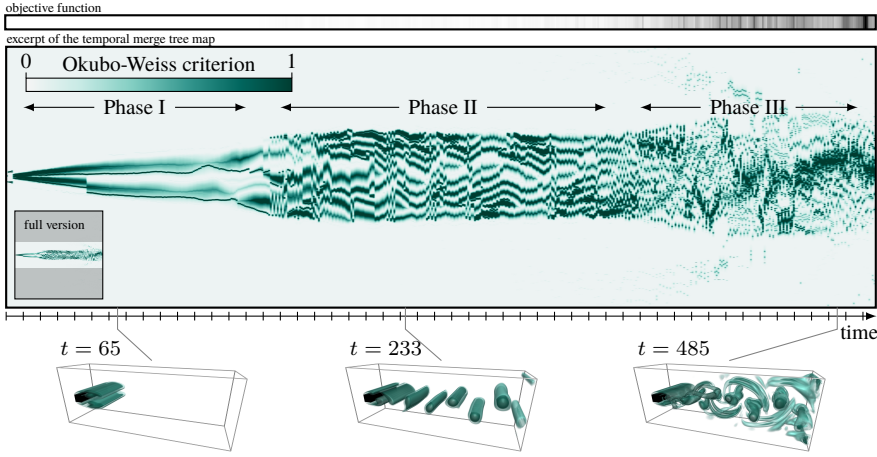


Figure 6.4: Our result for the *Cylinder* data set allows to correctly identify that the flow evolves with three distinct phases, each with increasing topological complexity in the underlying scalar field (Okubo-Weiss criterion restricted to values $Q > 0.04$): the fluid is still mostly at rest in the first phase, in the second phase primary vortex structures begin to shed, and in the last phase, we have arrived at a fully developed von Kármán vortex street.

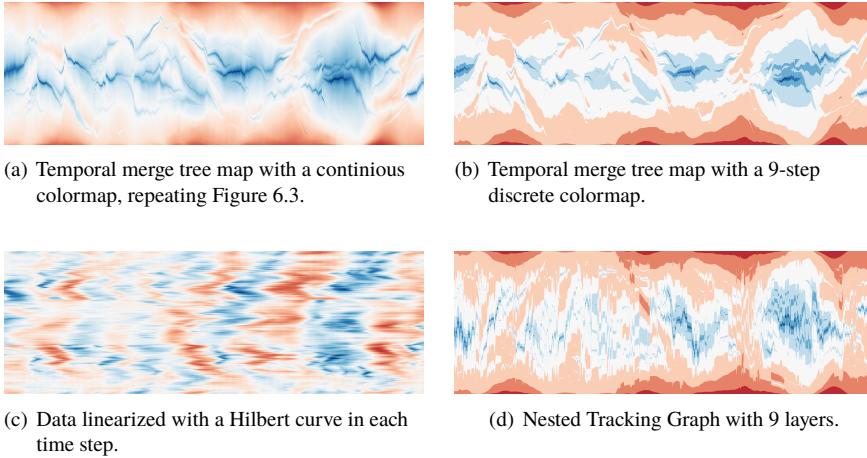


Figure 6.5: Several views of the *Storms* data set. A discretized temporal merge tree map bears resemblance to a *Nested Tracking Graph* with the same number of thresholds, but our method shows better temporal coherence. A space-filling domain linearization retains the overall data distribution, but not the individual features.

Finally, We study the effects of topological simplification using the *Tangaroa* data set and four different simplification thresholds. This particular data set is topologically complex. The unsimplified merge trees have up to 3655 supernodes, which leads to an unreasonably long running time for the optimization stage. Increased simplification leads to much lower computation times and also to less noisy and more distinct structures in the output images, while the overall Gestalt of the data is retained.

7

Concluding Remarks

In this thesis, we considered the challenge to statically visualize dynamic hierarchical data. To this end, we introduced novel visualization methods and expanded the applicability of existing methods to a more diverse set of hierarchical data.

With the *Temporal Treemap*, we presented a method featuring improved layout stability over *Nested Tracking Graphs* [136] and wider applicability due to an additional rendering scheme. The rendering scheme emphasizes both the hierarchical structure in each time step and the topological changes over time. The underlying layout is computed based on an optimized ordering of the leaves and efficient due to a compact temporal tree data structure that compresses redundancies where data or topology are static.

Our memory-distributed parallel algorithm for percolation analysis was the first algorithm to enable this type of analysis on large-scale scalar data exceeding typical shared memory. We exemplified this with a data set of dimensions 4096^3 , more than 6.8 billion data points, for which we were able to compute the percolation function on the order of minutes. Through a scaling analysis, we found that our algorithm scales well up to the point where the cost of synchronizing across processes outweighs the local workload. Therefore, the number of processes should be selected carefully and in relation to the size of the data set.

We further explored how results from percolation analysis can be better understood by proposing an alteration of the sampling, automatic extraction of threshold, and transition width. Our experiments confirmed that our adapted

normalization scheme shifts these values towards known ones for infinite graphs, as we would expect. Additionally, for data that deviates substantially from random data in spatial correlation, we observed that the percolation function exhibits clear discontinuities thus allowing one to estimate the degree of structure in a data set.

With the *Temporal Merge Tree Map* we presented a method that linearizes scalar data while retaining its merge tree. We found that the resulting visualizations can reveal more information about the structure of a data set than the alternatives of space-filling data traversal or feature-based methods. Our method considers a much larger number of time steps than exhibited by prior methods. Optimization for this method relied on the alignment of consecutive time steps and top-down traversal of individual trees.

For the works that incorporate optimization, we found that the respective approaches are stable with respect to the input parameters, i.e., similarly good results are obtained on the tested data sets when altering input parameters.

Our works have some limitations in common that relate to the size and complexity of the initial data and the fact that all of our methods result in a colored 2D image. The size of typical screens and hardware restrictions mean that there are only so many data points that we can show. For example, the GPU of our workstation has a texture size limit of 4096. The number of time steps or data points per time step, i.e., tree nodes, data points in a scalar field, or number of samples in the percolation function, can easily exceed the texture size. For all our methods, we make use of super- and sub-sampling strategies in line with the rendering modality to regardless fill a high-quality image. Nevertheless, this means that we may omit details by disregarding some of the data. Adding to this, human perception further limits how many entities a user can distinguish in terms of color variations. Some of these aspects can be alleviated by interactive exploration. We support this in the form of filtering on the input data and by exposing rendering settings such as color, lighting, and shading. For temporal trees, it is possible to select nodes to be excluded, to restrict tree depth, or filter any nodes below a certain size or that exist only for a few time steps. For trees stemming from scalar data, either the original scalar or the respective trees may be filtered or simplified. Finally, the topological complexity, i.e., the number of nodes and edges in temporal trees or the number of supernodes in merge trees, affects the runtime of the optimization algorithms. Both optimization approaches eventually struggle to find good solutions in reasonable time when these numbers start to exceed multiple hundreds of nodes per time step.

7.1 Future Work

Using the presented work as a starting point, there are several potential avenues for future work, which we will detail in the following.

Support for all Topological Changes

In Paper A, we assume that children do not change their parent. This means that moving nodes are not supported out of the box. Only for those moving within one hierarchy layer, we provide an ad-hoc solution (see Figure 5.8) based on introducing splits for all involved parents, noting that this comes at the expense of a less compact hierarchy representation. However, other patterns of moving nodes are common in hierarchies that do not originate from hierarchical scalar features. For example, files are often moved into newly created subfolders, and source code snippets are often encapsulated by additional conditionals or loops. Directly supporting this case would be a welcome extension, but is not straightforward. A node moving across hierarchy layers will always create a visual artifact in the form of an abrupt variation in the cushion or an intersection with other bands. An improved layout optimization would need to take this into account. Subsequent work that picked up this topic by Bolte et al. [21] does support moving nodes. However, the work operates in a slightly different context and mainly aims to highlight the hierarchy in each time step. Especially, the order of nodes is regarded as given and thus does not incorporate optimization.

Optimization

Similar to the dynamic visualization approaches we surveyed in Section 2.3.3, our layout optimization strategies either adopt a global view of the problem (Paper A) or operate incrementally on a time step to time step basis (Paper D). Additionally, they are based on objectives that pertain directly to the visual quality of the result. However, in Figure 6.5, we observe that a discretized version of a temporal merge tree map can emulate a good-quality temporal treemap. This suggests that a cross-over of the different layout approaches may not only enable the applicability to larger and more complex data, but improve visual results more generally. Incorporating aspects of the objective from Paper D into the currently discrete objective for temporal treemaps could be accomplished by assigning additional weights to constraints, thus measuring the severity of unfulfilled constraints. Aside from an adaptation of the objectives, it would also be interesting to explore alternatives for the navigation of the search space. We took initial steps towards this in a preliminary study in relation to Paper D, where we explored a formalization of the optimization problem in Equation (6.1) with a constraint modeling language in the hope that this would allow us to easily compare global and time-step-based navigation of the solution space with general-purpose constraints solvers. However, the overhead of translating into this language and invoking the external constraint solver incurred such a significant increase in runtime that further analysis was hindered. Nevertheless, we deem a more detailed exploration of this aspect to have

the potential to inform the development of alternative heuristics. These could then be implemented within our existing framework without the need to invoke external programs.

Use of Summaries to Steer Interactive Exploration

Assessing if a transfer function is suitable for visualization of the entire time span of an evolving scalar data set is a non-trivial task often involving many iterations of trial-and-error. Currently available graphical user interfaces for choosing transfer functions typically only provide the data histogram of a single time step and the overall data range as context. It would be interesting to investigate whether the parameter-free temporal overviews of scalar data in Papers B to D are a better-suited or complementary view for this task. In addition to the percolation function and the mapping with preserved topology, other one-dimensional functions in relation to the merge tree may be of interest for this. Bajaj et al. [11] suggest to include the gradient integral, the surface area, and the volume of isosurfaces in relation to the data value for similar purposes. A detailed evaluation of different complementary views would ideally also incorporate a comparison of the designed transfer function with results from *automatic* transfer function design, which can also be based on topological features [229].

Comparison of Feature Tracking Algorithms

An alternative view on the computation of a temporal merge tree map is to understand it as a dimensionality reduction technique that maps a dynamic scalar field of dimensions $nD + 1$ to $1D + 1$. This correspondence between positions can be used to display other information regarding the original locations, aside from the original scalar value. One particular example is the collection of results from different feature tracking algorithms based on the same scalar field as input. We saw in Section 2.2.2 that there are many general-purpose feature tracking algorithms. In addition, scientific fields often develop specialized algorithms incorporating domain knowledge. For example, the *Intercomparison of Mid-Latitude Storm Diagnostics (IMILAST)* project [147] compares 15 cyclone identification and tracking methods, 12 of which are entirely or partially based on the minima of mean sea level pressure. The inter-comparison includes heat maps of feature track density, statistical analysis of frequency, lifetime, intensity, and speed of the tracks, and track agreement between methods. Differences in time are shown only in the context of case studies or frequency. It would be interesting to explore what additional insight can be extracted from comparing temporal overviews of parameters such as track density or lifetime using the position correspondences from a temporal merge tree map.

References

- [1] A. Acharya and V. Natarajan. “A parallel and memory efficient algorithm for constructing the contour tree.” *2015 IEEE Pacific Visualization Symposium*. Apr. 2015, pp. 271–278.  10.1109/PACIFICVIS.2015.7156387.
- [2] J. Ahrens, B. Geveci, and C. Law. “ParaView: An End-User Tool for Large-Data Visualization.” In: *The Visualization Handbook*. Ed. by C. D. Hansen and C. R. Johnson. Elsevier München, 2005, pp. 717–731.  10.1016/b978-012387582-2/50038-1.
- [3] W. Aigner, S. Miksch, H. Schumann, and C. Tominski. *Visualization of Time-Oriented Data*. Springer, 2011. ISBN: 978-0-85729-079-3.  www.timeviz.net/.
- [4] G. M. Amdahl. “Validity of the single processor approach to achieving large scale computing capabilities.” *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*. ACM Press, 1967, pp. 483–485.  10.1145/1465482.1465560.
- [5] M. Atzori, **W. Köpp**, S. W. D. Chien, D. Massaro, F. Mallor, A. Peplinski, M. Rezaei, N. Jansson, S. Markidis, R. Vinuesa, E. Laure, P. Schlatter, and T. Weinkauff. “In-situ visualization of large-scale turbulence simulations in Nek5000 with ParaView Catalyst.” *Journal of Supercomputing* 78.3 (2022), pp. 3605–3620.  10.1007/s11227-021-03990-3.
- [6] M. Atzori, R. Vinuesa, A. Lozano-Durán, and P. Schlatter. “Characterization of turbulent coherent structures in square duct flow.” *Journal of Physics: Conference Series* 1001.1 (2018), p. 012008.  10.1088/1742-6596/1001/1/012008.
- [7] M. Atzori, R. Vinuesa, A. Lozano-Durán, and P. Schlatter. “Intense Reynolds-stress events in turbulent ducts.” *International Journal of Heat and Fluid Flow* 89 (June 2021), p. 108802.  10.1016/j.ijheatfluidflow.2021.108802.

- [8] M. Atzori, R. Vinuesa, and P. Schlatter. “Control effects on coherent structures in a non-uniform adverse-pressure-gradient boundary layer.” *International Journal of Heat and Fluid Flow* 97 (2022), p. 109036. [doi 10.1016/j.ijheatfluidflow.2022.109036](#).
- [9] B. Bach, P. Dragicevic, D. Archambault, C. Hurter, and S. Carpendale. “A Review of Temporal Data Visualizations Based on Space-Time Cube Operations.” *EuroVis - STARs*. The Eurographics Association, 2014. [doi 10.2312 / EUROVISSTAR . 20141171](#). [aviz . fr / ~bbach / spacetimecubes /](#).
- [10] Z. Bai, Y. Tao, and H. Lin. “Time-varying volume visualization: a survey.” *Journal of Visualization* 23.5 (June 2020), pp. 745–761. [doi 10.1007 / s12650-020-00654-x](#).
- [11] C. L. Bajaj, V. Pascucci, and D. R. Schikore. “The contour spectrum.” *Proceedings. Visualization '97 (Cat. No. 97CB36155)*. IEEE, 1997. [doi 10.1109/visual.1997.663875](#).
- [12] M. Balzer and O. Deussen. “Voronoi Treemaps.” *Proceedings of the 2005 IEEE Symposium on Information Visualization (INFOVIS'05)*. INFOVIS '05. Washington, DC, USA: IEEE Computer Society, 2005, pp. 49–56. [doi 10.1109/infovis.2005.40](#).
- [13] M. D. Bartolomeo and Y. Hu. “There is More to Streamgraphs than Movies: Better Aesthetics via Ordering and Lassoing.” *Computer Graphics Forum* 35.3 (2016), pp. 341–350. [doi 10.1111/cgf.12910](#).
- [14] D. Baur, B. Lee, and S. Carpendale. “TouchWave: Kinetic Multi-touch Manipulation for Hierarchical Stacked Graphs.” *Proceedings of the 2012 ACM international conference on Interactive tabletops and surfaces - ITS '12*. New York: ACM Press, 2012, pp. 255–264. [doi 10.1145/2396636.2396675](#).
- [15] B. B. Bederson, B. Shneiderman, and M. Wattenberg. “Ordered and quantum treemaps: Making effective use of 2D space to display hierarchies.” *ACM Transactions on Graphics* 21.4 (2002), pp. 833–854. [doi 10.1145 / 571647.571649](#).
- [16] M. Behrisch, M. Blumenschein, N. W. Kim, L. Shao, M. El-Assady, J. Fuchs, D. Seebacher, A. Diehl, U. Brandes, H. Pfister, T. Schreck, D. Weiskopf, and D. A. Keim. “Quality Metrics for Information Visualization.” *Computer Graphics Forum* 37.3 (June 2018), pp. 625–662. [doi 10.1111/cgf.13446](#).

- [17] K. Beketayev, G. H. Weber, D. Morozov, A. Abzhanov, and B. Hamann. “Geometry-Preserving Topological Landscapes.” *Proceedings of the Workshop at SIGGRAPH Asia. WASA '12*. 2012, pp. 155–160. [doi](#) 10.1145/2425296.2425324.
- [18] M. de Berg, K. Onak, and A. Sidiropoulos. “Fat Polygonal Partitions with Applications to Visualization and Embeddings.” *Journal of Computational Geometry* 4.1 (2013), pp. 212–239. [doi](#) 10.20382/jocg.v4i1a9.
- [19] E. Bertini, A. Tatu, and D. Keim. “Quality Metrics in High-Dimensional Data Visualization: An Overview and Systematization.” *IEEE Transactions on Visualization and Computer Graphics* 17.12 (Dec. 2011), pp. 2203–2212. [doi](#) 10.1109/tvcg.2011.229.
- [20] J. Bethge, S. Hahn, and J. Döllner. “Improving Layout Quality by Mixing Treemap-Layouts Based on Data-Change Characteristics.” *Vision, Modeling & Visualization*. Ed. by M. Hullin, R. Klein, T. Schultz, and A. Yao. The Eurographics Association, 2017, pp. 69–76. [doi](#) 10.2312/vmv.20171261.
- [21] F. Bolte, M. Nourani, E. Ragan, and S. Bruckner. “SplitStreams: A Visual Metaphor for Evolving Hierarchies.” *IEEE Transactions on Visualization and Computer Graphics* 27.8 (2021), pp. 3571–3584. [doi](#) 10.1109/TVCG.2020.2973564. [arXiv: 2002.03891 \[cs.GR\]](#). [🔗/CadanoX/SplitStreams](#).
- [22] R. L. Boyell and H. Ruston. “Hybrid techniques for real-time radar simulation.” *Proceedings of the IEEE Fall Joint Computer Conference*. ACM Press, 1963, pp. 445–458. [doi](#) 10.1145/1463822.1463869.
- [23] A. Brand, L. Allen, M. Altman, M. Hlava, and J. Scott. “Beyond authorship: attribution, contribution, collaboration, and credit.” *Learned Publishing* 28.2 (2015), pp. 151–155. [doi](#) 10.1087/20150211. [🔗 credit.niso.org/](#).
- [24] S. R. Broadbent and J. M. Hammersley. “Percolation processes: I. Crystals and mazes.” *Mathematical Proceedings of the Cambridge Philosophical Society* 53.3 (1957), pp. 629–641. [doi](#) 10.1017/S0305004100032680.
- [25] M. Bruls, K. Huizing, and J. J. van Wijk. “Squarified Treemaps.” *Proceedings of the 2000 Joint Eurographics and IEEE TCVG Symposium on Visualization, VisSym 2000* (May 29–30, 2000). Ed. by W. C. de Leeuw and R. van Liere. Amsterdam, The Netherlands: Eurographics Association, 2000, pp. 33–42. [doi](#) 10.1007/978-3-7091-6783-0_4.

- [26] C. Bu, Q. Zhang, Q. Wang, J. Zhang, M. Sedlmair, O. Deussen, and Y. Wang. “SineStream: Improving the Readability of Streamgraphs by Minimizing Sine Illusion Effects.” *IEEE Transactions on Visualization and Computer Graphics* 27.2 (Feb. 2021), pp. 1634–1643. doi 10.1109/tvcg.2020.3030404.
- [27] J. Buchmüller, D. Jäckle, E. Cakmak, U. Brandes, and D. A. Keim. “MotionRugs: Visualizing Collective Trends in Space and Time.” *IEEE Transactions on Visualization and Computer Graphics* 25.1 (2019); *Proceedings IEEE VIS*, pp. 76–86. doi 10.1109/TVCG.2018.2865049.
- [28] J. Buchmüller, U. Schlegel, E. Cakmak, E. Dimara, and D. A. Keim. “SpatialRugs: Enhancing Spatial Awareness of Movement in Dense Pixel Visualizations.” *EuroVis Workshop on Visual Analytics (EuroVA)*. 2020. doi 10.2312/eurova.20201078. arXiv: 2003.12282 [cs.HC].
- [29] M. Burch, T. Blascheck, C. Louka, and D. Weiskopf. “Visualizing hierarchy changes by dynamic indented plots.” *2014 International Conference on Information Visualization Theory and Applications (IVAPP)*. 2014, pp. 91–98.
- [30] M. Burch, N. Konevtsova, J. Heinrich, M. Hoferlin, and D. Weiskopf. “Evaluation of Traditional, Orthogonal, and Radial Tree Diagrams by an Eye Tracking Study.” *IEEE Transactions on Visualization and Computer Graphics* 17.12 (2011), pp. 2440–2448. doi 10.1109/TVCG.2011.193.
- [31] M. Burch, M. Raschke, A. Zeyfang, and D. Weiskopf. “A Scalable Visualization for Dynamic Data in Software System Hierarchies.” *2017 IEEE Working Conference on Software Visualization (VISSOFT)*. IEEE, Sept. 2017, pp. 85–93. doi 10.1109/VISSOFT.2017.16.
- [32] M. Burch, C. Vehlou, F. Beck, S. Diehl, and D. Weiskopf. “Parallel Edge Splatting for Scalable Dynamic Graph Visualization.” *IEEE Transactions on Visualization and Computer Graphics* 17.12 (2011), pp. 2344–2353. doi 10.1109/TVCG.2011.226.
- [33] M. Burch and D. Weiskopf. “Visualizing Dynamic Quantitative Data in Hierarchies. TimeEdgeTrees: Attaching Dynamic Weights to Tree Edges.” *IMAGAPP & IVAPP 2011 - Proceedings of the International Conference on Imaging Theory and Applications and International Conference on Information Visualization Theory and Applications*. 2011, pp. 177–186. doi 10.5220/0003323901770186.

- [34] L. Byron and M. Wattenberg. “Stacked Graphs - Geometry & Aesthetics.” *IEEE Transactions on Visualization and Computer Graphics* 14.6 (2008), pp. 1245–1252. doi 10.1109/TVCG.2008.166.
- [35] S. Camarri, M. V. Salvetti, M. Buffoni, and A. Iollo. “Simulation of the three-dimensional flow around a square cylinder between parallel walls at moderate Reynolds numbers.” *XVII Congresso di Meccanica Teorica ed Applicata*. 2005. doi 10.1400/56904.
- [36] G. Cantor. “Ein Beitrag zur Mannigfaltigkeitslehre.” *Journal für die reine und angewandte Mathematik (Crelles Journal)* 1878.84 (1878), pp. 242–258. PURL: PPN243919689_0084.
- [37] H. Carr. “Topological Manipulation of Isosurfaces.” PhD thesis. The University of British Columbia, 2004. doi 10.14288/1.0051287.
- [38] H. Carr, O. Rübél, G. H. Weber, and J. Ahrens. “Optimization and Augmentation for Data Parallel Contour Trees.” *IEEE Transactions on Visualization and Computer Graphics* (2021), pp. 1–1. doi 10.1109/tvcg.2021.3064385.
- [39] H. Carr, C. Sewell, L.-T. Lo, and J. Ahrens. “Hybrid Data-Parallel Contour Tree Computation.” *Proceedings of the Conference on Computer Graphics & Visual Computing. CGVC '16*. Bournemouth, United Kingdom: Eurographics Association, 2016, pp. 73–80. doi 10.2312/cgvc.20161299.
- [40] H. Carr, J. Snoeyink, and U. Axen. “Computing contour trees in all dimensions.” *Computational Geometry* 24.2 (2003): *Special Issue on the Fourth CGC Workshop on Computational Geometry*, pp. 75–94. doi 10.1016/S0925-7721(02)00093-7.
- [41] H. A. Carr, G. H. Weber, C. M. Sewell, O. Rbel, P. Fasel, and J. P. Ahrens. “Scalable Contour Tree Computation by Data Parallel Peak Pruning.” *IEEE Transactions on Visualization and Computer Graphics* 27.4 (Apr. 2021), pp. 2437–2454. doi 10.1109/TVCG.2019.2948616.
- [42] M. B. de Carvalho, B. S. Meiguins, and J. M. de Morais. “Temporal Data Visualization Technique Based on Treemap.” *20th International Conference Information Visualisation, IV 2016, Lisbon, Portugal, July 19-22, 2016*. Ed. by E. Banissi, M. W. M. Bannatyne, F. Bouali, R. Burkhard, J. Counsell, U. Cvek, M. J. Eppler, G. G. Grinstein, W. Huang, S. Kernbach, C. Lin, F. Lin, F. T. Marchese, C. M. Pun, M. Sarfraz, M. Trutschl, A. Ursyn, G. Venturini, T. G. Wyeld, and J. J. Zhang. IEEE Computer Society, 2016, pp. 399–403. doi 10.1109/IV.2016.65.

- [43] R. Chandra, R. Menon, L. Dagum, D. Kohr, and D. Maydan. *Parallel Programming in OpenMP*. Elsevier Science & Technology Books, 2000. ISBN: 9780080513539.
- [44] Y. Chen, X. Du, and X. Yuan. “Ordered small multiple treemaps for visualizing time-varying hierarchical pesticide residue data.” *The Visual Computer* 33.6 (June 1, 2017), pp. 1073–1084. doi 10.1007/s00371-017-1373-x.
- [45] F. Chevalier, D. Auber, and A. Telea. “Structural Analysis and Visualization of C++ Code Evolution Using Syntax Trees.” *Ninth International Workshop on Principles of Software Evolution: In Conjunction with the 6th ESEC/FSE Joint Meeting*. IWPSE ’07. 2007, pp. 90–97. doi 10.1145/1294948.1294971.
- [46] Y.-J. Chiang, T. Lenz, X. Lu, and G. Rote. “Simple and optimal output-sensitive construction of contour trees using monotone paths.” *Computational Geometry* 30.2 (Feb. 2005), pp. 165–195. doi 10.1016/j.comgeo.2004.05.002.
- [47] J. Cox, D. B. Karron, and N. Ferdous. “Topological Zone Organization of Scalar Volume Data.” *Journal of Mathematical Imaging and Vision* 18.2 (2003), pp. 95–117. doi 10.1023/a:1022113114311.
- [48] Python Software Foundation. *CPython Repository*. 2018. /python/cpython (visited on 2018).
- [49] E. Cuenca, A. Sallaberry, F. Y. Wang, and P. Poncelet. “MultiStream: A Multiresolution Streamgraph Approach to Explore Hierarchical Time Series.” *IEEE Transactions on Visualization and Computer Graphics* (2018). doi 10.1109/TVCG.2018.2796591.
- [50] W. Cui, S. Liu, L. Tan, C. Shi, Y. Song, Z. Gao, H. Qu, and X. Tong. “TextFlow: Towards Better Understanding of Evolving Topics in Text.” *IEEE Transactions on Visualization and Computer Graphics* 17.12 (Dec. 2011), pp. 2412–2421. doi 10.1109/TVCG.2011.239.
- [51] W. Cui, S. Liu, Z. Wu, and H. Wei. “How Hierarchical Topics Evolve in Large Text Corpora.” *IEEE Transactions on Visualization and Computer Graphics* 20.12 (Dec. 2014), pp. 2281–2290. doi 10.1109/tvcg.2014.2346433.
- [52] R. Dafner, D. Cohen-Or, and Y. Matias. “Context-based Space Filling Curves.” *Computer Graphics Forum* 19.3 (2000), pp. 209–218. doi 10.1111/1467-8659.00413.

- [53] J. C. Del Álamo, J. Jiménez, P. Zandonade, and R. D. Moser. “Self-similar vortex clusters in the turbulent logarithmic region.” *Journal of Fluid Mechanics* 561 (2006), pp. 329–358. doi 10.1017/S0022112006000814.
- [54] D. Demir, K. Beketayev, G. H. Weber, P.-T. Bremer, V. Pascucci, and B. Hamann. “Topology exploration with hierarchical landscapes.” *Proceedings of the Workshop at SIGGRAPH Asia on - WASA '12*. ACM Press, 2012. doi 10.1145/2425296.2425323.
- [55] I. Demir, C. Dick, and R. Westermann. “Multi-Charts for Comparative 3D Ensemble Visualization.” *IEEE Transactions on Visualization and Computer Graphics* 20.12 (2014), pp. 2694–2703. doi 10.1109/TVCG.2014.2346448.
- [56] Y. Deng and H. W. J. Blöte. “Monte Carlo study of the site-percolation model in two and three dimensions.” *Physical Review E* 72.1 (2005), p. 016126. doi 10.1103/PhysRevE.72.016126.
- [57] M.-S. Deroche, M. Choux, F. Codron, and P. Yiou. “Three variables are better than one: detection of european winter windstorms causing important damages.” *Natural Hazards and Earth System Sciences* 14.4 (2014), pp. 981–993. doi 10.5194/nhess-14-981-2014.
- [58] L. DeRose, B. Homer, D. Johnson, S. Kaufmann, and H. Poxon. “Cray Performance Analysis Tools.” In: *Tools for High Performance Computing*. Springer, 2008, pp. 191–199. doi 10.1007/978-3-540-68564-7_12.
- [59] S. Dutta and H.-W. Shen. “Distribution Driven Extraction and Tracking of Features for Time-varying Data Analysis.” *IEEE Transactions on Visualization and Computer Graphics* 22.1 (Jan. 2016), pp. 837–846. doi 10.1109/tvcg.2015.2467436.
- [60] S. Dutta, T. L. Turton, and J. P. Ahrens. “A Confidence-Guided Technique for Tracking Time-Varying Features.” *Computing in Science & Engineering* 23.2 (Mar. 2021), pp. 84–92. doi 10.1109/mcse.2020.3047979.
- [61] H. Edelsbrunner and J. Harer. *Computational Topology. An Introduction*. American Mathematical Society, Jan. 2010. ISBN: 0-8218-4925-5.
- [62] H. Edelsbrunner, D. Letscher, and A. Zomorodian. “Topological Persistence and Simplification.” *Discrete and Computational Geometry* 28.4 (2002), pp. 511–533. doi 10.1007/s00454-002-2885-2.
- [63] H. Edelsbrunner and E. P. Mücke. “Simulation of Simplicity: A Technique to Cope with Degenerate Cases in Geometric Algorithms.” *ACM Transactions on Graphics* 9.1 (1990), pp. 66–104. doi 10.1145/77635.77639. arXiv: math/9410209 [math.CO].

- [64] V. Eijkhout. *Introduction to High-Performance Scientific Computing*. 2022.  theartofhpc.com.
- [65] M. Evers, M. Herick, V. Molchanov, and L. Linsen. “Coherent Topological Landscapes for Simulation Ensembles.” *VISIGRAPP 2020: Computer Vision, Imaging and Computer Graphics Theory and Applications*. Ed. by K. Bouatouch, A. A. de Sousa, M. Chessa, A. Paljic, A. Kerren, C. Hurter, G. M. Farinella, P. Radeva, and J. Braz. Vol. 1474. Springer International Publishing, 2022, pp. 223–237.  10.1007/978-3-030-94893-1_10.
- [66] X. Feng, Y. Deng, and H. W. J. Blöte. “Percolation transitions in two dimensions.” *Physical Review E* 78.3 (2008), p. 031136.  10.1103/PhysRevE.78.031136.
- [67] C. Flamm, I. L. Hofacker, P. F. Stadler, and M. T. Wolfinger. “Barrier Trees of Degenerate Landscapes.” *Zeitschrift für Physikalische Chemie International journal of research in physical chemistry and chemical physics* 216.2/2002 (2002), p. 155.  10.1524/zpch.2002.216.2.155.
- [68] M. Franke, H. Martin, S. Koch, and K. Kurzhals. “Visual Analysis of Spatio-temporal Phenomena with 1D Projections.” *Computer Graphics Forum* 40.3 (2021): *Proceedings EuroVis*, pp. 335–347.  10.1111/cgfm.14311. /UniStuttgart-VISUS/spatiotemporal1d.
- [69] A. Friederici, M. Atzori, R. Vinuesa, P. Schlatter, and T. Weinkauff. “An efficient algorithm for percolation analysis and its application to turbulent duct flow.” *Euromech Colloquium 598: Coherent structures in wall-bounded turbulence*. 2018.
- [70] A. Friederici*, **W. Köpp***, M. Atzori, R. Vinuesa, P. Schlatter, and T. Weinkauff. “Distributed Percolation Analysis for Turbulent Flows.” *2019 IEEE 9th Symposium on Large Data Analysis and Visualization (LDAV)* (Vancouver, Canada, Oct. 21, 2019). IEEE, 2019, pp. 42–51.  10.1109/LDAV48142.2019.8944383. DiVA: diva2 : 1410816. /KTHVisualization/percMPI.
- [71] W. von Funck, T. Weinkauff, H. Theisel, and H.-P. Seidel. “Smoke Surfaces: An Interactive Flow Visualization Technique Inspired by Real-World Flow Experiments.” *IEEE Transactions on Visualization and Computer Graphics* 14.6 (Nov. 2008): *Proceedings IEEE Visualization 2008*, pp. 1396–1403.  10.1109/TVCG.2008.163.

- [72] E. R. Gansner and S. C. North. “An open graph visualization system and its applications to software engineering.” *Software: Practice & Experience* 30.11 (2000), pp. 1203–1233. [doi](#) 10.1002/1097-024X(200009)30:11<1203::AID-SPE338>3.0.CO;2-N.
- [73] C. Garth, X. Tricoche, and G. Scheuermann. “Tracking of Vector Field Singularities in Unstructured 3D Time-Dependent Datasets.” *Proceedings IEEE Visualization*. 2004, pp. 329–336. [doi](#) 10.1109/visual.2004.107.
- [74] B. Geveci and C. Garth. *Scientific Visualization Contest 2016*. 2016. [🔗](#) www.uni-kl.de/sciviscontest/.
- [75] J. Görtler, C. Schulz, D. Weiskopf, and O. Deussen. “Bubble Treemaps for Uncertainty Visualization.” *IEEE Transactions on Visualization and Computer Graphics* 24.1 (Jan. 2018), pp. 719–728. [doi](#) 10.1109/tvcg.2017.2743959.
- [76] D. Gotz. *Dynamic voronoi treemaps: A visualization technique for time-varying hierarchical data*. Tech. rep. IBM, 2011. [🔗](#) dominoweb.draco.res.ibm.com/82b477237b0fbcd485257868004c1635.html.
- [77] M. Graham and J. Kennedy. “A Survey of Multiple Tree Visualisation.” *Information Visualization* 9.4 (Nov. 2009), pp. 235–252. [doi](#) 10.1057/ivs.2009.29.
- [78] W. Gropp, E. Lusk, and A. Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. 3rd ed. The MIT Press, 2014. ISBN: 9780262256285. [🔗](#) ieeexplore.ieee.org/servlet/opac?bknumber=6267273.
- [79] J. A. Guerra-Gómez, A. Buck-Coleman, M. L. Pack, C. Plaisant, and B. Shneiderman. “TreeVersity: Interactive Visualizations for Comparing Hierarchical Data Sets.” *Transportation Research Record: Journal of the Transportation Research Board* 2392.1 (Jan. 2013), pp. 48–58. [doi](#) 10.3141/2392-06.
- [80] J. Guerra-Gómez, M. L. Pack, C. Plaisant, and B. Shneiderman. “Visualizing Change over Time Using Dynamic Hierarchies: TreeVersity2 and the StemView.” *IEEE Transactions on Visualization and Computer Graphics* 19.12 (Dec. 2013), pp. 2566–2575. [doi](#) 10.1109/TVCG.2013.231.
- [81] C. Gueunet, P. Fortin, and J. Jomier. “Contour forests: Fast multi-threaded augmented contour trees.” *2016 IEEE 6th Symposium on Large Data Analysis and Visualization (LDAV)*. Oct. 2016, pp. 85–92. [doi](#) 10.1109/LDAV.2016.7874333.

- [82] C. Gueunet, P. Fortin, J. Jomier, and J. Tierny. “Task-based Augmented Merge Trees with Fibonacci Heaps.” *2017 IEEE 7th Symposium on Large Data Analysis and Visualization (LDAV)*. Oct. 2017, pp. 6–15. [doi 10.1109/LDAV.2017.8231846](#).
- [83] C. Gueunet, P. Fortin, J. Jomier, and J. Tierny. “Task-Based Augmented Contour Trees with Fibonacci Heaps.” *IEEE Transactions on Parallel and Distributed Systems* 30.8 (Aug. 2019), pp. 1889–1905. [doi 10.1109/TPDS.2019.2898436](#). julien-tierny.github.io/stuff/software/gueunet_tpbs19.tgz.
- [84] T. Günther. *Visualization Data*. 2022. cgl.ethz.ch/research/visualization/data.php.
- [85] D. Guo and M. Gahegan. “Spatial ordering and encoding for geographic data mining and visualization.” *Journal of Intelligent Information Systems* 27.3 (2006), pp. 243–266. [doi 10.1007/s10844-006-9952-8](#).
- [86] H. Guo, D. Lenz, J. Xu, X. Liang, W. He, I. R. Grindeanu, H.-W. Shen, T. Peterka, T. Munson, and I. Foster. “FTK: A Simplicial Spacetime Meshing Framework for Robust and Scalable Feature Tracking.” *IEEE Transactions on Visualization and Computer Graphics* 27.8 (Aug. 2021), pp. 3463–3480. [doi 10.1109/tvcg.2021.3073399](#).
- [87] S. Hahn, J. Bethge, and J. Döllner. “Relative Direction Change - A Topology-based Metric for Layout Stability in Treemaps.” *Proceedings of the 12th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2017) - Volume 3: IVAPP, Porto, Portugal, February 27 - March 1, 2017*. Ed. by L. Linsen, A. Telea, and J. Braz. SciTePress, 2017, pp. 88–95. [doi 10.5220/0006117500880095](#).
- [88] S. Hahn, J. Trümper, D. Moritz, and J. Döllner. “Visualization of Varying Hierarchies by Stable Layout of Voronoi Treemaps.” *Proceedings of the 5th International Conference on Information Visualization Theory and Applications*. Ed. by R. S. Laramée, A. Kerren, and J. Braz. SciTePress, 2014, pp. 50–58. [doi 10.5220/0004686200500058](#).
- [89] T. E. Harris. “A lower bound for the critical probability in a certain percolation process.” *Mathematical Proceedings of the Cambridge Philosophical Society* 56.1 (1960), pp. 13–20. [doi 10.1017/S0305004100034241](#).
- [90] C. Harrison, J. Weiler, R. Bleile, K. Gaither, and H. Childs. “A Distributed-Memory Algorithm for Connected Components Labeling of Simulation Data.” *Topological and Statistical Methods for Complex Data*. Ed. by J.

- Bennett, F. Vivodtzev, and V. Pascucci. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 3–19. [doi](https://doi.org/10.1007/978-3-662-44900-4_1) 10.1007/978-3-662-44900-4_1.
- [91] W. Harvey and Y. Wang. “Topological Landscape Ensembles for Visualization of Scalar-Valued Functions.” *Computer Graphics Forum* 29.3 (Aug. 2010), pp. 993–1002. [doi](https://doi.org/10.1111/j.1467-8659.2009.01706.x) 10.1111/j.1467-8659.2009.01706.x.
- [92] S. Havre, E. G. Hetzler, P. Whitney, and L. T. Nowell. “ThemeRiver: Visualizing Thematic Changes in Large Document Collections.” *IEEE Transactions on Visualization and Computer Graphics* 8.1 (2002), pp. 9–20. [doi](https://doi.org/10.1109/2945.981848) 10.1109/2945.981848.
- [93] L. He, X. Ren, Q. Gao, X. Zhao, B. Yao, and Y. Chao. “The connected-component labeling problem: A review of state-of-the-art algorithms.” *Pattern Recognition* 70 (Oct. 2017), pp. 25–43. [doi](https://doi.org/10.1016/j.patcog.2017.04.018) 10.1016/j.patcog.2017.04.018.
- [94] R. van Hees and J. Hage. “Stable and predictable Voronoi treemaps for software quality monitoring.” *Information and Software Technology* 87 (July 2017), pp. 242–258. [doi](https://doi.org/10.1016/j.infsof.2016.10.003) 10.1016/j.infsof.2016.10.003.
- [95] C. Heine, H. Leitte, M. Hlawitschka, F. Iurich, L. De Floriani, G. Scheuermann, H. Hagen, and C. Garth. “A Survey of Topology-based Methods in Visualization.” *Computer Graphics Forum* 35.3 (June 2016), pp. 643–667. [doi](https://doi.org/10.1111/cgf.12933) 10.1111/cgf.12933.
- [96] C. Heine, G. Scheuermann, C. Flamm, I. L. Hofacker, and P. F. Stadler. “Visualization of Barrier Tree Sequences.” *IEEE Transactions on Visualization and Computer Graphics* 12.5 (Sept. 2006), pp. 781–788. [doi](https://doi.org/10.1109/TVCG.2006.196) 10.1109/TVCG.2006.196.
- [97] C. Heine, G. Scheuermann, C. Flamm, I. L. Hofacker, and P. F. Stadler. “Visualization of Barrier Tree Sequences Revisited.” In: *Mathematics and Visualization*. Springer Berlin Heidelberg, 2008, pp. 275–290. [doi](https://doi.org/10.1007/978-3-540-72630-2_16) 10.1007/978-3-540-72630-2_16.
- [98] C. Heine, D. Schneider, H. A. Carr, and G. Scheuermann. “Drawing Contour Trees in the Plane.” *IEEE Transactions on Visualization and Computer Graphics* 17.11 (2011), pp. 1599–1611. [doi](https://doi.org/10.1109/TVCG.2010.270) 10.1109/TVCG.2010.270.
- [99] M. Herick, V. Molchanov, and L. Linsen. “Temporally Coherent Topological Landscapes for Time-varying Scalar Fields.” *Proceedings of the 15th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications - IVAPP*, 2020, pp. 54–61. [doi](https://doi.org/10.5220/0008956300540061) 10.5220/0008956300540061.

- [100] H. Hersbach, B. Bell, P. Berrisford, G. Biavati, A. Horányi, J. Mu noz Sabater, J. Nicolas, C. Peubey, R. Radu, I. Rozum, D. Schepers, A. Simmons, C. Soci, D. Dee, and J.-N. Thépaut. *ERA5 hourly data on single levels from 1979 to present*. 2018. [doi](https://doi.org/10.24381/cds.adbb2d47) 10 . 24381 / cds . adbb2d47. (Visited on 03/10/2022).
- [101] D. Hilbert. “Ueber die stetige Abbildung einer Line auf ein Flächenstück.” *Mathematische Annalen* 38.3 (1891), pp. 459–460. [doi](https://doi.org/10.1007/BF01199431) 10 . 1007 / BF01199431.
- [102] J. Hoshen and R. Kopelman. “Percolation and cluster distribution. I. Cluster multiple labeling technique and critical concentration algorithm.” *Physical Review B* 14 (8 Oct. 1976), pp. 3438–3445. [doi](https://doi.org/10.1103/PhysRevB.14.3438) 10 . 1103 / PhysRevB . 14 . 3438.
- [103] I. Hotz, R. Bujack, C. Garth, and B. Wang. “Mathematical Foundations in Visualization.” In: *Foundations of Data Visualization*. Ed. by M. Chen, H. Hauser, P. Rheingans, and G. Scheuermann. Cham: Springer International Publishing, 2020, pp. 87–119. ISBN: 978-3-030-34444-3. [doi](https://doi.org/10.1007/978-3-030-34444-3_5) 10 . 1007 / 978-3-030-34444-3_5.
- [104] X. Huang, P. Klacansky, S. Petruzza, A. Gyulassy, P.-T. Bremer, and V. Pascucci. “Distributed Merge Forest: A New Fast and Scalable Approach for Topological Analysis at Scale.” *Proceedings of the ACM International Conference on Supercomputing*. ICS ’21. Virtual Event, USA: Association for Computing Machinery, 2021, pp. 367–377. [doi](https://doi.org/10.1145/3447818.3460358) 10 . 1145 / 3447818 . 3460358.
- [105] T. Itoh and K. Koyamada. “Isosurface Generation by Using Extrema Graphs.” *Proceedings of the Conference on Visualization '94*. VIS '94. Washington, DC, USA: IEEE Computer Society Press, 1994, pp. 77–83. [doi](https://doi.org/10.5555/951087.951105) 10 . 5555 / 951087 . 951105.
- [106] G. Ji and H.-W. Shen. “Efficient Isosurface Tracking Using Precomputed Correspondence Table.” *Eurographics / IEEE VGTC Symposium on Visualization*. The Eurographics Association, 2004. [doi](https://doi.org/10.2312/VISSYM/VISSYM04/283-292) 10 . 2312 / VISSYM / VISSYM04 / 283-292.
- [107] G. Ji and H.-W. Shen. “Feature tracking using earth mover’s distance and global optimization.” *Pacific Graphics*. Vol. 2. 2006.
- [108] G. Ji, H.-W. Shen, and R. Wenger. “Volume tracking using higher dimensional isosurfacing.” *IEEE Visualization, 2003. VIS 2003*. IEEE, 2003. [doi](https://doi.org/10.1109/visual.2003.1250374) 10 . 1109 / visual . 2003 . 1250374.

- [109] B. Johnson and B. Shneiderman. “Tree maps: A Space-Filling Approach to the Visualization of Hierarchical Information Structures.” *Proceedings IEEE Visualization*. Ed. by G. M. Nielson and L. J. Rosenblum. IEEE Computer Society Press, 1991, pp. 284–291.  10.1109/VISUAL.1991.175815.
- [110] E. Jones, T. Oliphant, P. Peterson, et al. *SciPy: Open source scientific tools for Python*. [Online]. 2001.  www.scipy.org/.
- [111] D. Jönsson, P. Steneteg, E. Sundén, R. Englund, S. Kottraval, M. Falk, A. Ynnerman, I. Hotz, and T. Ropinski. “Inwiwo — A Visualization System with Usage Abstraction Levels.” *IEEE Transactions on Visualization and Computer Graphics* 26.11 (2020), pp. 3241–3254.  10.1109/TVCG.2019.2920639.
- [112] A. Kamakshidasan and V. Natarajan. “Mergemaps: Treemaps for Scientific Data.” *Topological Methods in Data Analysis and Visualization VI*. Mathematics and Visualization. Springer International Publishing, 2021, pp. 19–38.  10.1007/978-3-030-83500-2_2.
- [113] H. Kesten. “The critical probability of bond percolation on the square lattice equals $\frac{1}{2}$.” *Communications in Mathematical Physics* 74.1 (1980), pp. 41–59.  10.1007/BF01197577.
- [114] P. Klacansky. *Open SciVis Datasets*. Dec. 2017.  klacansky.com/open-sci-vis-datasets/.
- [115] P. Klacansky, A. Gyulassy, P.-T. Bremer, and V. Pascucci. “Toward Localized Topological Data Structures: Querying the Forest for the Tree.” *IEEE Transactions on Visualization and Computer Graphics* 26.1 (Aug. 2019), pp. 173–183.  10.1109/TVCG.2019.2934257.
- [116] J. Klemelä. “Visualization of Multivariate Density Estimates With Level Set Trees.” *Journal of Computational and Graphical Statistics* 13.3 (Sept. 2004), pp. 599–620.  10.1198/106186004x2642.
- [117] J. Klemelä. “Visualization of Multivariate Density Estimates With Shape Trees.” *Journal of Computational and Graphical Statistics* 15.2 (June 2006), pp. 372–397.  10.1198/106186006x113007.
- [118] **W. Köpp**, M. Atzori, M. Rezaei, N. Jansson, R. Vinuesa, E. Laure, P. Schlatter, and T. Weinkauff. *Effects of Blowing and Suction on the Turbulent Flow around an Airfoil*. Presented at the Gallery of Fluid Motion exhibit at the 73rd Annual Meeting of the American Physical Society’s Division of Fluid Dynamics and in adapted form at Visualization Showcase of the ACM Practice & Experience in Advanced Research Computing (PEARC)

- Conference Series 2021. Nov. 2020. [doi 10.1103/APS.DFD.2020.GFM.V0058](https://doi.org/10.1103/APS.DFD.2020.GFM.V0058).
- [119] **W. Köpp***, A. Friederici*, M. Atzori, R. Vinuesa, P. Schlatter, and T. Weinkauff. “Notes on Percolation Analysis of Sampled Scalar Fields.” *Topological Methods in Data Analysis and Visualization VI. Theory, Applications, and Software*. Topology-Based Methods in Visualization (TopoInVis). Software and Applications (Norrköping, Sweden, June 17–19, 2019). Ed. by I. Hotz, T. B. Masood, F. Sadlo, and J. Tierny. Mathematics and Visualization. Springer International Publishing, 2021, pp. 39–54. [doi 10.1007/978-3-030-83500-2_3](https://doi.org/10.1007/978-3-030-83500-2_3). DiVA: diva2 : 1660119. [KTHVisualization/percAnalysis](https://kthvisualization.github.io/percAnalysis).
- [120] **W. Köpp** and T. Weinkauff. “Temporal Merge Tree Maps: A Topology-Based Static Visualization for Temporal Scalar Data.” *IEEE Transactions on Visualization and Computer Graphics* 29.1 (2022): *Proceedings IEEE VIS*. [doi 10.1109/TVCG.2022.3209387](https://doi.org/10.1109/TVCG.2022.3209387). DiVA: diva2 : 1707137. [Wiebke/TemporalMergeTreeMaps](https://wiebke.github.io/TemporalMergeTreeMaps).
- [121] **W. Köpp** and T. Weinkauff. “Temporal Treemaps: Static Visualization of Evolving Trees.” *IEEE Transactions on Visualization and Computer Graphics* 25.1, 8443124 (2019): *Proceedings IEEE VIS*, pp. 534–543. [doi 10.1109/TVCG.2018.2865265](https://doi.org/10.1109/TVCG.2018.2865265). DiVA: diva2 : 1273957. [Wiebke/TemporalTreeMaps](https://wiebke.github.io/TemporalTreeMaps).
- [122] M. Kraus. “Visualizing Contour Trees within Histograms.” *Computer Graphics and Imaging*. ACTAPRESS, 2010. [doi 10.2316/p.2010.679-010](https://doi.org/10.2316/p.2010.679-010).
- [123] M. van Kreveld, R. van Oostrum, C. Bajaj, V. Pascucci, and D. R. Schikore. “Contour Trees and Small Seed Sets for Isosurface Transversal.” *Proceedings of the 13th ACM Annual Symposium on Computational Geometry (SoCG)*. ACM Press, 1997, pp. 212–220. [doi 10.1145/262839.269238](https://doi.org/10.1145/262839.269238).
- [124] J. B. Kruskal and J. M. Landwehr. “Icicle Plots: Better Displays for Hierarchical Clustering.” *The American Statistician* 37.2 (May 1983), pp. 162–168. [doi 10.1080/00031305.1983.10482733](https://doi.org/10.1080/00031305.1983.10482733).
- [125] A. G. Landge, V. Pascucci, A. Gyulassy, J. C. Bennett, H. Kolla, J. Chen, and P. Bremer. “In-Situ Feature Extraction of Large Scale Combustion Simulations Using Segmented Merge Trees.” *SC ’14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Nov. 2014, pp. 1020–1031. [doi 10.1109/SC.2014.88](https://doi.org/10.1109/SC.2014.88).

- [126] D. Laney, P.-T. Bremer, A. Mascarenhas, P. Miller, and V. Pascucci. “Understanding the Structure of the Turbulent Mixing Layer in Hydrodynamic Instabilities.” *IEEE Transactions on Visualization and Computer Graphics* 12.5 (Sept. 2006): *Proceedings IEEE Visualization*, pp. 1053–1060. doi 10.1109/TVCG.2006.186.
- [127] G. Li, Y. Zhang, Y. Dong, J. Liang, J. Zhang, J. Wang, M. J. McGuffin, and X. Yuan. “BarcodeTree: Scalable Comparison of Multiple Hierarchies.” *IEEE Transactions on Visualization and Computer Graphics* 26.1 (2020), pp. 1022–1032. doi 10.1109/TVCG.2019.2934535.
- [128] M. Lima. *The Book of Trees: Visualizing Branches of Knowledge*. 2014, p. 208. ISBN: 1616892188.
- [129] Y. Liu, J. Li, S. Sun, and B. Yu. “Advances in Gaussian random field generation: a review.” *Computational Geosciences* 23.5 (Aug. 2019), pp. 1011–1047. doi 10.1007/s10596-019-09867-y.
- [130] A.-P. Lohfink, F. Gartzky, F. Wetzels, L. Vollmer, and C. Garth. “Time-Varying Fuzzy Contour Trees.” *2021 IEEE Visualization Conference (VIS)*. 2021, pp. 86–90. doi 10.1109/VIS49827.2021.9623286. arXiv: 2107.12682 [cs.HC].
- [131] A.-P. Lohfink, F. Wetzels, J. Lukasczyk, G. H. Weber, and C. Garth. “Fuzzy Contour Trees: Alignment and Joint Layout of Multiple Contour Trees.” *Computer Graphics Forum* 39.3 (2020): *Proceedings EuroVis*, pp. 343–355. doi 10.1111/cgf.13985.
- [132] W. E. Lorensen and H. E. Cline. “Marching Cubes: a High Resolution 3D Surface Reconstruction Algorithm.” *Computer Graphics* 21 (1987), pp. 163–169. doi 10.1145/37402.37422.
- [133] A. Lozano-Durán, O. Flores, and J. Jiménez. “The three-dimensional structure of momentum transfer in turbulent channels.” *Journal of Fluid Mechanics* 694 (2012), pp. 100–130. doi 10.1017/jfm.2011.524.
- [134] J. Lukasczyk, J. Beran, W. Engelke, M. Falk, A. Friederici, C. Garth, L. Hofmann, I. Hotz, P. Hristov, **W. Köpp**, T. B. Masood, M. Olejniczak, P. Rosen, J.-T. Sohns, T. Weinkauff, K. Werner, and J. Tierny. “Report of the TopoInVis TTK Hackathon: Experiences, Lessons Learned, and Perspectives.” *Topological Methods in Data Analysis and Visualization VI. Theory, Applications, and Software*. Topology-Based Methods in Visualization (TopoInVis). Software and Applications (Norrköping, Sweden, June 17–19, 2019). Ed. by I. Hotz, T. B. Masood, F. Sadlo, and J. Tierny. Mathematics and Visualization. Springer International Publishing, 2021, pp. 359–373. doi 10.1007/978-3-030-83500-2_18.

- [135] J. Lukasczyk, C. Garth, G. H. Weber, T. Biedert, R. Maciejewski, and H. Leitte. “Dynamic Nested Tracking Graphs.” *IEEE Transactions on Visualization and Computer Graphics* 26.1 (2020): *Proceedings IEEE VIS*, pp. 249–258. [doi](#) 10.1109/TVCG.2019.2934368.
- [136] J. Lukasczyk, G. Weber, R. Maciejewski, C. Garth, and H. Leitte. “Nested Tracking Graphs.” *Computer Graphics Forum* 36.3 (2017): *Proceedings EuroVis*, pp. 643–667. [doi](#) 10.1111/cgf.13164. [www.jluk.de/blog/NestedTrackingGraphs/](#).
- [137] J. Ma, J. Chen, and C. Yang. “Using optimized gaussian mixture model rules and global tracking graph for feature extraction and tracking in time-varying data.” *The Visual Computer* (Mar. 2022). [doi](#) 10.1007/s00371-022-02451-z.
- [138] S. Maadasamy, H. Doraiswamy, and V. Natarajan. “A hybrid parallel algorithm for computing and tracking level set topology.” *19th International Conference on High Performance Computing*. Dec. 2012, pp. 1–10. [doi](#) 10.1109/HiPC.2012.6507496.
- [139] F. Manne and M. M. A. Patwary. “A Scalable Parallel Union-Find Algorithm for Distributed Memory Computers.” *Parallel Processing and Applied Mathematics*. Ed. by R. Wyrzykowski, J. Dongarra, K. Karczewski, and J. Wasniewski. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 186–195. [doi](#) 10.1007/978-3-642-14390-8_20.
- [140] K. Misue, P. Eades, W. Lai, and K. Sugiyama. “Layout Adjustment and the Mental Map.” *Journal of Visual Languages & Computing* 6.2 (June 1995), pp. 183–210. [doi](#) 10.1006/jvlc.1995.1010.
- [141] F. Moisy and J. Jiménez. “Geometry and clustering of intense structures in isotropic turbulence.” *Journal of Fluid Mechanics* 513 (2004), pp. 111–133. [doi](#) 10.1017/S0022112004009802.
- [142] D. Morozov, T. Peterka, H. Guo, M. Raj, J. Xu, and H.-W. Shen. “IExchange: Asynchronous Communication and Termination Detection for Iterative Algorithms.” *2021 IEEE 11th Symposium on Large Data Analysis and Visualization (LDAV)*. 2021, pp. 12–21. [doi](#) 10.1109/LDAV53230.2021.00009.
- [143] D. Morozov and G. Weber. “Distributed Merge Trees.” *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. PPOPP ’13. Shenzhen, China: ACM, 2013, pp. 93–102. [doi](#) 10.1145/2442516.2442526.

- [144] D. Morozov and G. H. Weber. “Distributed Contour Trees.” *Topological Methods in Data Analysis and Visualization III*. Ed. by P.-T. Bremer, I. Hotz, V. Pascucci, and R. Peikert. Springer International Publishing, 2014, pp. 89–102. [doi](https://doi.org/10.1007/978-3-319-04099-8_6)10.1007/978-3-319-04099-8_6.
- [145] C. Muelder and K. L. Ma. “Interactive feature extraction and tracking by utilizing region coherency.” *IEEE Pacific Visualization Symposium*. IEEE, Apr. 2009, pp. 17–24. [doi](https://doi.org/10.1109/PACIFICVIS.2009.4906833)10.1109/PACIFICVIS.2009.4906833.
- [146] National Information Standards Organization. *CRedit, Contributor Roles Taxonomy*. Tech. rep. ANSI/NISO Z39.104-2022. American National Standards Institute, 2022. [doi](https://doi.org/10.3789/ansi.niso.z39.104-2022)10.3789/ansi.niso.z39.104-2022.
- [147] U. Neu, M. G. Akperov, N. Bellenbaum, R. Benestad, R. Blender, R. Caballero, A. Cocozza, H. F. Dacre, Y. Feng, K. Fraedrich, J. Grieger, S. Gulev, J. Hanley, T. Hewson, M. Inatsu, K. Keay, S. F. Kew, I. Kindem, G. C. Leckebusch, M. L. R. Liberato, P. Lionello, I. I. Mokhov, J. G. Pinto, C. C. Raible, M. Reale, I. Rudeva, M. Schuster, I. Simmonds, M. Sinclair, M. Sprenger, N. D. Tilinina, I. F. Trigo, S. Ulbrich, U. Ulbrich, X. L. Wang, and H. Wernli. “IMILAST: A Community Effort to Intercompare Extratropical Cyclone Detection and Tracking Algorithms.” *Bulletin of the American Meteorological Society* 94.4 (Apr. 2013), pp. 529–547. [doi](https://doi.org/10.1175/bams-d-11-00154.1)10.1175/bams-d-11-00154.1.
- [148] P. Neumann, S. Schlechtweg, and M. S. T. Carpendale. “ArcTrees: Visualizing Relations in Hierarchical Data.” *Proceedings Joint Eurographics and IEEE VGTC Symposium on Visualization*. Eurographics Association, 2005, pp. 53–60. [doi](https://doi.org/10.2312/VisSym/EuroVis05/053-060)10.2312/VisSym/EuroVis05/053-060.
- [149] M. E. J. Newman and R. M. Ziff. “Efficient Monte Carlo Algorithm and High-Precision Results for Percolation.” *Physical Review Letters* 85 (19 Nov. 2000), pp. 4104–4107. [doi](https://doi.org/10.1103/PhysRevLett.85.4104)10.1103/PhysRevLett.85.4104.
- [150] G. M. Nielson and B. Hamann. “The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes.” *Proceedings of the 2nd Conference on Visualization '91. VIS '91*. San Diego, California: IEEE Computer Society Press, 1991, pp. 83–91.
- [151] A. Nigmatov and D. Morozov. “Local-global merge tree computation with local exchanges.” *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, Nov. 2019. [doi](https://doi.org/10.1145/3295500.3356188)10.1145/3295500.3356188.

- [152] P. Oesterling, C. Heine, G. H. Weber, D. Morozov, and G. Scheuermann. “Computing and Visualizing Time-Varying Merge Trees for High-Dimensional Data.” *Topological Methods in Data Analysis and Visualization IV*. Ed. by H. Carr, C. Garth, and T. Weinkauff. Springer International Publishing, 2017, pp. 87–101. [doi](#) 10.1007/978-3-319-44684-4_5.
- [153] P. Oesterling, C. Heine, G. H. Weber, and G. Scheuermann. “Visualizing nD Point Clouds as Topological Landscape Profiles to Guide Local Data Analysis.” *IEEE Transactions on Visualization and Computer Graphics* 19.3 (2013), pp. 514–526. [doi](#) 10.1109/TVCG.2012.120.
- [154] A. Pandey, U. H. Syeda, C. Shah, J. A. Guerra-Gomez, and M. Borkin. “A State-of-the-Art Survey of Tasks for Tree Design and Evaluation with a Curated Task Dataset.” *IEEE Transactions on Visualization and Computer Graphics* (Mar. 2021), p. 1. [doi](#) 10.1109/TVCG.2021.3064037.
- [155] V. Pascucci and K. Cole-McLaughlin. “Efficient computation of the topology of level sets.” *IEEE Visualization, 2002. VIS 2002*. IEEE, 2002. [doi](#) 10.1109/visual.2002.1183774.
- [156] V. Pascucci and K. Cole-McLaughlin. “Parallel Computation of the Topology of Level Sets.” *Algorithmica* 38.1 (2004), pp. 249–268. [doi](#) 10.1007/s00453-003-1052-3.
- [157] V. Pascucci, K. Cole-McLaughlin, and G. Scorzelli. “The Toporrery: Computation and Presentation of Multi-resolution Topology.” In: *Mathematical Foundations of Scientific Visualization, Computer Graphics, and Massive Data Exploration*. Ed. by T. Möller, B. Hamann, and R. D. Russell. Springer Berlin Heidelberg, 2009, pp. 19–40. ISBN: 978-3-540-49926-8. [doi](#) 10.1007/b106657_2.
- [158] J. M. Patchett, B. Nouanesengsy, J. Pouderoux, J. Ahrens, and H. Hagen. “Parallel multi-layer ghost cell generation for distributed unstructured grids.” *IEEE 7th Symposium on Large Data Analysis and Visualization (LDAV)*. Oct. 2017, pp. 84–91. [doi](#) 10.1109/LDAV.2017.8231854.
- [159] G. Peano. “Sur une courbe, qui remplit toute une aire plane.” *Mathematische Annalen* 36.1 (1890), pp. 157–160. [doi](#) 10.1007/BF01199438.
- [160] R. Peikert and M. Roth. “The parallel vectors operator - a vector field visualization primitive.” *Proceedings IEEE Visualization 99*. 1999, pp. 263–270. [doi](#) 10.1109/VISUAL.1999.809896.

- [161] M. Pont, J. Vidal, J. Delon, and J. Tierny. “Wasserstein Distances, Geodesics and Barycenters of Merge Trees.” *IEEE Transactions on Visualization and Computer Graphics* 28.1 (Jan. 2022), pp. 291–301. doi 10.1109/tvcg.2021.3114839.
- [162] S. Popinet. “Free Computational Fluid Dynamics.” *ClusterWorld* 2.6 (June 2004).  gfs.sf.net/.
- [163] S. Popinet, M. Smith, and C. Stevens. “Experimental and Numerical Study of the Turbulence Characteristics of Airflow around a Research Vessel.” *Journal of Atmospheric and Oceanic Technology* 21.10 (Oct. 2004), pp. 1575–1589. doi 10.1175/1520-0426(2004)021<1575:eansot>2.0.co;2.
- [164] F. H. Post, B. Vrolijk, H. Hauser, R. S. Laramée, and H. Doleisch. “The State of the Art in Flow Visualisation: Feature Extraction and Tracking.” *Computer Graphics Forum* 22.4 (2003), pp. 775–792. doi 10.1111/j.1467-8659.2003.00723.x.
- [165] B. Raichel and C. Seshadhri. “Avoiding the Global Sort: A Faster Contour Tree Algorithm.” *Discrete & Computational Geometry* 58.4 (June 2017), pp. 946–985. doi 10.1007/s00454-017-9901-z.
- [166] F. Reinders, F. H. Post, and H. J. W. Spoelder. “Attribute-Based Feature Tracking.” *Data Visualization 1999. Proceedings VisSym 99*. Springer Vienna, 1999, pp. 63–72. doi 10.1007/978-3-7091-6803-5_7.
- [167] F. Reinders, F. H. Post, and H. J. W. Spoelder. “Visualization of time-dependent data using feature tracking and event detection.” *The Visual Computer* 17.1 (2001), pp. 55–71. doi 10.1007/p100013399.
- [168] E. M. Reingold and J. S. Tilford. “Tidier Drawings of Trees.” *IEEE Transactions on Software Engineering* SE-7.2 (Mar. 1981), pp. 223–228. doi 10.1109/tse.1981.234519.
- [169] J. Reininghaus, J. Kasten, T. Weinkauff, and I. Hotz. “Efficient Computation of Combinatorial Feature Flow Fields.” *IEEE Transactions on Visualization and Computer Graphics* 18.9 (Sept. 2012), pp. 1563–1573. doi 10.1109/TVCG.2011.269.
- [170] B. Rieck, F. Sadlo, and H. Leitte. “Hierarchies and Ranks for Persistence Pairs.” *Topological Methods in Data Analysis and Visualization V*. Springer International Publishing, 2020, pp. 3–17. doi 10.1007/978-3-030-43036-8_1.

- [171] M. D. Rintoul and S. Torquato. “Precise determination of the critical threshold and exponents in a three-dimensional continuum percolation model.” *Journal of Physics A: Mathematical and General* 30.16 (Aug. 1997), pp. L585–L592. [doi](#) 10.1088/0305-4470/30/16/005.
- [172] G. Robertson, J. Mackinlay, and S. Card. “Cone Trees: Animated 3D Visualizations of Hierarchical Information.” *Proceedings ACM CHI International Conference of Human Factors in Computing (CHI '91)*. 1991, pp. 189–194. [doi](#) 10.1145/108844.108883.
- [173] P. Rosen, J. Tu, and L. A. Piegl. “A hybrid solution to parallel calculation of augmented join trees of scalar fields in any dimension.” *Computer-Aided Design and Applications* 15.4 (Jan. 2018), pp. 610–618. [doi](#) 10.1080/16864360.2017.1419648.
- [174] M. Sahimi. *Applications Of Percolation Theory*. CRC Press, 2014. [doi](#) 10.1201/9781482272444.
- [175] H. Saikia, H.-P. Seidel, and T. Weinkauff. “Extended Branch Decomposition Graphs: Structural Comparison of Scalar Data.” *Computer Graphics Forum* 33.3 (June 2014): *Proceedings EuroVis*, pp. 41–50. [doi](#) 10.1111/cgf.12360.
- [176] H. Saikia and T. Weinkauff. “Fast Topology-based Feature Tracking using a Directed Acyclic Graph.” *Topological Methods in Data Analysis and Visualization V* (Tokyo, Japan). Springer International Publishing, 2020, pp. 155–169. [doi](#) 10.1007/978-3-030-43036-8_10.
- [177] H. Saikia and T. Weinkauff. “Global Feature Tracking and Similarity Estimation in Time-Dependent Scalar Fields.” *Computer Graphics Forum* 35.3 (June 2016): *Proceedings EuroVis*, pp. 1–11. [doi](#) 10.1111/cgf.13163.
- [178] J. W. Sammon. “A nonlinear mapping for data structure analysis.” *IEEE Transactions on Computers* 100.5 (1969), pp. 401–409. [doi](#) 10.1109/T-C.1969.222678.
- [179] R. Samtaney, D. Silver, N. Zabusky, and J. Cao. “Visualizing features and tracking their evolution.” *IEEE Computer* 27.7 (July 1994), pp. 20–27. [doi](#) 10.1109/2.299407.
- [180] W. Scheibel, D. Limberger, and J. Döllner. “Survey of treemap layout algorithms.” In: *VINCI '20: Proceedings of the 13th International Symposium on Visual Information Communication and Interaction*. New York, NY, USA: Association for Computing Machinery, Dec. 2020, pp. 1–9. [doi](#) 10.1145/3430036.3430041.

- [181] W. Scheibel, C. Weyand, and J. Döllner. “EvoCells - A Treemap Layout Algorithm for Evolving Tree Data.” *Proceedings of the 13th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. SCITEPRESS - Science and Technology Publications, 2018. doi 10.5220/0006617102730280.
- [182] A. Schnorr, D. N. Helmrich, H. Childs, T. W. Kuhlen, and B. Hentschel. “Feature Tracking Utilizing a Maximum-Weight Independent Set Problem.” *2019 IEEE 9th Symposium on Large Data Analysis and Visualization (LDAV)*. IEEE, Oct. 2019. doi 10.1109/ldav48142.2019.8944363.
- [183] A. Schnorr, D. N. Helmrich, D. Denker, T. W. Kuhlen, and B. Hentschel. “Feature Tracking by Two-Step Optimization.” *IEEE Transactions on Visualization and Computer Graphics* 26.06 (2020), pp. 2219–2233. doi 10.1109/TVCG.2018.2883630.
- [184] H.-J. Schulz. “Treevis.net: A Tree Visualization Reference.” *IEEE Computer Graphics and Applications* 31.6 (Nov. 2011), pp. 11–15. doi 10.1109/MCG.2011.103. treevis.net/.
- [185] B. Shneiderman. “Tree Visualization with Tree-Maps: 2-d Space-Filling Approach.” *ACM Transactions on Graphics* 11.1 (Jan. 1992), pp. 92–99. doi 10.1145/102377.115768.
- [186] B. Shneiderman and M. Wattenberg. “Ordered Treemap Layouts.” *Proceedings IEEE Symposium on Information Visualization*. Ed. by K. Andrews, S. F. Roth, and P. C. Wong. IEEE. IEEE Computer Society, 2001, pp. 73–78. doi 10.1109/INFVIS.2001.963283.
- [187] D. Silver and X. Wang. “Tracking and visualizing turbulent 3D features.” *IEEE Transactions on Visualization and Computer Graphics* 3.2 (1997), pp. 129–141. doi 10.1109/2945.597796.
- [188] D. Silver and X. Wang. “Tracking Scalar Features in Unstructured Datasets.” *Proceedings Visualization '98*. Ed. by D. S. Ebert, H. E. Rushmeier, and H. Hagen. IEEE Computer Society and ACM, 1998, pp. 79–86. doi 10.1109/VISUAL.1998.745288.
- [189] P. Skraba and B. Wang. “Interpreting Feature Tracking Through the Lens of Robustness.” *Topological Methods in Data Analysis and Visualization III. Theory, Algorithms, and Applications*. Ed. by P.-T. Bremer, I. Hotz, V. Pascucci, and R. Peikert. Springer, 2014, pp. 19–37. doi 10.1007/978-3-319-04099-8_2.

- [190] D. Smirnov and D. Morozov. “Triplet Merge Trees.” *Topological Methods in Data Analysis and Visualization V*. Springer International Publishing, 2020, pp. 19–36. [doi](https://doi.org/10.1007/978-3-030-43036-8_2) 10.1007/978-3-030-43036-8_2.
- [191] B. S. Sohn and C. Bajaj. “Time-varying contour topology.” *IEEE Transactions on Visualization and Computer Graphics* 12.1 (Jan. 2006), pp. 14–25. [doi](https://doi.org/10.1109/TVCG.2006.16) 10.1109/TVCG.2006.16.
- [192] M. Soler, M. Plainchault, B. Conche, and J. Tierny. “Lifted Wasserstein Matcher for Fast and Robust Topology Tracking.” *2018 IEEE 8th Symposium on Large Data Analysis and Visualization (LDAV)*. 2018, pp. 23–33. [doi](https://doi.org/10.1109/LDAV.2018.8739196) 10.1109/LDAV.2018.8739196.
- [193] M. Sondag, W. Meulemans, C. Schulz, K. Verbeek, D. Weiskopf, and B. Speckmann. “Uncertainty Treemaps.” *2020 IEEE Pacific Visualization Symposium (PacificVis)*. IEEE, June 2020. [doi](https://doi.org/10.1109/pacificvis48177.2020.7614) 10.1109/pacificvis48177.2020.7614.
- [194] M. Sondag, B. Speckmann, and K. Verbeek. “Stable Treemaps via Local Moves.” *IEEE Transactions on Visualization and Computer Graphics* 24.1 (2018), pp. 729–738. [doi](https://doi.org/10.1109/TVCG.2017.2745140) 10.1109/TVCG.2017.2745140. [git win.tue.nl/max/IncrementalTreemap](https://git.win.tue.nl/max/IncrementalTreemap).
- [195] R. Sridharamurthy, T. B. Masood, A. Kamakshidasan, and V. Natarajan. “Edit Distance between Merge Trees.” *IEEE Transactions on Visualization and Computer Graphics* 26.3 (Mar. 2020), pp. 1518–1531. [doi](https://doi.org/10.1109/TVCG.2018.2873612) 10.1109/TVCG.2018.2873612.
- [196] R. Sridharamurthy and V. Natarajan. “Comparative Analysis of Merge Trees using Local Tree Edit Distance.” *IEEE Transactions on Visualization and Computer Graphics* (Oct. 2021). [doi](https://doi.org/10.1109/TVCG.2021.3122176) 10.1109/TVCG.2021.3122176.
- [197] D. Stalling and T. Steinke. *Visualization of Vector Fields in Quantum Chemistry*. Tech. rep. SC-96-01. Zuse Institute Berlin, 1996. URN: urn:nbn:de:0297-zib-2124.
- [198] J. Stasko, R. Catrambone, M. Guzdial, and K. McDonald. “An evaluation of space-filling information visualizations for depicting hierarchical structures.” *International Journal of Human-Computer Studies* 53.5 (Nov. 2000), pp. 663–694. [doi](https://doi.org/10.1006/ijhc.2000.0420) 10.1006/ijhc.2000.0420.
- [199] D. Stauffer and A. Aharony. *Introduction To Percolation Theory*. Taylor & Francis, 1994. ISBN: 9781420074796. [doi](https://doi.org/10.1201/9781315274386) 10.1201/9781315274386.

- [200] A. Sud, D. Fisher, and H. Lee. “Fast Dynamic Voronoi Treemaps.” *Proceedings Seventh International Symposium on Voronoi Diagrams in Science and Engineering*. Ed. by M. A. Mostafavi. IEEE Computer Society, 2010, pp. 85–94. [doi](https://doi.org/10.1109/ISVD.2010.16) 10.1109/ISVD.2010.16.
- [201] S. Takahashi, Y. Takeshima, and I. Fujishiro. “Topological volume skeletonization and its application to transfer function design.” *Graphical Models* 66.1 (Jan. 2004), pp. 24–49. [doi](https://doi.org/10.1016/j.gmod.2003.08.002) 10.1016/j.gmod.2003.08.002.
- [202] S. P. Tarasov and M. N. Vyalyi. “Construction of Contour Trees in 3D in $O(n \log n)$ Steps.” *Proceedings of the 14th annual symposium on Computational geometry - SCG '98*. SCG '98. Minneapolis, Minnesota, USA: Association for Computing Machinery, 1998, pp. 68–75. [doi](https://doi.org/10.1145/276884.276892) 10.1145/276884.276892.
- [203] R. E. Tarjan. “Efficiency of a Good But Not Linear Set Union Algorithm.” *Journal of The Association for Computing Machinery* 22.2 (Apr. 1975), pp. 215–225. [doi](https://doi.org/10.1145/321879.321884) 10.1145/321879.321884.
- [204] A. Telea and D. Auber. “Code Flows: Visualizing Structural Evolution of Source Code.” *Computer Graphics Forum* 27.3 (May 2008), pp. 831–838. [doi](https://doi.org/10.1111/j.1467-8659.2008.01214.x) 10.1111/j.1467-8659.2008.01214.x.
- [205] J. M. Teuler and J. C. Gimel. “A direct parallel implementation of the Hoshen–Kopelman algorithm for distributed memory architectures.” *Computer Physics Communications* 130.1 (2000), pp. 118–129. [doi](https://doi.org/10.1016/S0010-4655(00)00046-1) 10.1016/S0010-4655(00)00046-1.
- [206] H. Theisel and H.-P. Seidel. “Feature Flow Fields.” *Proceedings of the Symposium on Data Visualisation 2003 (VisSym '03)*. 2003, pp. 141–148. [doi](https://doi.org/10.5555/769922.769938) 10.5555/769922.769938.
- [207] H. Theisel, J. Sahner, T. Weinkauff, H.-C. Hege, and H.-P. Seidel. “Extraction of Parallel Vector Surfaces in 3D Time-Dependent Fields and Application to Vortex Core Line Tracking.” *Proceedings IEEE Visualization 2005*. 2005, pp. 631–638. [doi](https://doi.org/10.1109/VISUAL.2005.1532851) 10.1109/VISUAL.2005.1532851.
- [208] J. Tierny, G. Favelier, J. A. Levine, C. Gueunet, and M. Michaux. “The Topology ToolKit.” *IEEE Transactions on Visualization and Computer Graphics* 24.1 (2018): *Proceedings IEEE VIS*, pp. 832–842. [doi](https://doi.org/10.1109/TVCG.2017.2743938) 10.1109/TVCG.2017.2743938.
- [209] D. Tiggemann. “Simulation Of Percolation On Massively-Parallel Computers.” *International Journal of Modern Physics C* 12.6 (2001), pp. 871–878. [doi](https://doi.org/10.1142/S012918310100205X) 10.1142/S012918310100205X.

- [210] X. Tricoche, T. Wischgoll, G. Scheuermann, and H. Hagen. “Topology tracking for the visualization of time-dependent two-dimensional flows.” *Computers & Graphics* 26 (2002), pp. 249–257. doi 10.1016/s0097-8493(02)00056-0.
- [211] TTK Topology Toolkit Data Examples. *Two simulation runs of the viscous finger dataset form the 2016 scientific visualization contest. ViscousFingers.cdb*. Ed. by G. A. Berk Geveci Christoph Garth and J. Lukasczyk. 2018. [/topology-tool-kit/ttk-data](https://github.com/topology-tool-kit/ttk-data).
- [212] Y. Tu and H. Shen. “Visualizing Changes of Hierarchical Data using Treemaps.” *IEEE Transactions on Visualization and Computer Graphics* 13.6 (2007), pp. 1286–1293. doi 10.1109/TVCG.2007.70529.
- [213] D. P. Tua, R. Minelli, and M. Lanza. “Voronoi Evolving Treemaps.” *2021 Working Conference on Software Visualization (VISSOFT)*. IEEE, Sept. 2021. doi 10.1109/vissoft52517.2021.00032.
- [214] United Nations. *United Nations Demographic Yearbooks 1984–2016*. New York: United Nations, 2018. unstats.un.org/unsd/demographic-social/products/dyb/.
- [215] United Nations. *World Population Prospects: The 2017 Revision. Total Population - Both Sexes*. Department of Economic and Social Affairs, Population Division. New York, 2017. esa.un.org/unpd/wpp/.
- [216] United Nations. *World Population Prospects: The 2017 Revision, Methodology of the United Nations Population Estimates and Projections*. Working Paper No. ESA/P/WP.250. New York: Department of Economic and Social Affairs, Population Division, 2017. population.un.org/wpp/Publications/.
- [217] C. Vehlow, F. Beck, and D. Weiskopf. “Visualizing Dynamic Hierarchies in Graph Sequences.” *IEEE Transactions on Visualization and Computer Graphics* 22.10 (Oct. 2016), pp. 2343–2357. doi 10.1109/tvcg.2015.2507595.
- [218] E. F. Vernier, J. Comba, and A. C. Telea. “Quantitative Comparison of Dynamic Treemaps for Software Evolution Visualization.” *2018 IEEE Working Conference on Software Visualization (VISSOFT)*. IEEE, Sept. 2018, pp. 96–106. doi 10.1109/VISSOFT.2018.00018.
- [219] E. F. Vernier, J. L. D. Comba, and A. C. Telea. “A Stable Greedy Insertion Treemap Algorithm for Software Evolution Visualization.” *2018 31st SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*. IEEE, Oct. 2018. doi 10.1109/sibgrapi.2018.00027.

- [220] E. F. Vernier*, M. Sondag*, J. Comba, B. Speckmann, A. C. Telea, and K. Verbeek. “Quantitative Comparison of Time-Dependent Treemaps.” *Computer Graphics Forum* 39.3 (2020), pp. 393–404. [doi](#) 10.1111/cgf.13989. arXiv: 1906.06014 [cs.CG].
- [221] R. Vinuesa, P. Schlatter, and H. M. Nagib. “Secondary flow in turbulent ducts with increasing aspect ratio.” *Physical Review Fluids* 3.5 (May 2018), p. 054606. [doi](#) 10.1103/PhysRevFluids.3.054606.
- [222] S. Volke, S. Bin, D. Zeckzer, M. Middendorf, and G. Scheuermann. “Visual Analysis of Discrete Particle Swarm Optimization Using Fitness Landscapes.” In: *Recent Advances in the Theory and Application of Fitness Landscapes*. Springer, 2014, pp. 487–507. [doi](#) 10.1007/978-3-642-41888-4_17.
- [223] S. Volke, M. Middendorf, M. Hlawitschka, J. Kasten, D. Zeckzer, and G. Scheuermann. “dPSO-Vis: Topology-based Visualization of Discrete Particle Swarm Optimization.” *Computer Graphics Forum* 32.3pt3 (June 2013), pp. 351–360. [doi](#) 10.1111/cgf.12122.
- [224] W. Wang, H. Wang, G. Dai, and H. Wang. “Visualization of large hierarchical data by circle packing.” *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Apr. 2006. [doi](#) 10.1145/1124772.1124851.
- [225] Y.-C. Wang, Y. Xing, F. Lin, H.-S. Seah, and J. Zhang. “OST: a heuristic-based orthogonal partitioning algorithm for dynamic hierarchical data visualization.” *Journal of Visualization* 25.4 (Feb. 2022), pp. 875–896. [doi](#) 10.1007/s12650-022-00830-1.
- [226] M. Wattenberg and J. Kriss. “Designing for Social Data Analysis.” *IEEE Transactions on Visualization and Computer Graphics* 12.4 (July 2006), pp. 549–557. [doi](#) 10.1109/TVCG.2006.65.
- [227] G. Weber, P.-T. Bremer, and V. Pascucci. “Topological Landscapes: A Terrain Metaphor for Scientific Data.” *IEEE Transactions on Visualization and Computer Graphics* 13.6 (2007), pp. 1416–1423. [doi](#) 10.1109/TVCG.2007.70601.
- [228] G. H. Weber, P.-T. Bremer, and V. Pascucci. “Topological Cacti: Visualizing Contour-Based Statistics.” *Topological Methods in Data Analysis and Visualization II*. Springer Berlin Heidelberg, Nov. 2011, pp. 63–76. [doi](#) 10.1007/978-3-642-23175-9_5.

- [229] G. H. Weber, S. E. Dillard, H. Carr, V. Pascucci, and B. Hamann. “Topology-Controlled Volume Rendering.” *IEEE Transactions on Visualization and Computer Graphics* 13.2 (Mar. 2007), pp. 330–341. doi 10.1109/TVCG.2007.47.
- [230] T. Weinkauff, H. Theisel, A. V. Gelder, and A. Pang. “Stable Feature Flow Fields.” *IEEE Transactions on Visualization and Computer Graphics* 17.06 (2010), pp. 770–780. doi 10.1109/TVCG.2010.93.
- [231] T. Weinkauff. *3D Flow Around a Confined Square Cylinder*. 2015.  www.csc.kth.se/~weinkauff/notes/squarecylinder.html.
- [232] J. Weissenböck, B. Fröhler, E. Gröller, J. Kastner, and C. Heinzl. “Dynamic Volume Lines: Visual Comparison of 3D Volumes through Space-filling Curves.” *IEEE Transactions on Visualization and Computer Graphics* 25.1 (2019): *Proceedings IEEE VIS*, pp. 1040–1049. doi 10.1109/TVCG.2018.2864510.
- [233] R. Wenger. *Isosurfaces. Geometry, Topology, and Algorithms*. An A K Peters Book. CRC Press LLC, 2013. ISBN: 9781466571020.
- [234] K. Werner and C. Garth. “Alternative Parameters for On-The-Fly Simplification of MergeTrees.” *Eurographics Symposium on Parallel Graphics and Visualization* (2020). doi 10.2312/PGV.20201077.
- [235] K. Werner and C. Garth. “Unordered Task-Parallel Augmented Merge Tree Construction.” *IEEE Transactions on Visualization and Computer Graphics* 27.8 (2021), pp. 3585–3596. doi 10.1109/TVCG.2021.3076875.
- [236] K. Wetzel. *Pebbles – using Circular Treemaps to visualize disk usage*. retrieved 26-APR-2010. 2003.  lip.sourceforge.net/ctreemap.html.
- [237] F. Wetzels, H. Leitte, and C. Garth. “Branch Decomposition-Independent Edit Distances for Merge Trees.” *Computer Graphics Forum* 41.3 (June 2022), pp. 367–378. doi 10.1111/cgf.14547.
- [238] W. Widanagamaachchi, C. Christensen, P.-T. Bremer, and V. Pascucci. “Interactive Exploration of Large-Scale Time-Varying Data Using Dynamic Tracking Graphs.” *IEEE Symposium on Large Data Analysis and Visualization (LDAV)*. Ed. by R. S. Barga, H. Pfister, and D. H. Rogers. IEEE, Oct. 2012, pp. 9–17. doi 10.1109/LDAV.2012.6378962.
- [239] J. J. van Wijk and H. van de Wetering. “Cushion Treemaps: Visualization of Hierarchical Information.” *Proceedings IEEE Symposium on Information Visualization*. IEEE Computer Society, 1999, pp. 73–78. doi 10.1109/INFVIS.1999.801860.

- [240] R. M. Wilson and R. D. Bergeron. “Dynamic hierarchy specification and visualization.” *Proceedings 1999 IEEE Symposium on Information Visualization (InfoVis’99)*. 1999. doi 10.1109/infvis.1999.801859.
- [241] M. Wittenhagen, C. Cherek, and J. Borchers. “Chronicler: Interactive Exploration of Source Code History.” *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. ACM, May 2016, pp. 3522–3532. doi 10.1145/2858036.2858442.
- [242] K. Wu and S. Zhang. “A contour tree based visualization for exploring data with uncertainty.” *International Journal for Uncertainty Quantification* 3.3 (2013), pp. 203–223. doi 10.1615/int.j.uncertaintyquantification.2012003956.
- [243] J. Wulms, J. Buchmüller, W. Meulemans, K. Verbeek, and B. Speckmann. “Stable Visual Summaries for Trajectory Collections.” *2021 IEEE 14th Pacific Visualization Symposium (PacificVis)*. 2021, pp. 61–70. doi 10.1109/PacificVis52677.2021.00016. arXiv: 1912.00719 [cs.HC].
- [244] J. Xu, H. Guo, H.-W. Shen, M. Raj, X. Wang, X. Xu, Z. Wang, and T. Peterka. “Asynchronous and Load-Balanced Union-Find for Distributed and Parallel Scientific Data Visualization and Analysis.” *IEEE Transactions on Visualization and Computer Graphics* 27.6 (2021), pp. 2808–2820. doi 10.1109/TVCG.2021.3074584.
- [245] L. Yan, T. B. Masood, R. Sridharamurthy, F. Rasheed, V. Natarajan, I. Hotz, and B. Wang. “Scalar Field Comparison with Topological Descriptors: Properties and Applications for Scientific Visualization.” *Computer Graphics Forum* 40.3 (June 2021), pp. 599–633. doi 10.1111/cgf.14331.
- [246] L. Yan, Y. Wang, E. Munch, E. Gasparovic, and B. Wang. “A Structural Average of Labeled Merge Trees for Uncertainty Visualization.” *IEEE Transactions on Visualization and Computer Graphics* 26.1 (Jan. 2020), pp. 832–842. doi 10.1109/tvcg.2019.2934242.
- [247] P. K. Yeung, M. P. Clay, S. Hamilton, R. Ravoori, G. Lemson, R. Burns, and C. Meneveau. *Johns Hopkins Turbulence Databases. Forced Isotropic Turbulence Dataset on 4096³ Grid*. Johns Hopkins Turbulence Databases, 2017. doi 10.7281/T1DN4363.  turbulence.pha.jhu.edu.

- [248] P. K. Yeung, D. A. Donzis, and K. R. Sreenivasan. “Dissipation, enstrophy and pressure statistics in turbulence simulations at high Reynolds numbers.” *Journal of Fluid Mechanics* 700 (2012), pp. 5–15. [doi 10.1017/jfm.2012.5](https://doi.org/10.1017/jfm.2012.5).
- [249] F. Yonezawa, S. Sakamoto, and M. Hori. “Percolation in two-dimensional lattices. I. A technique for the estimation of thresholds.” *Physical Review B* 40 (1 July 1989), pp. 636–649. [doi 10.1103/PhysRevB.40.636](https://doi.org/10.1103/PhysRevB.40.636).
- [250] L. Zhou, C. R. Johnson, and D. Weiskopf. “Data-Driven Space-Filling Curves.” *IEEE Transactions on Visualization and Computer Graphics* 27.2 (2020): *Proceedings IEEE VIS*, pp. 1591–1600. [doi 10.1109/TVCG.2020.3030473](https://doi.org/10.1109/TVCG.2020.3030473). [🔗/zhou-l/DataDrivenSpaceFillCurve](https://github.com/zhou-l/DataDrivenSpaceFillCurve).
- [251] R. M. Ziff. “Results for a critical threshold, the correction-to-scaling exponent and susceptibility amplitude ratio for 2d percolation.” *Physics Procedia* 15 (2011). Proceedings of the 24th Workshop on Computer Simulation Studies in Condensed Matter Physics (CSP2011), pp. 106–112. [doi 10.1016/j.phpro.2011.06.009](https://doi.org/10.1016/j.phpro.2011.06.009).
- [252] R. M. Ziff and C. R. Scullard. “Exact bond percolation thresholds in two dimensions.” *Journal of Physics A: Mathematical and General* 39.49 (Nov. 2006), pp. 15083–15090. [doi 10.1088/0305-4470/39/49/003](https://doi.org/10.1088/0305-4470/39/49/003).

Part II

Paper Reprints

